

Leant2: Design for Further Improvements

Integrated expert answers, engineering plan,
and the remaining research agenda

Prepared for Vladimir Reshetnikov

September 22, 2026

Abstract

The original improvement plan began with measured Leant2 behavior at commit `33cec8b`: saturated outcome-category regression suites, thirteen open Church-operation stretch cases, sixteen edge probes, and a workload-specific search profile. Those historical measurements remain identified by revision. This revision integrates nine independently prepared expert-answer packages into one design, rather than retaining nine overlapping answers to the same 31 questions.

The integrated design separates exact-query certification, conservative pruning, bounded coverage, ranking, replay, and executable publication. It retains constructive carrier and motive methods, corrected lower bounds, typed partial-evaluation obligations, recursive realization and local proof interfaces, dependent retrieval, and an explicit boundary for learned proposals. Established answers replace the corresponding expert questions; the active agenda asks narrower questions about the remaining formalization, algorithmic and empirical gaps. A common roadmap and adversarial evaluation plan connect the subsystems. Preserved Python model checks reproduce finite examples and countermodels; they are not Lean proofs or new synthesis results.

Status. This is a design document. Section 1 preserves the original measurements at `33cec8b`, not a fresh measurement of the implementation. The expert packages inspected selected code at `784664f`; their source-level observations are also historical. The integration inventory pins all nine packages to repository revision `5c3a53c`. New validation in this revision covers the preserved finite Python suites and the consolidated documentation. It does not establish a new Lean build, a formalization of the mathematical arguments, or an engine speedup. Literature and version boundaries are recorded in the references and integration ledger.

Contents

I	Where the engine stands	6
1	Evidence	6
1.1	What passes	6
1.2	What does not pass	6
1.3	Where the time goes	7
1.4	What the search is	8
2	Constraints and classification	8
2.1	Constraints that stay	8
2.2	Three classes of work	8
2.3	Work items	9
3	A common semantic boundary for the integrated design	10
3.1	Freeze the problem before proposing a solution	10
3.2	Six guarantees and their separate evidence	10
3.3	Necessary observations, alternatives and refutation	11
3.4	Results must identify what was established	12
3.5	Services with deliberately different authority	12
II	Work with known solutions	13
4	Engineering items	13
4.1	E1. Results as compilable definitions	13
4.2	E2. Frontends	14
4.3	E3. Receipts, replay and resumable search	14
4.4	E4. Substrate cost	15
4.5	E5. Parallel lanes	15
4.6	E6. A decision procedure for propositional targets	15
4.7	E7. A wider proof portfolio	16
4.8	E8. Benchmarks beyond Leant	16
5	Adaptation items	16
5.1	A1. Conditionals and guard abduction	17
5.2	A2. Recursion on natural numbers, literals and arithmetic	17
5.3	A3. Normal forms and symmetry reduction	18
5.4	A4. Parametricity-directed test inputs	18
5.5	A5. Rule indexing	19

III	Integrated answers and remaining problems	19
6	R1. Search control with coupled goals under a wall-clock budget	19
6.1	The problem and the corrected design	19
6.2	Conditional factorization and contextual reuse	20
6.3	Admissible bounds for joint completion	21
6.4	Learning priorities without changing the objective	22
6.5	Deterministic prefixes and real deadlines	23
6.6	Implementation sequence and evidence still needed	24
6.7	New questions for experts	24
7	R2. Carriers, accumulators and auxiliary state	25
7.1	The problem is a representation together with its operations	25
7.2	Minimal semantic state and what samples cannot identify	26
7.3	Two constructive bounded problems	26
7.4	Sound sample lower bounds and their counterexamples	27
7.5	Polymorphic carriers: shapes, positions and available observations	29
7.6	Constructive fusion, feature discovery and closure	30
7.7	Worked representations and scheme-dependent invariants	31
7.8	Motive enumeration and a shared strengthening service	32
7.9	Evaluation and new expert questions	33
8	R3. Evaluation of programs with holes	34
8.1	The problem and revised boundary	34
8.2	Typed closures and relational unevaluation	34
8.3	Assignment-driven normalization and rollback	35
8.4	Checked evaluation before a new normalizer	36
8.5	Angelic calls and the bounded guarantee	38
8.6	Validation and new questions	39
9	R4. Indexed families and the motive problem	39
9.1	The problem and corrected starting point	39
9.2	A finite motive language with explicit generalization	40
9.3	Typed transports and restricted canonicalization	41
9.4	Demand-driven unfolding and index narrowing	42
9.5	A version-pinned native motive adapter	43
9.6	New questions after the four original answers	44
10	R5. General recursion from incomplete examples	44
10.1	The missing capability and revised comparisons	44
10.2	Typed capabilities and explicit realization	45
10.3	Interpreter agreement is generally propositional	46

10.4 Publishing an executable definition	47
10.5 Top-down angels, sharing, and helper invariants	47
10.6 A staged recursion portfolio and its limits	48
10.7 Coverage experiments and new questions	49
11 R6. Contracts beyond decidable observations	50
11.1 The problem and the implementation boundary	50
11.2 Dependency-sensitive program and proof construction	50
11.3 A certificate for a recognizable induction class	51
11.4 A bounded native induction service	52
11.5 Paying proof debt without starvation	53
11.6 Acceptance measurements and remaining questions	54
12 R7. Component retrieval at library scale	54
12.1 The problem and established precedents	54
12.2 Constraint-carrying AND-hypergraphs	55
12.3 A concrete refinement and retrieval loop	56
12.4 Dictionary-aware obligations and caching	56
12.5 The data-goal adapter and a leakage-controlled corpus	57
12.6 Remaining questions	58
13 R8. Ranking and candidate equivalence	58
13.1 A declared preference rather than a correctness criterion	58
13.2 Exact identity, proved equality, and observational groups	59
13.3 What a stopping certificate must cover	60
13.4 Remaining ranking questions	61
14 R9. Learned guidance under a kernel gate	61
14.1 A measured proposal service	61
14.2 Restricted proposals and independent authority	62
14.3 Reordering, extension, and replay	63
14.4 Remaining guidance questions	64
IV Plan	64
15 A dependency-ordered implementation and evaluation plan	64
15.1 One joint state, several bounded services	64
15.2 Responsibilities, not a required file layout	65
15.3 Phases and exit evidence	65
15.4 Benchmark dimensions and accounting	66
15.5 Adversarial tests before stronger claims	67

15.6 Risks and falsifiable decisions	68
16 A focused agenda for outside expertise	68
17 Integration provenance and evidence boundaries	70
17.1 What was consolidated	70
17.2 The historical question inventory	71
17.3 What the preserved executable checks establish	72
17.4 Versioned source observations and disagreements	72

Part I

Where the engine stands

1 Evidence

This section records what was measured. Everything later in the document is argued from it.

1.1 What passes

Table 1 lists the acceptance harnesses in `tools/`. Each replays a Leant suite through the Leant2 REPL. Leant ran most of them once per engine (Djinn, Exference, Both); Leant2 has one adaptive engine and no settings, so each case runs once. A case is scored by outcome category, not by transcript text.

Harness	Source in Leant	Score	Wall time
<code>run_baseline.py</code>	<code>test/synth-*.txt</code> , 30 transcripts	278/278	109 s
<code>run_recursive.py</code>	<code>test-recursive</code>	9/9	5 s
<code>run_church.py</code>	<code>test-church</code> behavior probes (core, extended, partial)	28/28	162 s
<code>run_context.py</code>	<code>test-behavioral</code> , <code>test-context</code> , nested forall	95/95	8 s
<code>run_corpus.py</code>	<code>test-church</code> signature corpus	350/350	11 s
<code>run_session.py</code>	session provider identity	6/6	19 s

Table 1: Acceptance harnesses at commit `33cec8b`, 10 s budget per query. The same scores hold with a 5 s budget. The Church wall time is dominated by thirteen unscored stretch cases that each run to the budget.

After the last round of work the slowest scored case takes 2.0 s (Church `foldl1`); the next slowest take 1.1 s and 0.7 s. Most contract queries finish in the 400 ms grace period that follows the first accepted candidate.

1.2 What does not pass

Two groups of cases fail.

Stretch cases. Thirteen of the nineteen partial Church operations (`foldl1`, `foldr1`, `reduce`, `maximum`, `maximumBy`, `minMax`, `at` and their variants) are not solved within 10 s, and four that were retried are not solved within 60 s either. Leant’s own behavior ledger records every one of them as failed or unresolved in all engines, so this is parity, not regression. Their reference solutions need a fold whose carrier holds more than the result: an optional first element, a pair of running extremes, or an index together with a continuation.

A probe just outside the corpus. Sixteen small queries were written to find the edge of the engine (Table 2). They are ordinary first exercises in the program-synthesis literature. Seven fail, and one succeeds with an error.

#	Query	Outcome	Reason
1	reverse a <code>List</code> , two examples	found, 0.4 s	library provider <code>List.reverse</code>
2	append two lists	found	provider
3	double a <code>Nat</code>	found	<code>Nat.add a a</code>
4	2^n on <code>Nat</code> , three examples	rejected	no recursion on <code>Nat</code> ; 3675 programs refuted
5	length of a list	found	provider
6	sum of a <code>List Nat</code>	found, error	<code>List.rec</code> candidate; binding it fails in the code generator
7	head of <code>Vec A (n+1)</code>	found	dependent case analysis works
8	map over <code>Vec A n</code>	not found	induction with an index-dependent motive
9	maximum of two <code>Nat</code>	rejected	no conditionals or comparisons
10	drop zeros from a <code>List Nat</code>	rejected	conditional inside a fold
11	in-order flatten of a binary tree	rejected	combines two recursive results; depth
12	quantified length-preservation contract	found	identity; proved by <code>simp</code>
13	$\exists n, P n$ from $P 3$	found	witness from a local
14	an element of <code>Fin (n+1)</code>	timeout	needs a library lemma ($0 < n + 1$)
15	symmetry of equality	found	
16	transitivity of $\leq$ on <code>Nat</code>	not found	library lemma, no retrieval

Table 2: Edge probe, 6 s budget. “Rejected” means programs of the type were proposed and every one failed the contract.

1.3 Where the time goes

The lane trace (`set_option leant2.trace true`) reports self times of the search steps. On the slowest query before the last optimization, the Church `maybeEither` probe (5.4 s, 2858 search nodes), the breakdown was:

Step	Time	Share
Residual evaluation of the contract on partial programs	2.4 s	45%
of which kernel reduction	1.8 s	
of which substituting delayed assignments	0.4 s	
Deciding the contract on closed programs	0.7 s	13%
Forward-application preparation (rule 7b)	0.2 s	4%
Unification in exact-local and apply steps	0.2 s	4%
Save and restore of the metavariable context	0.07 s	1%
Everything else instrumented	0.2 s	4%
Not attributed by any timer	1.7 s	30%

Table 3: Self times on one contract-heavy query. Roughly 1.9 ms per search node.

Three facts from this profile matter later.

1. **One rule was 90% of the search.** Disabling the accumulator-instantiation rule (7a) alone took the query from 5.4 s to 0.5 s. The rule was then restricted to recursive eliminators. Rule cost is dominated by the subtree a rule opens, not by the rule’s own time, and we have no model that predicts it.
2. **Evaluation dominates per-node cost** whenever a contract is present, even after three optimizations (deciders synthesized once per query, kernel instead of meta-level reduction, skipping re-evaluation while it is stuck on the same holes).

3. **A third of the time is unexplained.** Save and restore are cheap, so the usual suspect is cleared. The remainder is spread over many small monadic operations per node.

1.4 What the search is

For the rest of the document it helps to state the search precisely. A state is a Lean metavariable context together with an ordered list of open goals, each carrying a remaining depth, a deferral count and a set of consumed eliminators. A step takes the first goal, tries an ordered list of about fifteen rules, and for each alternative saves the context, applies the rule, and continues with the rule's subgoals prepended to the remaining goals. Failure restores the context. The order of goals is changed by deferral rules. The whole tree is searched depth first with iterative deepening on the depth bound, under lane deadlines, and the first accepted candidate starts a 400 ms grace period. There is no memoization across branches, no cost model beyond depth, and no learning within or across queries.

This design was chosen because goals share metavariables: assigning a hole in one goal changes the type of another, so goals cannot be solved independently. Carrying all unfinished obligations in the continuation makes backtracking correct by construction. Its price is that the search order is fixed by the rule order and the goal order, and every improvement in Part III has to live with that or replace it.

2 Constraints and classification

2.1 Constraints that stay

The owner's requirements and the decisions of the unified proposal are not reopened here.

- **One adaptive engine, no settings.** Every improvement must decide for itself when it applies. A technique that needs the user to choose a mode is out of scope; a technique that needs a tuning constant must come with a way to set the constant from the query.
- **The kernel is the only authority.** A candidate is published only after the exact program and the exact contract proof pass `addDeclCore` and the axiom audit. Heuristics, evaluators, learned models and external tools may propose and may prune; they never certify.
- **Pruning must be sound with respect to the contract.** A branch may be abandoned only when no completion of the partial program can satisfy the contract, or when the loss is an accepted incompleteness of a bounded search (depth, splits). The first kind needs an argument; Section 8 returns to this.
- **Preserve regression evidence.** The six historical harnesses of Table 1 retain their original revision and scope. An implementation change must also run the maintained acceptance suite at both 10 s and 5 s; an old score is not a current-source receipt.
- **Core-only by default.** The toolchain is Lean 4.34.0 without Mathlib. Mathlib, Aesop, Plausible and similar packages may become optional lanes, never requirements.
- **Replayable logical work.** A fixed query, environment, grammar, policy and proposal transcript should reproduce a recorded logical search prefix and its candidates. Equal wall-clock budgets need not complete the same prefix. Parallel result sorting alone cannot remove differences in discovery.

2.2 Three classes of work

[E] Engineering. The algorithm is known and has been used in a comparable setting. The risk is effort and integration.

[A] Adaptation. The algorithm is known for simply typed or ML-like languages, or inside Lean for proofs rather than programs. It must be re-derived for dependent types, universe polymorphism, type classes, and a search whose state is a metavariable context. The risk is that the adaptation loses the property that made the original work.

[R] Research. We do not know of an algorithm that is optimal, or even clearly adequate, for the problem as it arises here. These are the places where time spent reading, experimenting and talking to specialists is likely to pay more than time spent coding.

2.3 Work items

The motivation column is the original 33cec8b planning snapshot. It is retained to explain the research questions, not as an inventory of defects in the current checkout. The integrated answers below revise the proposed algorithms and their guarantees; current implementation status belongs to the maintained implementation notes and revision-specific acceptance archives.

Id	Item	Class	Evidence that motivates it
E1	Emit results as compilable definitions	E	probe 6: <code>List.rec</code> fails in the code generator
E2	Frontends: term, tactic, code action, sketches with holes	E	only a REPL and <code>#leant2</code> exist
E3	Receipts, replay and resumable search	E	unified proposal, not built
E4	Substrate cost: the unattributed 30%	E	Table 3
E5	Parallel lanes	E	lanes run one after another
E6	Decision procedure for intuitionistic propositional targets	E	negative results are bounded-search only
E7	Wider proof portfolio (<code>grind</code> , bounded instantiation)	E	quantified contracts rely on <code>simp</code>
E8	Benchmark suite beyond Leant	E	corpus saturated at 100%
A1	Conditionals and guard abduction	A	probes 9, 10
A2	Recursion on <code>Nat</code> , literals and arithmetic components	A	probe 4
A3	Normal forms and symmetry reduction in enumeration	A	junk candidates in traces
A4	Parametricity-directed test inputs	A	contracts are user-written examples only
A5	Rule indexing with discrimination trees	A	every rule scans every local
R1	Search control with coupled goals under a wall-clock budget	R	one rule was 90% of a search
R2	Carriers, accumulators and auxiliary state	R	13 stretch cases; probe 11
R3	Evaluation of programs with holes	R	45% of time on contract queries
R4	Indexed families and the motive problem	R	probe 8
R5	General recursion from incomplete examples	R	probes 4, 11
R6	Contracts beyond decidable observations; program and proof together	R	portfolio is five tactics
R7	Component retrieval at library scale	R	probes 14, 16; twenty curated providers
R8	Ranking and candidate equivalence	R	three-key lexicographic ranking
R9	Learned guidance under a kernel gate	R	none today

The research items share machinery without having identical obligations. R2 and R5 both invent state or helper interfaces absent from the specification; recursive realization also needs termination, scope and interpreter-agreement arguments. R3 can make both searches more affordable through conservative observations. R1 allocates their work within the interactive

budget. Useful bounded carriers, native induction and sketches need not wait for an unrestricted carrier-minimization algorithm. Section 15 gives the revised dependency order.

3 A common semantic boundary for the integrated design

The nine expert responses, identified as N1–N9 in the integration ledger, agree on the most important correction to the original plan: certification, search coverage and useful execution are different obligations. This section states their shared foundation once. Later sections retain the distinct constructions, counterexamples and unresolved questions that depend on it. The arguments are mathematical design arguments, not new Lean formalizations.

3.1 Freeze the problem before proposing a solution

Let the original query be

$$Q = (E, \Gamma, T, C, A, \mathcal{L}, \mathcal{G}, B, \Pi),$$

where E is the imported environment, Γ the local telescope and its instances, T the target, $C : T \rightarrow \text{Prop}$ the contract, A the permitted axiom policy, $\mathcal{L}$ the providers, and $(\mathcal{G}, B)$ a declared grammar and its bounds. The frozen policy Π identifies the ranking objective, logical resource accounting, transparency/equality settings, search and proposal admission policy, and their versions. A policy change is recorded as a new experiment or an explicit epoch transition; it must not silently reuse a certificate or cache entry whose assumptions changed. An absent behavioral contract means $C(p) = \text{True}$. Closing a local answer abstracts the original telescope; it does not replace its dependencies with similarly named variables. Original universe parameters remain rigid. A term at $T[\ell := 0]$ answers a specialized query, not an arbitrary original universe-polymorphic query. Any permitted generalization of new inference variables must preserve the jointly checked program/proof types.

Definition 3.1 (Completion set). A partial state S contains the typed program root, expression and universe assignments, delayed constraints, local contexts and instances, recursion capabilities, residual observations and proof obligations. Write $\text{Comp}_{\mathcal{G}, B}(S)$ for its compatible closing substitutions under the declared grammar and bounds. A completion is a native well-scoped substitution, not a textual replacement of question marks.

Finiteness needs actual bounds on literals, universe expressions, auxiliary declarations, recursion schemes and proof plans as well as term depth. A finite program grammar alone does not turn a bounded tactic portfolio into a decision procedure for arbitrary Lean propositions. Conversely, a finite example conjunction can itself have a fully checked proof: its finite scope does not make the proof less valid, but it does not specify an unstated universal intention.

3.2 Six guarantees and their separate evidence

Positive certification. The closed abstractions of $p : T$ and $\pi : C(p)$ check in E , and the transitive axiom dependencies of the program, original type, proof and original contract are permitted by A . A kernel-checked theorem may use allowed axioms; it need not be axiom-free.

Conservative pruning. A refuted partial branch has no permitted satisfying completion. This requires a separate argument about the refuter. Checking all surviving answers cannot reveal a solution discarded before the gate.

Relative coverage. Every member of a named candidate grammar is considered or safely excluded by a fair exhaustive run. Certification completeness additionally needs a decisive verifier for the advertised fragment, or a separate finite proof grammar. Top- k retrieval, queue eviction and a deadline are budget choices, not exclusion proofs.

Ranking optimality. No eligible candidate improves one fixed objective. First success, best among visited terms, immutable program cost and a learned dispatch score are not interchangeable. Outstanding alternatives need valid bounds before an optimality claim is made.

Replay. Proof replay, logical search replay and reproduction of model inference have different prerequisites. A fixed logical prefix can be replayed; equal wall time on different machines does not imply an equal completed prefix.

Executable publication. Declaration transport, compilation and advertised execution are independently checked. A well-typed recursor expression does not by itself establish that the supported publication path compiles or executes it.

These are six obligations rather than six promised completed features. The responses group them differently: most distinguish the first five, N6 groups its principal guarantees more coarsely, and N5 and several realization discussions make the sixth explicit. No disagreement is resolved by silently folding execution into kernel acceptance.

3.3 Necessary observations, alternatives and refutation

For a partial root p_S , a sound hard rejection requires

$$\forall \theta \in \text{Comp}_{G,B}(S), \quad \neg C(p_S \theta). \quad (1)$$

An inferred observation or angelic constraint D is safe to add to every remaining alternative only if

$$\forall \theta \in \text{Comp}_{G,B}(S), \quad C(p_S \theta) \implies D(\theta). \quad (2)$$

If D has no compatible realization, these two statements imply safe refutation: a satisfying completion would satisfy D , contradicting its impossibility. An observation need not hold for completions that violate the contract. This is why an example can constrain a recursive call conditionally on correctness of the whole synthesized function.

A sufficient guess has a different role. One guessed invariant, recursive result or helper can define a search alternative; failure of that alternative does not reject all other guesses. Repeated equal recursive calls must remain correlated. Fuel exhaustion, an unsupported evaluator case, a failed tactic or an unsuccessful unification attempt remains inconclusive unless a stronger decisive fragment supplies the missing exclusion argument. Differential testing can falsify a refuter's correctness, but passing tests cannot prove Equation 1 for every open completion.

3.4 Results must identify what was established

Table 5: A result algebra with explicit evidence boundaries.

Result	Required meaning
Verified	A term of the frozen target and a checked proof of its exact requested contract under its policy.
Observed / unrefuted	Recorded observations for a typed candidate; required certification is still missing.
Candidate refuted	A particular candidate or partial branch violates a checked necessary condition.
Impossible	A checked theorem excludes solutions of the original query under the advertised policy.
Bound completed	The named finite program/proof search was covered; unresolved verification obligations are recorded separately.
Interrupted / unsupported	Work stopped or a feature is unavailable; neither states that the target is uninhabited.
Publication status	Separate export, compilation and execution results for the accepted artifact.

Refuting every visited candidate is not proving impossibility. Exhausting syntax while some verification attempts remain unresolved is not semantic exhaustion. A finite Kripke countermodel for a reified propositional calculus establishes nonvalidity in that calculus; interpreting it as a Lean negation of an arbitrary contextual target needs a further bridge. These distinctions belong in receipts and diagnostics and need not become user-facing engine settings.

3.5 Services with deliberately different authority

```

ProposalService.propose : DecisionPoint -> SearchView -> Alternatives
ObservationService.check : PartialState -> ValuesOrBlockersOrRefutation
ProofService.tryProve : TypedObligation -> Budget -> ProofOrUnknown
AcceptanceGate.check : FrozenQuery -> Candidate -> Proof -> GateReply
PublicationService.realize : Accepted -> ExportCompileExecuteReceipt

```

This is interface pseudocode. A proposal changes the order or adds an alternative; it carries no authority to remove every other alternative. A refutation carries checked evidence or invokes a component with established conservative semantics. The gate reconstructs $C(p)$ from the original query, rather than trusting an unrelated supplied proof type. All services use the same dependency information and retain their active resource checks.

The source packages inspected selected code at [784664ff34e819422510a4d470ab07fe48bb736b](#); their API observations are historical. They correctly distinguish a gate accepting a proof/type pair from an end-to-end caller that constructs the requested contract. Their recommendation to freeze the query at the gate is not evidence that such a caller accepted an unrelated proof. Likewise, N5’s observations about negative certificate retention, universe specialization and intermediate partial reduction motivate explicit invariants; they must not be presented as unchecked defects in a later implementation. This documentation integration does not change the implementation or claim a new acceptance run.

State ownership must be explicit across these services: immutable reusable content, branch-local assignments/views, and monotone work/diagnostic counters are different classes. An `IO.Ref` is not automatically a globally valid cache. The joint dependency and scheduling obligations are

developed in Sections 6 and 8; proof and realization obligations remain attached to the same original query throughout.

Part II

Work with known solutions

4 Engineering items

Each item retains a motivation from the original 33cec8b planning snapshot, a revised design, and prospective exit evidence. These descriptions are not a current-checkout defect inventory. The integrated semantic boundary in Section 3 and the phased plan in Section 15 qualify the original implementation suggestions.

4.1 E1. Results as compilable definitions

Historical motivation. The profiled REPL bound candidates as `it1`, `it2`, ... with `addAndCompile`. A candidate built from a recursor, such as the list sum

```
fun a => List.rec Nat.zero (fun head tail tail_ih => head.add tail_ih) a
```

was accepted by the kernel and then rejected by the code generator (“does not support recursor `List.rec`”). That run displayed a correct answer followed by errors and did not provide an evaluable published result.

Design. Separate the certified term from the published definition.

1. Preserve the exact certified program and original contract proof. Export them into a staging environment, abstracting the original local telescope and universe parameters, and recheck the resulting declarations. Separating `addDecl` from compilation avoids conflating two operations; it does not prove that declaration transport or binding cannot fail.
2. Construct an executable presentation through Lean’s equation compiler. Where a recursor plan supports a pattern-matching definition, elaborate that definition, compile it, and prove its correspondence with the logical term or prove the original contract directly for the executable definition. Try definitional equality first; use induction or the checked equation theorems when required. A source-level `match` rendering by itself is not a compilation or correspondence proof.
3. Publish the checked executable definition when all promised obligations hold. An `@[implemented_by]` route requires its own audited backend and correspondence assumptions; the attribute is not the theorem connecting two programs. If only the logical artifact is available, label export and compilation statuses explicitly, and offer a `noncomputable` presentation only if that declaration has itself checked. It does not satisfy an executable publication request.

Constant-motive non-indexed folds are an appropriate first supported fragment; arbitrary recursor syntax can still contain nested eliminations or scope and sharing issues. Nonconstant motives and dependent arguments require the realization obligations in Sections 9 and 10. Use version-pinned native elaboration interfaces and retain explicit unsupported outcomes rather than asserting a universal mechanical translation.

Closing test. Probe 6 exports and compiles without error and `#eval it1 [1,2,3]` prints 6. Every supported recursive candidate has separate logical, exported and compiled receipts and a readable source presentation whose elaboration is rechecked. Printing a `match` is a presentation test, not a substitute for those gates.

4.2 E2. Frontends

The original plan began with the REPL and `#leant2`. A shared query interface can support additional frontends, but dependent local contexts, universe preservation, hole ownership and replay make their adapters substantive correctness boundaries rather than merely syntax wrappers.

Term elaborator `synth%` elaborates to a certified candidate under the declared ranking. Postpone unresolved expected-type dependencies when necessary; do not specialize the original universe parameters or assign foreign holes merely to make the query easier. Timeout without a certified candidate remains an incomplete result.

Tactic `leant2` closes a goal whose target is data or a proposition, reporting the term through `Try this`. It competes with `exact?`; its advantage is construction (case analysis, recursion, contracts) rather than lookup.

Sketches A body such as `def f xs := List.foldr ?step ?init xs`, together with an explicitly associated contract, supplies a partial program and carrier hint. Distinguish owned holes from pre-existing elaborator metavariables, close each typed local interface, preserve dependencies and apply the final original contract to the whole completed program. Sketch surface syntax and hole elaboration must be specified and tested, rather than inferred from the existence of an internal metavariable search state.

Code action Offer “synthesize body” at a `sorry` with explicitly associated tests. Recognize and check each test’s relation to the declaration; textual proximity alone does not identify the intended contract. Replay the replacement source in its original declaration context.

Library API A documented `Leant2.synthesize : Query -> MetaM Outcome` with the result algebra, for use from other metaprograms, including Forge.

Closing test. Each frontend has native success, scope, policy and interruption cases. The prospective sketch target is to solve all thirteen stretch tasks with their reference carriers and unchanged contracts. Record body-search failures separately: supplying a carrier does not establish that transition and finishing synthesis will succeed.

4.3 E3. Receipts, replay and resumable search

The unified proposal specifies candidate receipts containing query, environment, program, contract proof, axioms and ledger. Preserve proof replay separately from logical search replay and executable publication. A resumable incomplete outcome also needs the original problem, pending alternatives and proofs, semantic state, expansion cursor, policy/model epoch and monotone work charges. Section 6.5 gives the full checkpoint contract.

A deterministic lane/depth/alternative path is one possible reconstruction format if all effects, ordering and external proposals needed to reproduce it are pinned. It is not sufficient by itself for branch caches, delayed constraints or dynamic priorities. Compare reconstruction cost and native persistent snapshots experimentally; neither is universally cheaper or more robust by construction.

Closing test. Cutting and resuming at recorded logical checkpoints reproduces the candidate and event sequence of uninterrupted execution to the same final logical prefix. Separately measure

reconstruction and startup time. Three two-second executions need not reach the same prefix as one six-second execution, as Proposition 6.3 explains.

4.4 E4. Substrate cost

The historical profile left thirty percent of contract-query time unattributed (Table 3). The plan is measurement first.

1. Build the executable with a sampling profiler attached (on Windows, ETW; on Linux, `perf`) and read the C call stacks of the Lean runtime. This distinguishes allocation, reference counting, persistent-map operations and `instantiateMVars` traversals without guessing.
2. Re-profile the repeated scans and instantiations identified by the historical source inspection. Cache per-local facts only with their complete assignment, universe, transparency and instance dependencies; introducing a local once does not make its type immutable while metavariables remain open. Keep branch-valid caches in the goal/state representation, with invalidation and rollback tests from Section 8.
3. Replace linear scans by indexes (item A5).

Closing test. The original targets of unattributed time below 10% and node cost below 1 ms on the Church probes remain measurable hypotheses. Report query mix, inclusive/exclusive accounting, cache overhead and retained memory; moving time between counters is not a speedup.

4.5 E5. Parallel lanes

Whole-lane parallelism is a candidate optimization when each worker owns its semantic state and the shared environment is frozen. Audit all external references, environment mutations and services before declaring the workers independent; an `IO.Ref` does not establish safe ownership. Shared work accounting and cancellation require an explicit synchronization protocol.

For deterministic logical replay, give jobs stable identities, fixed work quanta and a canonical commit order. Workers may speculate, but admit completed epochs by that order. First-found cancellation is a separate opportunistic policy: ranking its survivors afterwards cannot recover candidates lost to the race. Preserve its actual event trace without claiming identical timed answer sets. The detailed boundary is in Section 6.5.

OR-parallel alternatives and carrier proposals follow the same ownership discipline. AND-parallel subgoals additionally need the factorization conditions of Section 6.2. Measure memory, duplicated work, publication latency and lost useful work on cancellation, not only CPU utilization.

Closing test. Compare sequential and parallel lanes at matched logical work and separately under a common interactive budget, preserving the same certified outcomes and replaying completed checkpoints. Faster classical or refutation results are an empirical target, not an automatic consequence of scheduling tasks concurrently.

4.6 E6. A decision procedure for propositional targets

For the reified implicational-conjunctive-disjunctive propositional fragment, intuitionistic provability is decidable and contraction-free LJ_T calculi give terminating search [11]. A checked finite Kripke countermodel establishes nonvalidity in that calculus. It does not by itself prove a Lean negation for an arbitrary contextual target whose atoms have an interpretation. Positive reconstruction and any negative transfer both need the reification bridge and complete treatment

of the relevant assumptions specified in Section 3. Likewise, a SAT result about the reified classical formula needs its appropriate checked certificate and interpretation.

Such results can justify scheduling constructive and policy-permitted classical work for the supported fragment. They do not decide arbitrary Lean propositions merely because `Prop` occurs in their type. General intuitionistic propositional validity is PSPACE-complete [29]; small practical instances and latency must be measured.

Closing test. Inventory which `synth-basic` and `synth-manual` queries satisfy the supported reification assumptions. Each covered query receives a checked positive reconstruction or a precisely scoped negative certificate; unsupported contextual queries remain explicit. The original under-50-ms target is prospective and reported separately from certificate validity.

4.7 E7. A wider proof portfolio

The historical portfolio `first | rfl | decide | simp | omega` motivated a transactional adapter with typed replies and per-tier budgets. Evaluate available `grind`, `try?`, arithmetic simplification and bounded instantiation adapters at the pinned toolchain; names alone do not establish integration or cost. Local-context proof obligations and induction hypotheses need the richer interface of Section 11.

For $\forall x : \tau, P x$, a concrete well-typed input with a checked proof of $\neg P(x)$ refutes the candidate. Enumerating a few values can find such a counterexample without a small-model theorem; passing those tests does not prove the universal contract. Finite exhaustive validation requires an enumeration coverage proof and exact evaluation. Retain unknown evaluation or proof outcomes and the axiom policy for negative evidence. Schedule testing and proving fairly rather than making an arbitrary finite test prefix the sole gate to all proof work.

Closing test. A suite of twenty quantified contracts over lists and naturals (length preservation, idempotence, involution) where wrong candidates are refuted by instantiation in under 5 ms each.

4.8 E8. Benchmarks beyond Leant

The historical scored Leant corpus was saturated; that does not establish coverage of its unscored stretch tasks or of broader synthesis. Port suitably scoped tasks from Myth [24], λ^2 [19], Smyth [28], Burst [5], Trio [34], and the type-only Agda/Coq studies [3, 26], deduplicating semantic families and recording each source’s actual grammar, supplied helpers, examples and recursion scope. Add Lean tasks by erasing small core `List`, `Option` and `Nat` definitions and selecting suitable lemmas as explicit contracts. A published score in another language is not a matched Leant2 baseline by default. The historical stretch cases and failing probes provide initial tasks, with separate carrier/motive-given diagnostics. Use the complete timing, evidence and ablation matrix in Section 15; report certification and executable publication separately from first candidate discovery.

5 Adaptation items

These techniques have precedents in other settings; their guarantees do not transfer automatically to dependent Lean synthesis. Gap descriptions below refer to the original 33cec8b planning snapshot. The revised adaptations state the extra conditions required by the integrated answers, rather than claiming an inventory of currently missing features.

5.1 A1. Conditionals and guard abduction

Historical motivation. The original grammar lacked comparison components and a Boolean/decidable guard rule for terms such as `if a <= b then b else a`. Probes 9 and 10 motivated this extension. Adding a missing production removes that particular expressibility obstacle; it does not establish that the extended search will find and certify a result.

What is known. Enumerative synthesizers separate *terms* from *conditions*: find terms that are each correct on some examples, then learn a decision tree of guards that routes every example to a correct term (the divide-and-conquer strategy of EUSolver [13]). Type-directed synthesizers abduce the guard instead: Synquid generates a branch, asks a solver for the weakest condition under which it satisfies the refinement type, and synthesizes a guard that implies it [30]. Example-directed tools split on a scrutinee when the examples disagree about the result [24].

Adaptation. For a supported observation fragment, extend residual evaluation to report which observations a typed branch candidate satisfies, with its local input environment, instead of returning only the first failure. Then:

1. collect branch candidates whose satisfied sets cover all observations;
2. enumerate guards from a small grammar over the locals (decidable comparisons of locals of the same type, Boolean-valued locals and their applications, `isEmpty`-style tests obtained from constructors);
3. evaluate each guard on those typed input environments and search a bounded decision-tree grammar for a covering tree; call it smallest only after complete search under a fixed cost and bound;
4. emit `if g then t1 else t2` (or `match`), and let the ordinary closed-program check certify it.

The difficult adaptation is identifying “the inputs of an observation.” A contract can relate several calls to f or invoke it under a fold. A trace must retain their correlations; routing each call independently because a whole conjunct held for one constant branch candidate does not preserve that conjunct after branch mixing. Use checked observation decomposition, or keep the relational constraint and validate the assembled decision tree against the original contract. Necessary inferred observations need the implication in Section 3; unknown decomposition remains opaque. Recursive call environments add the capability and constraint obligations in Sections 8 and 10.

Closing test. Probes 9 and 10; the `max`, `min`, `filter`, `insert-sorted` and `compare` entries of the ported benchmarks.

5.2 A2. Recursion on natural numbers, literals and arithmetic

Historical motivation. The original plan excluded natural-number structural recursion after nested induction caused search growth, and listed only `Nat.add`, `Nat.succ` and `Nat.zero` as numeric components. Its probe 4 (2^n) therefore motivated both recursion and arithmetic extensions. This is a statement about that restricted planning grammar, not current implementation coverage or mathematical nonexistence.

Adaptation. Keep three choices separate. First, prioritize an outermost `Nat.rec` schema with explicit decreasing capabilities before nested recursion; an outermost-only tier is a useful restriction, not coverage of every primitive-recursive program. Second, seed literal enumeration with constants from the contract plus 0, 1 and 2, charging literal encoding size. Broader

bounds must eventually admit omitted literals if relative coverage is claimed. Third, evaluate demand-directed `Nat.mul`, `Nat.sub`, `Nat.pred` and comparison components while preserving natural-number truncated-subtraction semantics. Carrier, motive and recursion bounds belong to the joint grammar of Section 7.8.

Measure native reduction on closed numerals and partially known arithmetic separately. An unresolved term may block an optimized reduction; that alone does not establish that every nearby hole forces unary evaluation or identify the dominant cost. Use residual traces to justify any specialized evaluator and preserve its unknown outcomes and arithmetic semantics.

Closing test. Probe 4, factorial, Fibonacci with a pair accumulator (which needs R2), sum to n , and the `nat_*` entries of the Myth suite.

5.3 A3. Normal forms and symmetry reduction

Historical motivation. The original traces showed duplicate construction paths, administrative $\beta\eta$ variants and redundant fold steps. These motivate canonical construction and duplicate detection, but a well-typed simplification and a behavior-preserving search quotient have different obligations. For example, an element-ignoring step may yield the seed; it must not be called the identity on the input list without checking the entire fold and its type.

What is known. Focused inhabitation calculi use canonical β -normal η -long forms for their specified calculi; Canonical is a relevant dependent-search reference [6]. Applying that statement to an unrestricted Lean grammar additionally requires accounting for definitional reductions, proof arguments and supported syntax. Bottom-up synthesizers use finite observational signatures to compress component spaces [33, 12]; their interchangeability claim is relative to the observation environments and compositional language, not arbitrary contexts.

Adaptation. Audit overlapping exact-local and introduction routes and redundant forward applications, then normalize only within a documented typed fragment. A restriction that removes forward applications needs either a completeness argument for the chosen normal-form grammar or an explicit incompleteness label. Symmetry reduction for a commutative component requires its actual commutativity law and a representative policy consistent with the candidate objective.

Finite signatures are safe hard quotients only when equivalent representatives remain interchangeable in *all admitted surrounding contexts*, or when discarded alternatives can be recovered by the reference search. Ground type and finitely instantiated free variables alone do not prove that congruence: a higher-order or recursive context can invoke the fragment in an unobserved environment. Keep observational buckets as ranking/proposal structures by default and use checked equality or a certified fragment theorem for permanent merging. The same restriction applies to the bottom-up companion of Section 6.

5.4 A4. Parametricity-directed test inputs

Conditional idea. In an explicitly parametric fragment, a family `f : (A : Type) -> List A -> List A` commutes with `map`; the surface type alone does not prove this for arbitrary Lean providers or classical constructions. Theorem 7.4 and Equation 5 derive the shape/position reduction, including repeated elements, empty types and its universe assumptions. The polymorphic-testing literature supplies its own relational hypotheses [36, 4]; a theorem about a suitable

monomorphic instance does not make finitely many values of that instance globally complete tests.

Use in Leant2. Explain a candidate with a symbolic example such as $[0,1,2] \mapsto [2,1,0]$, and show a distinguishing input when accepted candidates disagree. Group equal sampled behaviors with an explicit sample scope; do not call them equal programs. Under proved naturality, agreement on canonical inputs through length N proves equality only on inputs through length N . Equality for every program of a syntax-size bound would require a separate small-model theorem, which the incoming reports do not establish.

Hard deduplication needs that stronger justification or checked candidate equality. Equality/order dictionaries, opaque callbacks and indexed fibers require provider-specific preservation laws and suitable symbolic structures; see Section 7.5. Use canonical tests as useful probes immediately, and claim complete reduction only for a certified fragment.

5.5 A5. Rule indexing

The historical repeated local/provider scans motivate Lean’s `DiscrTree` or `LazyDiscrTree`, with Aesop as a rule-selection reference [2, 79]. These indexes can avoid scanning unrelated rigid heads, but retrieval cost also depends on matching branches, wildcard entries and returned candidates. It is not bounded solely by goal size independent of the index or output size.

Index provider conclusions for a pinned environment and normalize keys under an explicit transparency policy. Maintain local entries with valid dependency identities as contexts and assignments change. Persistent structure permits sharing; placing it inside the actual branch snapshot or validating its keys is still necessary for rollback. Bare-variable conclusions require broad matches, and reductions or transports accepted by the native checker must not be silently excluded by stricter keys. Demand filtering is a heuristic unless its necessary-condition argument is proved. Section 12 states the dependent and dictionary obligations for stronger filtering.

Closing test. Retain the original target of less than 10% node-time growth with two thousand session declarations as a hypothesis to measure. Report cold index construction, warm retrieval, wildcard density, returned candidate count and native checking separately; verify retrieval recall on known usable components rather than measuring speed alone.

Part III

Integrated answers and remaining problems

6 R1. Search control with coupled goals under a wall-clock budget

6.1 The problem and the corrected design

The historical profile in Section 1.3 motivates three separate decisions: which partial program to expand, which coupled hole to refine, and how to allocate a finite budget among lanes and depths. In that profile one rule opened 90% of the tree; a depth-4 pass consumed 4.7 seconds without a candidate while a depth-5 pass found one in 0.5 seconds. These observations motivate experiments with resumable search, not a theorem that best-first search or a particular cost function will improve every query.

Incoming answers N1–N9 agree on a corrected design: retain a joint native constraint state, share persistent structure, factor only components whose independence is justified, and separate the final candidate objective from exploration priority and charged work. Two original requirements need repair. “One unit per open non-leaf goal” is not generally an admissible heuristic; and identical candidate lists under a fixed wall-clock deadline cannot be guaranteed across differing machine speeds while reporting every improvement reached. The following results answer the historical R1 questions; the active expert agenda at the end asks about the remaining implementation and research gaps.

6.2 Conditional factorization and contextual reuse

There is a useful middle ground between copying all work and imposing one global sequential order. Represent alternatives as a persistent whole-state OR graph and factor certified independent constraints into AND components. Genuinely incompatible assignments still require distinct logical branches. Aesop already tracks transitive clusters of metavariable-coupled goals, including dependencies through local contexts; independence detection itself is not a new idea absent from Aesop [2]. Its copying of affected goal nodes should not be confused with wholesale copying of every search subtree. Canonical supplies a comparison for metavariable refinement order, and the incoming reports identify Canonical-min’s suspended constraints and assignment-triggered resumption as another useful reference [38]. Guarantees for that calculus do not transfer automatically to arbitrary Lean tactics.

For each obligation, conservatively close its support through expression and universe metavariables, assigned expressions, declaration types, local types and values, delayed-assignment environments, postponed equations, relevant instance arguments, provider types, and admissible local expressions. Add residual observations as constraint nodes: $x, y : \mathbb{N}$ may have independent types while $x + y = 7$ couples their acceptable assignments. Likewise $f(0) + g(0) = 1$ couples two function bodies. An immutable shared parameter or constant need not connect components; a writable variable in its type does.

The hypergraph can use unresolved variables as vertices and obligations as hyperedges, or its incidence-graph dual. What matters is conservative support, including a rule’s possible writes. Treat an opaque tactic with unknown effects as touching its entire reachable state. Overestimated dependencies lose an optimization; underestimated ones can remove compatible completions even if the final kernel gate rejects malformed survivors.

Proposition 6.1 (Conditional component factorization). *After fixing a shared interface assignment η , suppose the remaining typing, grammar and behavioral constraints factor as $\Phi_1(X_1; \eta) \wedge \cdots \wedge \Phi_k(X_k; \eta)$ with disjoint writable sets X_i . Every allowed transition in component i reads only X_i , η and immutable shared data, writes only X_i and fresh private variables, and preserves this separation. Then the completion set at η is the Cartesian product of the component completion sets. Their substitutions commute up to fresh-name renaming.*

Proof. A substitution on the disjoint union of the X_i is a tuple of substitutions on its components. No right-hand side or typing dependency reads another component’s writable variables, so their capture-avoiding union is well formed. Each conjunct is satisfied exactly when its corresponding substitution satisfies it. Conversely, restricting a joint solution satisfies every factor. Transition confinement ensures that this argument continues to hold after refinement, rather than only at the initial graph snapshot. $\square$

A useful operational frame condition for components I, J is

$$W(I) \cap (R(J) \cup W(J)) = \emptyset, \quad W(J) \cap (R(I) \cup W(I)) = \emptyset.$$

With complete read/write footprints, immutable environment and policy, and fresh identifiers renamed apart, each transition reads the same values in either order and their writes unite. This is a sufficient interface contract, not a claim that collecting visible target metavariables establishes it. For a small unfixed separator, subsearches export relations between interface assignments and solutions; the global result is a union of products over compatible η . Joining those relations may still be exponential.

Update supports after assignments or context extensions. New shared helpers or observations can merge components; assigning a shared variable can split one. Union-find alone handles the former but not the latter. Recompute small affected components or maintain persistent adjacency and a wakeup queue. Rules that introduce a cross-component dependency must rejoin components or be prohibited after a component is sealed. A global size budget, a provider usable at most once, or a nonlinear objective is also a connecting constraint. For additive size bounds, combine cost-indexed component solution streams; choosing only each component's first answer can discard the only globally admissible combination.

Three levels of reuse. Share immutable expressions and context nodes; share persistent branch prefixes while keeping assignment maps distinct; and, only under checked interface conditions, reuse a semantic solution. A reusable entry is a typed recipe: abstract the minimal dependency-closed local telescope, canonicalize binders, record the target, residual constraints, interface guard, environment, transparency, instance policy, grammar and relevant dependency fingerprints. On reuse freshen universe and expression variables, substitute actual locals, validate the guard, and type-check the instantiated term. Raw `MVarIds` or `FVarIds` cannot be transplanted into another branch. A proof of $P(x)$ does not become a proof of $P(y)$ through similar printing.

Keep a lazy stream or guarded family of data solutions unless a checked argument makes every remaining alternative irrelevant to the surrounding contract. A success cache and a failure cache have different obligations: closed contextual successes can be replayed; negative reuse requires a checked refutation or exhaustion of a matching finite grammar and bound. Timeout, blocked unification and a failed tactic are scheduling information, not uninhabitability certificates.

6.3 Admissible bounds for joint completion

Begin with $h(S) = 0$ under fixed nonnegative costs. The original unit-per-goal bound fails twice: exact-local closure may cost zero, and one assignment may close several obligations. For example, assigning $?n := 0$ at cost one can make both $n = 0$ and $n + 1 = 1$ reflexive with free propagation. Two visible obligations then have remaining cost one. Repeating the same constraint makes the overestimate arbitrarily large. A model charging each proof constructor would give different costs; therefore the objective must be stated first.

Let $d(S)$ be minimum additional completion cost. If $\ell_i(S) \leq d(S)$ for each justified relaxation of obligation i , then

$$h_{\max}(S) = \max_i \ell_i(S) \leq d(S).$$

A sum is valid only if every completion's cost can be partitioned into component shares that dominate the respective bounds. Semantic independence alone is insufficient when one provider action or shared subterm is charged once but serves several components. Explicitly partition each concrete action's cost without charging more than that action costs.

Proposition 6.2 (Abstract-distance lower bound). *Suppose an abstraction maps concrete solved states to abstract solved states, and every concrete transition of cost c to an abstract path of cost at most c . Then the shortest abstract completion distance $d^\#(\alpha(S))$ is an admissible lower bound on $d(S)$.*

Proof. Map a concrete completion to the concatenation of its simulating abstract paths. It reaches an abstract solution at no greater cost. The shortest abstract completion costs no more than this mapped path. Taking the infimum over concrete completions gives the result. $\square$

Candidate abstractions erase dependent indices, treat proof and dictionary obligations as free, allow extra providers, retain a small separator, or use a finite type-head production hypergraph or constructor-pattern database. A minimum-cost cover or fractional covering relaxation can account for actions that close several goals together. Each construction still needs the simulation argument for *every* admitted production: omitting a cast, higher-order introduction, zero-cost local, or multi-goal unification effect can turn a purported relaxation into an overestimate.

Derivation cost, failed search work and final-expression size are distinct. Filling a shared hole may duplicate syntax; normalization may erase it; a later assignment may change which inputs a completed program uses. A lower bound on production count does not prove optimality under a different final ranking tuple. Use a fixed additive derivation objective initially, or prove the lower bound for the actual advertised candidate objective.

Nonnegative costs alone do not imply fair cost enumeration. Every genuinely generative cycle needs positive cost or a finite secondary structural grade. Administrative zero-cost closure must terminate through a well-founded normalization argument, finite saturation or justified duplicate control; infinitely many distinct zero-cost states evade simple cycle detection. If an admissible but inconsistent heuristic is later used on a graph, reopen a state when a smaller path cost is found. Uniform-cost search is the simpler reference until a stronger bound pays for its computation.

6.4 Learning priorities without changing the objective

Adaptive order preserves accepted-answer correctness under the acceptance gate, but it can change which answers appear before a deadline. Probe supplies a just-in-time learning precedent [25], not a theorem of invariant finite-budget answers or a dependent-search A* guarantee. Keep four fields distinct: final objective $J(p)$, charged work, a proved frontier bound $L(S)$ on unseen J , and dispatch priority $P_e(S)$ for epoch e .

Freeze J for the query. Learning may reward observations survived, useful blockers removed, estimated information gain, or low evaluation cost in P_e . Correlated examples should not automatically outweigh a structurally different observation, and training-example survival may reward overfitting. Measure held-out behavior separately. If the advertised objective itself changes, version it and recompute the affected bounds; silently decreasing edge costs does not preserve the old A* argument.

Use deterministic logical epochs: collect a fixed batch of completed events, aggregate evidence in canonical order, record the update inputs and model digest, then freeze the next epoch's scores. Rekey affected queue entries, rebuild the queue, or use an explicitly separate guidance queue; stale heap keys establish neither the new order nor a reproducible old one. Canonical ties must not depend on hash iteration, timestamps, memory addresses or timing-derived names. Deterministic integer or rational updates are one possible implementation, not a required statistical model.

Reserve a nonzero deterministic share of logical work for a fixed grammar frontier. With finite branching, fair admission and terminating local work, every finite reference derivation eventually receives service. A beam or top- k list in a fast lane may be useful, but omitted alternatives must remain scheduled in the reference lane for a coverage claim. Introducing a helper or carrier outside the grammar is a recorded language extension, not merely reordering. Exhaustion of the old grammar is not exhaustion of the enlarged one.

The appropriate default receipt is “best certified candidates found under objective version v in this logical prefix.” A stronger minimum-cost claim requires an incumbent p and coverage of *all*

unexplored alternatives under the same objective, with

$$J(p) \leq \inf_{S \text{ on the complete frontier}} L(S).$$

A grace period after the first answer is not such a certificate. Stable sorting gives stable order of a given verified set, not stability of the set that a time-limited run discovers. Test user-facing stability under irrelevant renaming, harmless provider permutations, repeated runs and additional examples rather than treating an algorithm name as evidence of predictability.

6.5 Deterministic prefixes and real deadlines

A reproducible logical schedule and a hard deadline are compatible, but the deadline may truncate that schedule at different positions. The anytime literature supplies a quality-versus-deliberation perspective [71]; it does not establish machine-independent equality of timed answer sets.

Proposition 6.3 (Deadline obstruction). *Suppose a deterministic search first certifies a new answer after positive work w . If one execution reaches w before deadline D and another does not, an interface publishing every certified improvement reached by D cannot return the same answer set in both executions.*

Proof. The answer is available only in the first completed prefix. Sorting cannot insert an undiscovered certified answer into the other. Suppressing the new answer or continuing beyond the deadline changes one of the stated requirements. $\square$

Even a restricted speed variation breaks equality if a cutoff crosses an answer event. A known throughput bound can justify fixed work for a specified machine class; startup calibration alone cannot account for cache warmth, instance search, reduction costs, load or allocation. Nor does one expensive node represent a stable amount of work.

Provide an interactive execution contract that respects cancellation and reports its completed prefix, a replay contract that consumes that prefix regardless of elapsed time, and fixed-work runs for experiments. Pin query, environment, grammar, policy, provider order, model, seed and tie-breaks. Checkpoints contain the frontier, semantic branch states, expansion cursors, model state, monotone work ledger, unresolved tasks and exhausted cost bands, not only a candidate count. Charge rule, unification, normalization, instance, observation and proof work at specified points even after rollback. Resuming to the same logical checkpoint should reproduce the stream; three timed two-second runs need not equal one six-second run because startup and replay also consume time.

Parallel workers may speculate, but stable task identities and fixed logical allotments must feed a deterministic commit order. Publish completed epochs; label any opportunistic extras separately. A race to the first answer followed by cancellation is not repaired by sorting its survivors. Cooperative node checks and heartbeats are not universal real-time interrupts: bound expensive services and use a worker cancellation boundary where necessary. Reserve resources for final checking; interruption cannot promote an unchecked term.

Iterative deepening remains a useful baseline: its familiar large-branching overhead argument concerns completed brute-force passes [18], not arbitrarily cut heterogeneous lanes. Likewise the Luby restart theorem assumes a Las Vegas runtime model [22]; correlated deterministic lanes with shared learned state do not automatically meet it. Prefer resumable work to restarting the identical prefix. Compare restarts only when a new policy or random trial gives them a substantive purpose.

6.6 Implementation sequence and evidence still needed

First make the *whole semantic state* and a resumable expansion cursor explicit while reproducing the continuation baseline with a LIFO frontier. `Meta.SavedState` alone is not a specification of every branch-dependent cache or elaborator effect. Audit external references and residual-cache identities; semantic caches need valid branch keys, while global work charges remain outside rollback. Snapshot creation cost does not measure the memory retained by a broad persistent frontier. External process snapshotting and in-process persistent contexts have different cost regimes. N5’s supplied snapshotting study is a relevant tactic-search precedent [64], not evidence for the cost of this proposed in-process frontier.

Next use the dependency graph for goal pressure and blocker wakeups, then memoize closed contextual successes, then introduce certified independent commits and separator tables if repeated work justifies them. Fail-first selection can estimate the applicable rule domain from retrieval, with ties favoring assignments that constrain other goals, and aging prevents starvation. Apsy and Canonical are useful comparisons [3, 6]; a dependent goal’s domain is an estimate, not an exact finite set supplied by those systems. The historical initial weights (exact 0, constructor/local application 1, provider 2, split 3, recursion 4, accumulator choice 5) remain hypotheses to measure under the zero-cost safeguards, not a proved optimal cost model.

A bottom-up companion remains a distinct option: enumerate small typed terms over observation environments and offer components to top-down holes, following the broad ideas of Escher and Burst [12, 5]. Finite observational signatures may organize buckets, but retaining only one representative needs a contextual congruence argument for the admitted grammar or a way to recover discarded alternatives. Observational agreement is not unrestricted program equality. Grammar learning and divide-and-conquer enumeration remain useful comparisons [9, 13, 33], not replacements for these checks.

Measure solved count at 1, 10 and 60 seconds, first certified answer, best fixed objective at equal logical work, proof debt, frontier memory, repeated work, component sizes, cache replay cost and nodes off the successful path. Compare the same grammar with and without goal ordering, factorization, heuristics and learning. Replace the impossible demand of zero answer-set variance under $\pm 30\%$ speed changes with equal outcomes at equal logical checkpoints; report time-dependent prefix differences and actual latency separately. The incoming Python countermodels illustrate failed bounds and deadline claims. They are not native Lean validation or performance measurements.

6.7 New questions for experts

The historical questions have the conditional answers above. The next questions concern the missing constructive analyses and matched experiments.

- R1.N1.** What conservative read/write analysis covers Lean’s delayed assignments, local instances and elaborator effects cheaply enough to justify component factorization, and which workloads repay separator joins and retained frontier memory?
- R1.N2.** Which finite relaxation admits a rule-by-rule cost simulation for the actual final candidate objective, including sharing and normalization, while providing enough pruning to repay its computation?
- R1.N3.** Which deterministic learning signals improve held-out synthesis at equal logical work without destabilizing fixed-objective ranking under irrelevant renaming, provider-order changes and correlated examples?
- R1.N4.** What resumable checkpoint and interruption protocol can reproduce native search, proof services and parallel proposal admission at equal logical prefixes with acceptable latency and memory overhead?

7 R2. Carriers, accumulators and auxiliary state

7.1 The problem is a representation together with its operations

A Church list exposes its fold, $\mathbf{xs} : (\mathbf{R} : \mathbf{Type}) \rightarrow (\mathbf{A} \rightarrow \mathbf{R} \rightarrow \mathbf{R}) \rightarrow \mathbf{R} \rightarrow \mathbf{R}$; using it requires choosing R . Ordinary recursors expose the same choice through their motive. When the requested result is already the fold result, unification may infer that type. Otherwise the useful representation need not occur in the query. This motivates carrier invention, but does not establish that every bounded search failure is caused by a missing carrier: transition synthesis, finishing, proof and search allocation must be measured separately.

For a list function $h : \text{List } A \rightarrow T$, search a *package*

$$(K, z, s, o), \quad z : K, \quad s : A \rightarrow K \rightarrow K, \quad o : K \rightarrow T, \quad h(xs) = o(q(xs)), \quad (3)$$

where $q([]) = z$ and $q(a :: xs) = s(a, q(xs))$. A direct fold has $K = T$ and $o = \text{id}$; a fold with a general finisher is a different problem. The package may depend on fixed query parameters, but must not capture the major input xs outside its fold. Otherwise $K = \text{Unit}$ can ignore the fold while its finisher computes the answer from the captured input. Do not allow calls to the unfinished target except through approved recursion capabilities (N4's scope restriction).

Even without that capture, $K = \text{List } A$, $z = []$, $s = \text{cons}$ and $o = h$ always reconstruct the input and defer the entire problem to the finisher. Charge and bound *all* package components, required helpers and proofs. State cardinality, carrier-type syntax, executable description length, proof cost, runtime and storage are different objectives. A natural number can encode a pair of natural numbers; a short function type may contain closures retaining input-sized data. Neither makes the corresponding encoding cheap or useful.

Task or scheme	Useful representation	Information or operation supplied
map , append , filter	result type	direct fold
right-fold	$T \rightarrow T$	delayed accumulator
implementation of foldl		
right-associated nonempty reduction	Option A	empty/nonempty phase
left-associated nonempty reduction	Option $A \rightarrow A$	phase and continuation
maximum with a default	Option A	seen element and comparison
minMax	Option $(A \times A)$	two extrema and empty phase
index with supplied default	$\text{Int} \rightarrow A$	consume an index through the fold
fib	$\mathbb{N} \times \mathbb{N}$	adjacent recurrence values
efficient reversal/tree flattening	$\text{List } A \rightarrow \text{List } A$	continuation/difference list

These are templates, not semantic necessities or new measured successes. In particular, do not assume that an optional element alone implements an arbitrary nonassociative *left*-associated operation through a right fold. The historical targeted $T \rightarrow T$ rule and the cost of the blind carrier frontier motivate keeping useful specializations early in a general grammar; uniform enumeration need not erase them. The same problem appears as motive generalization, tupling, invariant strengthening and helper invention: each proposes information whose specification is not explicitly supplied.

7.2 Minimal semantic state and what samples cannot identify

The classical direct-fold characterization requires that the kernel of h respect **cons**: equal results on tails remain equal after prepending an element [37]. For a fold with hidden state and a finisher, the more appropriate relation is

$$x \sim_h y \iff \forall u : \text{List } A, \quad h(u ++ x) = h(u ++ y). \quad (4)$$

The contexts *prepend* a list because a right fold has already processed the tail. Reversing this orientation changes the state problem.

Theorem 7.1 (Residual quotient and reachable state). *For every realization (3), $q(x) = q(y)$ implies $x \sim_h y$. The quotient by $\sim_h$ supports a realization of h , and the reachable states of any realization map surjectively onto that quotient. In particular, r pairwise inequivalent tails require r distinct states.*

Proof. Equal states remain equal after applying the same sequence of prepend transitions; induction on u gives equal states on $u ++ x$ and $u ++ y$, and the common finisher gives equal outputs. For the quotient, choose $z = []$, $s(a, [x]) = [a :: x]$ and $o([x]) = h(x)$. The step is well defined because a test with prefix u on $a :: x$ is a test with prefix $u ++ [a]$ on x ; the empty prefix makes the output well defined. Induction gives $q(x) = [x]$. Sending a reachable state $q(x)$ to $[x]$ is well defined by the first implication and reaches every class. $\square$

This is a semantic characterization, not an effective minimal-type algorithm. The quotient can be infinite, its equality undecidable, and its useful representation absent from the chosen grammar. The finite lower bounds use only finite counting; stronger comparisons of arbitrary infinite cardinalities need their set-theoretic assumptions stated separately. Isomorphic representations need not have the same syntax or operational cost.

Finite samples do not determine this universally quantified relation in general. Over a nonempty alphabet, for any finite collection of input words choose an unobserved word w . Constant zero and the function returning one precisely on w agree on zero-valued samples, yet only the first has a one-state deterministic realization. Both functions are total and computable. Thus samples do not determine even whether the intended function requires more than one state.

7.3 Two constructive bounded problems

Theorem 7.2 (Minimum finite machine consistent with finite samples). *Let A, B be finite explicitly enumerable sets with decidable equality and B nonempty. A finite consistent sample set $D \subseteq \text{List } A \times B$ admits a terminating procedure that finds a minimum-state deterministic right-fold machine fitting D .*

Proof. Let S contain $[]$ and all suffixes of sample inputs. Use states $S \cup \{\perp\}$, start at $[]$, and set $s(a, x) = a :: x$ if $a :: x \in S$, otherwise $\perp$; the sink stays at $\perp$. Processing a sample from right to left stays in S . Assign its specified output to each sample state and a fixed $b_0 \in B$ elsewhere. Consistency makes this well defined and gives an upper bound $|S| + 1$ on state count. Equivalently, construct a trie of reversed sample words and add a sink.

For $k = 1, \dots, |S| + 1$, enumerate every transition table $A \times \text{Fin } k \rightarrow \text{Fin } k$ and output table $\text{Fin } k \rightarrow B$, fixing the seed to zero. Any nonempty finite machine can rename its initial state to zero. Every sample check terminates; the first successful k is minimal and the upper bound ensures a success. $\square$

With $a = |A|$ and $b = |B|$, there are $k^{ak}b^k$ tables at fixed seed, or $k^{ak+1}b^k$ with all labeled seeds. These different conventions explain otherwise different incoming enumeration counts. The theorem establishes decidability, not interactive tractability. Direct backtracking with early sample propagation is a Lean-only option; SAT or finite-domain solving, symmetry breaking and partition refinement are alternative accelerators. One encoding uses states q_x , transitions $d_{a,k}$ and outputs o_k with

$$q_{a::x} = d_{a,q_x}, \quad o_{q_x} = h(x)$$

where observations exist, preserving consistency whenever states are shared. A satisfying finite table still needs a permitted typed emitter, finite enumeration/equality operations and native checking. It is not automatically expressible within an arbitrary small step grammar.

Bounded typed packages. Fix finite type and term productions, providers, literals, universes, helpers, recursion schemes and proof choices, together with a total syntax grade for (K, z, s, o) . Enumerate in a fixed objective order, type-check and validate every observation, and prune only by valid necessary conditions. If the resulting space $\mathcal{P}_b$ is finite and each check terminates decisively, this returns a minimum sample-consistent member or proves none exists in $\mathcal{P}_b$: every member is visited, no satisfying member is pruned, and the first success is least in the chosen order. These are all needed premises. For arbitrary Lean proof or reduction attempts, retain unknown outcomes; “enumeration exhausted with unresolved checks” is not semantic absence (Section 3). Universal contracts still require their actual proofs.

Bounding only type syntax does not give this procedure. For prescribed $K = X$ and target Unit, even the single empty-list observation has a realization exactly when X has a closed inhabitant: the seed supplies one, and conversely a chosen seed can be preserved by the step and discarded by the finisher. Conversely an unbounded finisher $\text{Unit} \rightarrow T$ already contains arbitrary inhabitation of T . These reductions from N8 and N9 show why type enumeration alone cannot decide arbitrary package existence.

7.4 Sound sample lower bounds and their counterexamples

Build a witnessed incompatibility graph on selected tails. Join x and y only when a common prefix u has two known, different outputs for $u ++ x$ and $u ++ y$.

Proposition 7.3 (Observed-context capacity bound). *Any deterministic package fitting these observations assigns different states to adjacent tails. For finite K ,*

$$\chi(G) \leq |\text{Reach}(K)| \leq |K|.$$

A clique of size r therefore refutes a carrier with fewer than r possible reachable states.

Proof. If $q(x) = q(y)$, processing the common prefix gives equal states and equal finished outputs, contradicting its witnessed unequal observations. Hence $x \mapsto q(x)$ is a proper coloring by reachable states. $\square$

The certificate consists of the actual conflicting observations and a proved capacity bound, or an inductively closed invariant bounding the reachable state set for *every* admitted program in the claimed class. A clique is cheap to check. A general chromatic lower bound needs its own evidence; the number of colors found by greedy coloring is an *upper* bound on $\chi(G)$, not a refutation certificate. Even a correct coloring need not extend to a deterministic transition system: equal states must have equal successor states under the same constructor.

Three further mistakes recur in the incoming answers’ counterexamples.

- Partial rows $(0, ?)$, $(?, 1)$, $(1, 1)$ show that compatibility is not transitive. Unknown entries are neither outputs nor inequalities. Different partial rows may describe the same state; merging all compatible pairs or counting distinct strings of entries is unjustified.
- Values found by a depth-limited enumerator are not an upper bound on inhabitants or reachable states. A short transition can build arbitrarily large values when iterated. Refuting `Nat`, lists or functions because only two values were found is unsound without a separate bounded-image proof.
- A direct-fold obstruction does not exclude the same carrier with a finisher. For total list tail, $h([]) = h([b]) = []$, but after prepending a , $h([a]) = [] \neq [b] = h([a, b])$. This excludes a direct fold whose state is $h(xs)$. The same list carrier with $q = \text{id}$ and $o = \text{tail}$ works, so the restriction on finishing code is essential.

Three states for a Boolean result. For “length divisible by three”, tails of lengths 0, 1, 2 under prefix lengths 0, 1, 2 have rows $(1, 0, 0)$, $(0, 0, 1)$, $(0, 1, 0)$. They form a three-clique. A cyclic three-state counter with output true at zero attains the bound; induction identifies the state with length modulo three. Thus `Bool` is too small even though the output is Boolean; `Option Bool` supplies three values. Unary fixed-seed table enumeration uses 2, 16, 216 tables at sizes one, two and three; changing the alphabet or allowing every labeled seed changes these counts. N1, N3, N4 and N9 contain variants of this finite illustration.

Four states from nine distinct observations. Let h mean that a word over $\{a, b\}$ contains both symbols. The tails $[], [a], [b], [a, b]$ under prefixes $[], [a], [b]$ yield

tail	$[]$	$[a]$	$[b]$
$[]$	0	0	0
$[a]$	0	0	1
$[b]$	0	1	0
$[a, b]$	1	1	1

Every pair is incompatible. The twelve cells use nine distinct input words; the stronger sample set of all fifteen words of length at most three contains them. Four states suffice: retain two seen-symbol flags, set the relevant flag on each step and finish with conjunction. An induction establishes the flag invariant on all words. Fixed-seed enumerations of the smaller binary machines have 2, 64, 5832 tables; including all seeds gives 2, 128, 17496. N2, N5 and N7 use different sample extensions and witness-test bounds. These are compatible finite experiments, not interchangeable receipts. Their Python checks corroborate the construction, not native Lean synthesis, universal correctness or performance; the mathematical induction supplies the universal argument. N4’s binary parity example similarly needs two states, demonstrating that these bounds concern the particular observed behavior.

A minimum sample machine can still overfit. N8’s evaluation section supplies a distinct constructive example. For the unary target “length exactly two”, initially observe lengths zero through five. Suffix lengths 0, 1, 2, 3 form a four-clique: each pair has an observed common prefix taking exactly one member to length two. The four-state cycle $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$, with output true only at state two, attains this sample minimum but returns true incorrectly at length six. After adding the length-six counterexample, the machine $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3$ with the same output convention still uses four states and fits the strengthened samples. Its reached state is $\min(n, 3)$ by induction, proving the intended behavior at every length. Thus certified state capacity, minimum sample consistency and identification of the intended function are three different claims even in a tiny exact finite language. The extra counterexample comes from the

experiment’s reference oracle, which an arbitrary example-only query does not automatically provide.

Active automata learning supplies useful observation-table ideas, but Angluin’s membership and equivalence oracles are stronger than a fixed sample set [39]. Do not assume missing table entries, closure or consistency have been learned. Use state constraints to suggest options, products or continuations; the table is not a decoder of an intended Lean type.

7.5 Polymorphic carriers: shapes, positions and available observations

The correct reduction is conditional on a relational or naturality theorem, not merely a polymorphic-looking Lean type. The incoming reports connect polymorphic testing [4, 36] with Mulleners, Jeuring and Heeren’s container-based realizability work [42]. N9 supplies the general construction; the list instance used by the other answers follows from it.

Theorem 7.4 (Shape–position representation). *For containers $F(A) = \sum_{s \in S} (P_s \rightarrow A)$ and $G(A) = \sum_{t \in U} (Q_t \rightarrow A)$, a natural transformation $\tau : F \Rightarrow G$ is determined by a shape map $\phi : S \rightarrow U$ and maps $\psi_s : Q_{\phi(s)} \rightarrow P_s$, with*

$$\tau_A(s, k) = (\phi(s), k \circ \psi_s).$$

Conversely these maps define a natural transformation.

Proof. Evaluate τ at the generic input (s, id_{P_s}) of element type P_s ; its result defines $(\phi(s), \psi_s)$. Naturality along $k : P_s \rightarrow A$ gives the displayed equation. Conversely, associativity of composition makes this equation commute with every element map. $\square$

For lists the shape is a length n and positions are $\text{Fin } n$. If $f_A : \text{List } A \rightarrow \text{List } A$ is natural, let $e_n = [0, \dots, n-1]$ and let $v : \text{Fin } n \rightarrow A$ select an input list’s entries. Then

$$f_A(xs) = \text{map}(v)(f_{\text{Fin } n}(e_n)). \quad (5)$$

The output word records deletion, duplication or permutation of positions. Repeated elements are covered because v need not be injective. At $n = 0$ the position type is empty and its list result is empty; no nonexistent function $\mathbb{N} \rightarrow A$ or arbitrary element of an empty type is assumed. Use appropriate universe lifts at a higher fixed universe. For multiple containers tag position sets by argument and retain specified cross-input sharing.

If two natural functions agree on e_n for $n \leq N$, applying (5) proves equality on every input of length at most N . This does not prove equality at greater lengths or for all programs of a syntax-size bound without a separate small-model theorem. Identity and the natural function that returns its input through length N and then returns $[]$ agree on those tests and disagree at $N + 1$.

Nor does quantifying over **Type** establish naturality for arbitrary Lean providers. Classical decidability permits a noncomputable family returning its input when A is subsingleton and $[]$ otherwise. Mapping a singleton list between one- and two-element types witnesses failure of naturality. An axiom-profile check alone is not a parametricity certificate.

With lawful equality, arbitrary maps that collapse elements need not preserve behavior; bounded inputs require realizable partitions of positions, not just the all-distinct case. Orders, hashing, algebraic dictionaries, distinguished elements and callbacks expose further structure. A raw BEq dictionary need not be lawful; retain its operational semantics until a preservation law permits replacing it with mathematical equality. Nominal or register-style models can separate finite control from stored atoms and symbolic payloads; nominal automata provide a relevant semantic

precedent [66], but reversing or copying arbitrarily long lists may require unbounded storage. Finite symbolic alphabets do not imply finite concrete state.

A practical provider classification is token-preserving, shape-observing, equality-observing, order-observing or opaque. Certify the admitted fragment and providers before using generic positions for permanent pruning or merging. Without that theorem, symbolic observations remain useful tests and proposal features. Indexed tokens additionally carry their index and typing telescope; renaming must preserve fibers, and caching modulo token renaming needs an evaluator invariance theorem.

7.6 Constructive fusion, feature discovery and closure

The original claim that auxiliary-state synthesis has no relevant published algorithm is too broad. AutoLifter synthesizes auxiliary summaries and combinators from a reference computation and a prescribed algorithmic paradigm; its decomposition can choose necessary conditions with unrealizable later subtasks. It is neither a global minimal-type method nor a complete sparse-example learner [40]. The incoming reports also identify Eguchi and colleagues' auxiliary-type inference, Para's recursion templates, and Synduce's unrealizability witnesses as useful but differently scoped precedents [43, 44, 47]. Tupling and homomorphism calculation remain relevant [35, 32].

The earlier comparisons remain useful when their task boundaries are retained. λ^2 propagates examples into a fixed combinator vocabulary; its fold deduction uses tail-complete observations and a result-typed accumulator [19]. Myth, Smyth, Burst and Trio investigate explicit recursion, trace information, live evaluation, angelic results or construction from nonrecursive blocks [24, 28, 5, 34]. Those mechanisms do not by themselves constitute the joint carrier/package learner specified here. Synquid's supplied auxiliary refinements and DreamCoder's abstractions learned across a corpus address different sources of missing specification [30, 10]. These distinctions identify adaptation opportunities; they do not support the original universal claim that synthesis research never invents auxiliary state.

An evidence-dependent constructive fold. Weber and Caldwell's constructive work qualifies the idea that fold characterization is purely nonconstructive [41]. The following sufficient condition from N6 makes the supplied evidence explicit.

Proposition 7.5 (Fold construction from a section). *Let $h : \text{List } A \rightarrow B$ have a section $r : B \rightarrow \text{List } A$ satisfying $h(r(b)) = b$. If $h(x) = h(y)$ implies $h(a :: x) = h(a :: y)$, then*

$$h = \text{foldr } s \ z, \quad z = h([]), \quad s(a, b) = h(a :: r(b)).$$

Proof. The base case is immediate. Since $h(r(h(x))) = h(x)$, cons congruence gives $h(a :: r(h(x))) = h(a :: x)$. Substitute the induction hypothesis for the fold of the tail to obtain the constructor equation. $\square$

This consumes h , a section and a congruence proof; it does not discover them from samples. A section on the reachable image, with its typing and closure obligations explicit, can weaken the unnecessarily global surjectivity premise. For a polynomial functor the corresponding construction applies $F(r)$ before the constructor and h , with the analogous congruence and induction premises.

Feature separation is only the first stage. Choose a bounded typed feature library $\phi_j : \text{List } A \rightarrow S_j$: emptiness, optional first/last element, length modulo a small modulus, flags for a concrete alphabet, or available measures. Features must use only the actual operations of the query; abstract elements cannot be compared with an unsupplied distinguished value. Each

feature separates some observed incompatibility edges. Weighted set cover proposes a small set J and

$$K_J = \prod_{j \in J} S_j, \quad \phi_J(xs) = (\phi_j(xs))_{j \in J}.$$

Now synthesize the seed, updater and finisher with the closure equations

$$\phi_J([]) = z, \quad \phi_J(a :: xs) = s(a, \phi_J(xs)), \quad o(\phi_J(xs)) = h(xs). \quad (6)$$

Induction on xs proves $\text{foldr } s \ z \ xs = \phi_J(xs)$ from the first two equations; the third gives the requested result. For a partially specified task retain the actual contract instead of inventing a reference equality. Finite observations may suggest the equations but do not prove their universal versions.

If two tails agree on selected features but need different successor features after the same constructor, add a closure-distinction obligation and seek more information. A feature set that separates outputs may still be impossible to update cheaply. Retain alternative sets and backtrack; greedy feature selection is a heuristic, not a complete lifting procedure. Exhaustively enumerate bounded feature subsets, algebras and finishers with complete checking for a relative completeness theorem. Preserve unknown proof alternatives. This is the constructive separation-and-closure procedure emphasized by N2, N3 and N7.

Alternatively propose a relation $I(xs, k)$ describing stored information and prove initialization, preservation and finishing:

$$I([], z), \quad I(xs, k) \Rightarrow I(a :: xs, s(a, k)), \quad I(xs, k) \Rightarrow C(xs, o(k)).$$

The invariant is itself a synthesis proposal. A reference computation may generate counterexamples; sparse contracts provide only their stated observations. For $F(X) = \sum_c A_c \times X^{r_c}$, each constructor receives an algebra $A_c \times K^{r_c} \rightarrow K$, using the summaries of *all* recursive children. The general closure equation is $H \circ \text{in} = \alpha \circ F(H)$; for a lifted $H = \langle h, b \rangle$, projecting the fold gives h . A fixed carrier and bounded constructor algebra give a finite synthesis class. Polynomial functors alone do not make unrestricted helper and invariant invention decidable.

7.7 Worked representations and scheme-dependent invariants

Reversal is already a list-valued fold. All nine answers correct the semantic use of reversal in the original proposal:

$$\text{reverse}(xs) = \text{foldr } (\lambda a \ r. r ++ [a]) \ [] \ xs.$$

The induction step is exactly the defining reversal equation. A shallow or append-free grammar may not express the step, but that is a language restriction, not a capacity obstruction. Under ordinary singly linked lists, the successive append traversals cost $0 + 1 + \dots + (n - 1) = \Theta(n^2)$. For the continuation representation set

$$K = \text{List } A \rightarrow \text{List } A, \quad z = \text{id}, \quad s(a, k) = \lambda \text{acc}. k(a :: \text{acc}), \quad o(k) = k([]).$$

Induction proves $q(xs)(\text{acc}) = \text{reverse}(xs) ++ \text{acc}$: the step uses the induction hypothesis at $a :: \text{acc}$ and append associativity. With constant-time cons and closure creation, construction and final traversal of the closure chain take linear work. A left-fold scheme can also reverse linearly with an ordinary list carrier. These alternatives show why the scheme, operation grammar and cost model belong in the optimization problem.

Adjacent values, empty phases and defaults. For Fibonacci, $q(n) = (F_n, F_{n+1})$, seed $(0, 1)$ and transition $(a, b) \mapsto (b, a + b)$ close the recurrence; induction proves the invariant. This does not prove that no clever single-natural encoding exists. Optional maximum state avoids inventing an element before one has been seen, and an optional pair similarly handles extrema. A total maximum, nonempty reduction or index must preserve the supplied empty/out-of-range default or inhabitation assumption. Carrier invention cannot create an element of an uninhabited type. Difference lists for tree flattening avoid repeated copying under the same explicit operational assumptions used for reversal.

Maximum prefix sum: the recursion scheme changes the needed state. N9 supplies a useful distinction not captured by the earlier examples. For integer lists, let h be maximum prefix sum including the empty prefix. A right fold needs only an integer:

$$h([]) = 0, \quad h(a :: xs) = \max(0, a + h(xs)).$$

But the result alone cannot combine arbitrary adjacent segments: $[0]$ and $[-1]$ both give zero, while appending $[1]$ gives one and zero respectively. Lift to $H(xs) = (\text{sum}(xs), h(xs))$, with

$$(s, m) \otimes (t, n) = (s + t, \max(m, s + n)). \quad (7)$$

Every prefix of $x++y$ lies inside x or contains all of x and a prefix of y , so $H(x++y) = H(x) \otimes H(y)$. The reachable invariant is $I(s, m) : 0 \leq m \wedge s \leq m$.

Proposition 7.6 (Invariant-restricted summary monoid). *Operation (7) is associative on $\mathbb{Z}^2$, preserves I , and has two-sided identity $(0, 0)$ on states satisfying I .*

Proof. Both bracketings have first coordinate $s+t+u$ and second coordinate $\max(m, s+n, s+t+p)$. For invariant operands the second coordinate is at least $m \geq 0$ and at least $s+t$ because $n \geq t$. Left identity uses $\max(0, m) = m$; right identity uses $\max(m, s) = m$. $\square$

Left identity fails on the unrestricted pair $(1, -1)$. Demanding algebra laws for all syntactic pairs can therefore reject this correct reachable-state representation; assuming I without proving its initialization and preservation can validate an incorrect one. Carrier type, scheme and invariant are related search choices. “Minimal carrier” has no scheme-independent meaning here.

7.8 Motive enumeration and a shared strengthening service

Retain a short prioritized carrier grammar, for example

$$K ::= T \mid A \mid S_i \rightarrow K \mid K \times K \mid \text{Option } K \mid \text{Bool} \mid \mathbb{N},$$

then its indexed and dependent extensions when needed. A size-three frontier is an engineering starting point, not a theorem of coverage. Every proposal opens seed, algebra and finishing obligations together; a type that elaborates but cannot be initialized is not a solved representation. Charge the whole subtree and preserve specialized priorities such as $T \rightarrow T$.

For $f : \mathbb{N} \rightarrow S_1 \rightarrow \dots \rightarrow S_k \rightarrow T$, a constant recursor result $K = S_1 \rightarrow \dots \rightarrow S_k \rightarrow T$ lets the successor branch apply its predecessor result at newly computed parameter values. This is a complete schema for the declared nondependent primitive-recursive fragment provided its entire base and step language is enumerated. It is not a small complete procedure for every useful dependent primitive recursion.

Separate existing-parameter generalization, result strengthening, index abstraction and recursion nesting. A dependent template is

$$M(\vec{v}, x) = \prod_{j \in J} (a_j : S_j(\vec{v}, x, \vec{a}_{<j})). K(\vec{v}, x, \vec{a}),$$

with J dependency-closed and binders in valid telescope order. Dependent accumulators need Π rather than ordinary arrows; coupled values and proof-carrying summaries may need Σ rather than plain products. Type-check motives before opening all method searches, and deduplicate only with justified equality, not a display-level index simplification.

Dovetail over generalized binders, carrier size, total term size, helper count, active recursion frames and proof/universe bounds. One possible joint grade charges $|K| + |M| + |z| + |s| + |o|$ plus recursive-scheme introductions. Fair expansion covers the union of the declared finite grammars under the appropriate checking assumptions, not all Lean inhabitants at an interactive deadline. Keep outermost natural-number recursion and existing arithmetic providers early, use observed constants and a small literal basis first, and charge nested eliminators separately. Permanently excluding later schemes or numerals loses the corresponding completeness claim. A beam or a fixed top- k carrier list needs a fair fallback retaining its omissions.

Failure-directed strengthening. Carrier choice, motive generalization and helper invention should share a bounded service that consumes structured failed obligations. Record the available induction hypothesis, required recursive application, changed parameters, unmatched indices, blocked equations and distinguishing residual observations. Generalize a changing accumulator and its dependencies; add a missing statistic as a product; add an empty phase tag; propose a typed helper for a repeated fragment; or retrieve an algebraic reassociation lemma when that is the actual missing step. A helper that merely adds another unsolved function is not useful until both output recovery and closure become easier.

Use typed anti-unification and rippling-inspired comparison to generate a small ranked menu, preserving the original branch. Tests reject false conjectures cheaply; survivors remain conjectures until native proof and policy checking. HipSpec and Hipster supply theory-exploration precedents [45, 46], not a turnkey Lean import established by these reports. Induction with generalization in superposition reasoning is another supplied precedent for strengthening induction problems [65]; transferring its first-order inference machinery requires a separate typed Lean design. Canonical's supplied generalized induction example demonstrates predicate discovery with a suitable principle available [6]; discovering that principle itself is a separate task. If a guessed helper fails, backtrack its consumers. Record environment and axiom profile before reusing a proved lemma across tasks.

7.9 Evaluation and new expert questions

Use at least three matched tracks: carrier supplied, carrier selected from a fixed template grammar, and carrier inferred from failures or observations. Add an efficiency track when runtime/storage, rather than merely sample consistency, is the objective. Hold the original contract, defaults, allowed providers, universe policy and body grammar fixed across relevant comparisons. Compare blind and distinction-guided ordering of the same representation grammar; measure subtree size and time per carrier, transition/finisher search, proof debt and final code cost. Scheme selection, motive discovery, body construction, universal proof and executable publication are separate results.

The historical thirteen Church stretch cases and accumulator tasks such as `fib`, `list_rev_tailcall`, `tree_flatten`, `maximum` and `index` remain useful evaluation families. The original goal of solving them within ten seconds without new task-specific rules and without more than a factor-two slowdown of solved Church probes is a prospective acceptance target, not a claim supplied by the incoming finite models. No exact modern accuracy or speedup follows from their Python checks.

The historical five expert questions now have the conditional answers and explicit limits above. The active next questions are:

- R2.N1.** Can a Lean-native carrier proposer emit and cheaply check distinguishing-context certificates against finite capacity or a proved reachable-state invariant, while sharing work with bounded algebra synthesis?
- R2.N2.** Which provider-certified parametric fragment supports a proved shape/position or equality-pattern reduction for higher-order and indexed queries, and can a useful small-model bound connect program size to test depth?
- R2.N3.** Can failure-directed feature separation and closure synthesis retain relative completeness while materially reducing search over the same carrier, updater and finisher grammar on sparse-contract tasks?
- R2.N4.** What joint grammar and cost model can select recursion scheme, dependent motive, carrier and reachable-state invariant without pushing the original computation into an unrestricted finisher or helper?
- R2.N5.** Which structured induction failures reliably distinguish a missing parameter, missing state component and missing algebraic lemma, and how much does that diagnosis improve discovery after charging its extra proof debt?

8 R3. Evaluation of programs with holes

8.1 The problem and revised boundary

The proposal’s measured starting point is repeated residual-contract evaluation: substitute the partial program into an observation, reduce its decision procedure, reject a decided false result, and continue when reduction is stuck. The historical profile attributed 45% of contract-query time to this work; that is a workload measurement, not a prediction for the designs below. Repeated folds restart; results and blockers are aggregated; backward demands are unavailable; and unfinished recursive calls often block reduction. Independent observations would serve search scheduling (Section 6) and guard synthesis without changing the contract.

Responses N1–N9 correct the starting literature description. Smyth’s simplified formal core is first-order, but its implementation supports higher-order examples and polymorphism, interdependent holes, and examples that are not trace-complete[28]. Its scope must not be reduced to its core exposition. The research boundary is a dependency-correct adaptation to Lean. A correct kernel primitive does not certify arbitrary preprocessing of an open term; closed differential tests do not prove that partial-state pruning preserves every completion. The guarantees and evidence categories of Section 3 apply throughout.^[N1–N9]

8.2 Typed closures and relational unevaluation

Hazelnut Live supplies semantics with typed hole closures; Smyth resumes computations and propagates required results backward. Scribe supplies a later example-propagation reference. Combinator-specific inversion in Myth and λ^2 , and the interpretation of examples as refinements, give further implementation choices[15, 28, 50, 24, 19, 14]. None supplies a ready-made conservative evaluator for all native Lean expressions, universes, delayed assignments, and dependent conversion.

A hole declared $m : (\Delta \vdash T)$ is encountered in context Γ through a well-typed substitution $\rho : \Gamma \rightarrow \Delta$. Its closure records

$$H = (m, \Delta, \rho, T[\rho], \kappa, \nu),$$

where κ contains suspended eliminations or typed observations and ν identifies the immutable state version. All occurrences share one contextual unknown. Evaluating `map ?f [a, b]` can preserve the list spine and retain two closures for the same f , not two independently fillable

holes. A vector-producing hole observed at lengths zero and three has constraints in different fibers. Preserve its substitutions, equality evidence, and transports; erased lists or host-language equality cannot replace them. Proofs and dictionaries can affect legal reduction even when proof arguments are erased at runtime.^[N1–N9]

Higher-order examples are finite trees of demanded applications: a function-valued result must satisfy a relation when applied to particular typed arguments. Church inputs are observed through applications to eliminators. This is not an extensional oracle at unobserved inputs, nor a reason to merge functions that agree on the current table. An unobserved application stays neutral unless another justified rule applies.

Unevaluation returns a relation on completions, not necessarily one value. Constructor injectivity turns `cons ?x ?xs = a :: ys` into necessary field constraints. For noninjective g , $g(?x) = y$ requires the original equation, a proved preimage rule, or alternatives covering all possibilities. An unknown conditional yields both guard/branch alternatives; an application with unknown function and argument may remain a joint constraint. Conditions such as $f(0) = f(1)$ must preserve correlations between calls. Choosing one sufficient explanation is useful guidance, but its failure refutes the branch only if the alternatives collectively cover every satisfying completion.^[N2,N3,N5,N6,N8,N9]

Certified observation extraction. For each extracted $O_i(p)$ retain $C(p) \rightarrow O_i(p)$ as in Equation 2. Refutation needs this direction; acceptance by assembling observations also needs $(\bigwedge_i O_i(p)) \rightarrow C(p)$ or another reconstruction theorem. Register justified decompositions of conjunctions, Boolean conjunction required to be true, **decide** wrappers, and **List.all** over a fully known finite list. A failed disjunct does not refute a disjunction. Infinite quantification remains symbolic unless a sound reduction applies.^[N1–N9]

An arbitrary BEq dictionary is computational data. Replacing `x == y = true` by $x = y$ requires the appropriate lawfulness theorem; otherwise keep the exact Boolean test, including unconventional instances. Multiple calls in a relational observation stay coupled. Give each observation its own identity, decider when applicable, true/false/unknown status, residual, and dependencies. New counterexamples add identities rather than silently changing existing cache entries.

Fragment progression. Start with typed first-order observations on Booleans, naturals, lists, and products; admit lambdas, applications, let bindings, constructors, projections, known-constructor cases, selected constant-motive recursors, and named pure primitives with checked semantics. Then add higher-order applications and indexed constructors with explicit fibers, before arbitrary dependent eliminators. Constructor disagreement in a closed prefix may refute while a tail is open; a match blocked on an unknown index normally suspends. Each extension needs substitution stability and rollback tests. Unsupported inversions retain residual constraints or fall back; they never reinterpret a value in the wrong fiber.^[N1–N9]

8.3 Assignment-driven normalization and rollback

Self-adjusting computation, Adapton, and named incremental computation give methodology for demand tracking and *from-scratch consistency*: reuse must agree with reevaluation in the current branch. Canonical-min gives a more direct control precedent, suspending constraints on metavariables, resuming on assignment, and supporting push/pop search state. These results concern their own calculi; they do not certify a full Lean normalizer. The reviewed Canonical-min account leaves full mechanization as future work. [48, 49, 63, 38]^[N1–N9]

Use a residual computation DAG with immutable expression/closure nodes, values or residuals, read sets, and dependency edges. Cache identity includes the frozen observation, evaluator

version, exact expression, local telescope and substitution, environment, transparency/rewrite policy, and relevant mutable reads. Reads include already assigned expression metavariables, universes, delayed-assignment links, local declaration types and values, instances, options, and equation/inversion lemmas whenever consulted. Final unassigned blockers alone miss assigned values that chose a reduction branch. Dynamic read sets may omit an unused branch of a known Boolean match and so be more precise than whole-term syntactic dependencies.^[N1–N9]

Proposition 8.1 (Read-set cache validity). *Suppose a deterministic evaluator has fixed immutable semantic inputs and records a sound over-approximation of every mutable value read. If those inputs and all recorded values agree in another state, its cached result agrees with reevaluation there.*

Proof. Replay the finite evaluation trace. Every read returns the same value, so the same pure operation and next read are selected. Induction gives the same result and termination point. An omitted dependency invalidates the hypothesis; an extra dependency only causes unnecessary recomputation. $\square$

The minimal counterexample evaluates $?m + 1$ with $?m := 0$, rolls back, and assigns $?m := 1$: the second answer is 2, not the cached 1. Equivalently, a false result for $?m = 1$ in the first branch must not reject the second. Both states contain an assigned hole and may reuse a rolled-back numerical generation. This ABA identity error is a model counterexample, not a reproduced Leant2 defect. Use immutable assignment nodes, persistent deltas, validated content identities, or globally fresh stamps. A fresh global timestamp invalidating everything is safe but sacrifices reuse. Hashes index entries; correctness-sensitive collisions require structural confirmation.^[N1–N9]

Watchers enqueue affected observations on assignment. Rollback restores the persistent branch-store root or revalidates read fingerprints; mutable watcher lists must also restore or be indexed by branch lineage. Share closed subexpressions in the frozen environment, and open residuals only through validated closures. Historical inspections found `residualBlockers` in an `IO.Ref` outside `Meta.SavedState`; that motivates ownership tests, not an asserted false-result bug. In contrast, resource charges intentionally survive rollback and are never refunded.

Cache unknown carefully: a stuck result is reusable only while its reads agree; a timeout is a cost estimate, not evidence that a larger allowance cannot decide it. Global false caching requires a certificate or an identity establishing the same proposition, program, assumptions, and policy. Contract-relative oracle facts must not become unconditional cache values.

Implement exact closed-result caching first, branch-local residuals next, and cross-branch reuse only after assignment-version and delayed-substitution tests. Measure hits, invalidations, recomputation, demand-queue work, closure retention, and old-world memory. A large hit rate can still lose overall when persistence retains too many environments.^[N4–N9]

8.4 Checked evaluation before a new normalizer

N4, N6, and N7 add an existing backend to the common small-fragment proposal: benchmark Lean’s proof-producing CBV machinery. The inspected Lean 4.34.0 source exposes `Lean.Elab.Tactic.Cbv`, `evalDecideCbv`, `cbvGoal`, `cbvHyp`, and `cbvDecideGoal`. The `decide_cbv` wrapper applies `of_decide_eq_true` and invokes the CBV goal checker. Equation-based propositional rewriting handles useful closed observations and well-founded definitions without code-generator trust of the kind used by `native_decide`. Its proof dependencies still require the selected axiom audit, and documented dependent-position limitations prevent treating it as a complete open-hole evaluator. Source availability is not a performance or compiled integration result.^[52, 53]

Kernel conversion and proved equations are different specifications. A checked rewriter may prove $e = v$ without conversion at the chosen strategy reducing e to v . This is useful semantic evidence, not an under-approximation of conversion. If conversion is the specification, restrict the fragment to kernel-convertible reductions. Compare current kernel reduction, ordinary proof-producing simplification, CBV, and the custom fragment separately.

Proposition 8.2 (A sufficient open-refutation certificate). *Let a partial program p be abstracted over a well-typed dependency-ordered completion telescope Δ , and let every allowed completion satisfy D . A checked certificate*

$$\forall \theta : \Delta, \quad D(\theta) \rightarrow \neg C(p[\theta])$$

justifies removing the branch. Additional assumptions are permitted only when every contract-satisfying completion entails them.

Proof. A hypothetical accepted completion supplies its typed telescope values and the required assumptions, yielding a contradiction. In the conditional case, the hypothesis that it satisfies C supplies the additional assumptions. $\square$

A Boolean certificate $\text{decide}(O(p[\theta])) = \text{false}$ for a necessary observation is one instance. A proof about one arbitrary filling rejects only that filling; successful simulation of closed programs is weaker than this open obligation. This instantiates Equation 1.^[N1–N9]

A *substitution-stable reducer* proves that each permitted reduction commutes with every well-typed completion; substituting into a false-ending trace preserves false. An *abstract evaluator* instead proves

$$\{\llbracket O(p[\theta]) \rrbracket : \theta \text{ allowed}\} \subseteq \gamma(\widehat{\text{eval}}(O, p)).$$

If this over-approximation has no permitted outcome, the branch is refuted. Under-approximating possible behaviors is unsafe for exclusion. Under-approximating *proved judgments* is safe: more answers are unknown, while each reported decision still has evidence.

The reports identify a specific proof boundary in `instantiatePartial`: its reduction-only delayed-assignment expansion permits intermediate metavariables whose local contexts no longer match. This is not a demonstrated counterexample, but it prevents assuming ordinary well-typed substitution. Carry typed substitution spines or prove a relation from that representation to every admissible native completion. A correct `Kernel.whnf` invocation alone cannot establish the preprocessing relation.^[N1–N7,N9]

Deployment alternatives. First let fast custom evaluation change priority only. Next let it propose compact negative evidence checked before a prune. Finally prove a typed object language’s denotation and fuelled interpreter, or a verified abstract transformer. A proof-producing reducer composes step equalities and integrates incrementally; a reflected evaluator uses a shared soundness theorem and may lower checking overhead after a larger initial proof investment. Compact traces can be checked when committing a prune and certified summaries cached. These remain alternative engineering choices, not measured superiority claims. ^[N1–N9]

Listing 1: Proposed evidence-bearing evaluation interface; pseudocode.

```
EvalResult :=
  value      (typedValue, equalityEvidence, readSet)
| suspended (typedClosure, blockers, readSet)
| rejectHint (diagnostic, readSet)
| refuted    (checkedBranchCertificate, assumptions, readSet)
| resourceExhausted
```

Only **refuted** removes a branch in the conservative lane. Unsupported syntax, ill-scoped residuals, unproved side conditions, exhausted fuel, and failed certificate construction give unknown or propagate resource interruption, never false. Later acceptance checking cannot recover an earlier unsound prune.

Lean4Lean supplies a checker and mechanized metatheory, with verification work extending over time. It supplies identified definitions and proved lemmas, not automatic extraction of every later kernel optimization into a verified incremental evaluator. A new closure machine still needs its environment relation, substitution lemmas, and simulation theorem. [57, 58]

8.5 Angelic calls and the bounded guarantee

Angelic calls are existential results constrained by the specification. A sound relaxation admits every ordinary execution of every allowed completion. Unsatisfiability can then refute; a satisfying angelic execution is merely permission to continue. A finite list of guessed outputs under-approximates behavior unless it covers every realization. One failed guess, incomplete solving, or a timeout cannot establish universal failure.^[N1–N9]

Call identities include the function or recursion frame, full typed argument tuple, implicit computational parameters, universes, and relevant environment. Known-equal calls share outputs or compatibility constraints in the appropriate fiber. Independent angels can satisfy $f(0) \neq f(0)$: this is a loose over-approximation, not itself an unsound refutation, but cannot count as an actual program. Imposing equality on unequal calls, or silently bounding an infinite result domain, may instead remove a real completion. Syntactically different symbolic inputs are not necessarily unequal.

If $C(f) \rightarrow f(v) = w$ is established, the oracle may use w under C . A contradiction excludes contract-satisfying completions, not necessarily every arbitrary completion. Retain the implication in certificates and cache identities. If several outputs are permitted, retain their relation. Unobserved calls may remain unconstrained. Observations of $f(s) = g(s, a_0)$ do not specify the invented accumulator helper $g(s, a)$ at arbitrary a (Section 10.5).

Burst contributes strengthening, ordinary candidate checks, and backtracking; Smyth contributes a different live-constraint organization. Their guarantees do not transfer merely by changing search direction[5, 28]. The responses' bounded claims can be stated once as follows.

Theorem 8.3 (Finite checked refinement). *Fix a finite explicitly represented set F of well-typed concrete candidates, including bounds on bodies, literals, universes, helpers, instances, recursion schemes, and proof plans. Suppose validation terminates and decides the stated observations, selection is fair, refinements preserve every satisfying member, and each unsuccessful completed iteration removes its selected failing candidate without indefinitely repeating unresolved auxiliary states. Then refinement terminates with a satisfying member or correctly exhausts F .*

Proof. Satisfying members are never removed. Every unsuccessful completed iteration strictly decreases the finite remaining set, so at most $|F|$ such iterations occur. Fair terminating selection and validation eventually recognize a satisfying member or exhaust the set. $\square$

Another sufficient implementation bounds candidate–oracle pairs, uses an exact terminating constraint solver and well-founded recursive calls, and strictly removes an incompatible pair or failed candidate on each refinement. Finite syntax alone does not bound integer outputs, fresh calls, helper declarations, or refinement sequences. Incomplete unification, top- k retrieval without fallback, and proof attempts indefinitely returning unknown prevent exhaustion claims. An expanding grammar with fair dovetailing supports only a relative semidecision guarantee; fuel or wall-clock exhaustion stays unknown.

8.6 Validation and new questions

The implementation sequence is certified observation extraction, immutable reports and watchers, closed proof-producing backends, a typed fragment with checked negatives, then oracle constraints with certified pruning. Angelic guidance alone is a useful earlier milestone. A small bottom-up pool of typed closed realizations can supplement top-down holes; behavioral summaries index alternatives and new observations split or regenerate buckets instead of permanently discarding their contents.

Regressions must include open terms under multiple completions, repeated hole applications, universes, delayed assignments, indexed constructors, casts, proof-dependent types, local dictionaries, transitive reads, exceptions, sibling rollback, and reuse of conditional facts outside their assumptions. Compare cached and fresh evaluation. Closed generated-term differential testing supplements these checks; even 10^5 agreements do not prove the completion theorem. Charge certificate generation/checking and recomputation: a fast hint followed by a full replay at every node need not save time. The submitted Python cache/oracle models illustrate failure modes; they are neither Lean verification nor engine performance measurements.^[N1–N9]

The original four questions are answered above at their stated boundaries. The remaining research questions are more specific:

- R3.N1. What is the smallest typed residual fragment covering the measured hot observations for which substitution stability, delayed-assignment closure, and a proof-producing refutation path can all be established?
- R3.N2. Which complete read-set representation gives useful cross-branch reuse without retaining excessive old environments, and can its from-scratch consistency be proved for the actual Lean state interface?
- R3.N3. Under each permitted axiom profile, when do kernel reduction, equation- based CBV, reflected evaluation, and checked speculative traces improve end-to-end observation cost, including certificate construction and checking?
- R3.N4. Which finite relational oracle language preserves all bounded concrete recursive behaviors while supporting useful complete refutation, especially for dependent fibers and invented helpers outside their initial seed?

9 R4. Indexed families and the motive problem

9.1 The problem and corrected starting point

The historical evidence distinguishes elimination from synthesis of a recursive motive. Head of a nonempty vector succeeds through native dependent `cases`; vector map requires

```
(A B : Type) -> (n : Nat) -> (A -> B) -> Vec A n -> Vec B n
```

with motive $M(n, x) = \text{Vec } B \ n$. The inspected structural-recursion rule excluded indexed families and `Nat` before invoking Lean induction. Those source restrictions are not limitations of Lean’s induction facilities. An initial supported indexed adapter is consequently a separate, smaller milestone than unrestricted motive invention.^[N1–N9]

Four capabilities remain distinct: infer a motive for a prepared target; generalize locals so recursive hypotheses apply at changing arguments; repair propositional index equalities with transport; and expose a type family $F(?n)$ whose head depends on an unknown index. General higher-order unification is undecidable, while pattern unification has decidable fragments and practical elaborators add postponement and pruning^[17, 23, 1]. Native dependent pattern matching supplies index unification, not a complete enumerator of all useful strengthened motives^[8].

Canonical already demonstrates indexed recursor applications and generalized induction with a supplied stronger principle and predicate synthesis. The former claim that existing search never handles indexed or generalized induction is therefore too strong. Its translation restrictions remain relevant, and neither example establishes arbitrary motive invention. Agsy/Mimer, Idris proof search, and **sauto** remain useful comparisons; **sauto**'s dependent elimination and unfolding should not be mistaken for automatic induction discovery, which its documentation explicitly excludes. [6, 3, 26, 59]^[N1,N3–N9]

9.2 A finite motive language with explicit generalization

For a major premise $x : I \vec{p} \vec{v}$, separate three choices:

1. abstract the indices and major premise already present in the target;
2. revert a dependency-closed telescope of existing parameters;
3. invent a bounded carrier, auxiliary result, or invariant absent from the original context.

Mandatory dependency generalization is primarily an elaboration task; optional reversion is a search choice; invention is a larger synthesis problem. They share the carrier service of Section 7, but should not be charged to one undifferentiated powerset search.^[N1–N9]

Fix scoped goal syntax and a finite collection of eligible occurrence positions. With k positions, at most 2^k raw subsets arise before scope, typing, symmetry, and justified equality filters. This proves finite syntactic generation, not low complexity or completeness for higher-order unification. Nonoverlapping maximal occurrences are one useful policy, not the only one: a target containing $\text{Vec } B \ (n + k)$ may need abstraction of the occurrence of n inside the index while retaining k . A policy abstracting only the entire maximal $n + k$ need not generate that motive. Occurrences are scoped expressions with dependencies, not printed substrings.^[N3,N4]

Nor can occurrence abstraction invent a new binder or result component:

$$M(n, x) = \text{Vec } B \ n, \quad M(n, x) = \prod_{a:S(n,x)} T(n, x, a), \quad M(n, x) = T(n, x) \times U(n, x)$$

are different recipe families. The latter two can introduce S , U , or a strengthened invariant absent from the original target. Explicitly close a bounded template language under dependent products, products, dependent pairs, subtypes, option-like initialization state, and permitted carrier constructors. Fix its terminals, literals, universe-expression size, auxiliary type formers, number of new binders, and total expression size; otherwise a nominal syntax bound need not produce a finite space.^[N1–N9]

Generalization preserves telescope order and closes over both local declaration types and values. If $j : \text{Fin } n$, moving n across induction while leaving j fixed is ill scoped. If a recursive call changes accumulator a , generalize a even when its type mentions no index. Conversely, reverting every local makes branches unnecessarily large. For vector map, keep A , B , and $f : A \rightarrow B$ fixed; the empty branch produces $\text{Vec } B \ 0$ and the cons branch combines f of the head with the induction hypothesis at the tail length. Vector zip and lookup require the stronger parameter interfaces already traced in the unified proposal, not independent rediscovery here.

Compound or repeated indices need explicit preparation. Replace a compound index such as $n + m$ by a fresh variable only while retaining its defining equality and dependent consequences. Then use native elimination to create the obligations. A generalized variable without that equation changes the problem. A motive producing proof-carrying results must retain dependence on the major value itself as well as its numerical indices.^[N1–N9]

The recipe algorithm. The bounded service enumerates scoped recipes and validates each native eliminator attempt before exposing its branches.

Listing 2: Bounded motive service; pseudocode.

```

enumerateMotiveRecipes(goal, major, bounds):
  expose the inductive head and read native recursor metadata
  prepare mandatory dependent telescope and index equations
  yield native inferred motive
  yield small dependency-closed generalizations
    ranked by failed recursive-hypothesis applications
  yield bounded parameter/carrier/result strengthenings
  yield the declared wider scoped-occurrence abstractions
  for each recipe, within its fair bounded schedule:
    validate scope, sort, universes, and binder dependencies
    attempt native eliminator construction transactionally
    retain branches, substitutions, transports, and replay recipe
    check final specialization back to the original target

```

A recursive hypothesis fixed at the wrong accumulator suggests generalizing that argument and its dependents. A missing result component suggests product strengthening; an algebraic reassociation may instead need a lemma. These explanations rank proposals, not prove that omitted recipes cannot work. Prefer the native inferred motive before expensive alternatives and retain a fair bounded fallback.^[N1–N9]

Proposition 9.1 (Restricted motive completeness). *Fix a finite scoped occurrence set, a finite family of dependency-closed generalization telescopes, and a finite result-lifting grammar. If candidate construction and exact checking terminate, enumeration of every combination, discarding only ill-typed combinations, terminates and finds every well-typed motive in that class.*

Proof. There are finitely many combinations and each is processed in finite time. Every member of the specified class is generated by definition, and every well-typed member survives the stated filter. $\square$

A timeout is not exact checking and remains unresolved. This theorem describes exhaustion of the declared recipe family, not absence of any useful motive. A fair size-graded union enlarges coverage without making a fixed finite family complete for arbitrary accumulators or invariants.

9.3 Typed transports and restricted canonicalization

For $F : I \rightarrow \text{Sort } u, v : F(i)$, and $e : i = j$, transport constructs a term of $F(j)$, for example $\text{cast}(\text{congrArg } F \ e, v)$. Keep the family, source and destination indices/types, equality proof, transported term, and normalization provenance. A deferred transport must be a typed IR node whose interpretation inserts that cast, never an ordinary Lean term temporarily treated as though it had a different type.^[N1–N9]

Three notions must remain separate. Definitional equality permits conversion; propositional equality provides transport evidence; finite observational agreement generally does not permit dependent substitution. Proof irrelevance identifies proofs of the same proposition, not arbitrary endpoint types or data in different fibers. Runtime proof erasure is a fourth implementation fact, not a license to erase the typing role of `Eq.rec`.

The original example needs correction: with the standard Lean natural addition, $n + 0$ reduces to n , whereas $0 + n = n$ at symbolic n uses the corresponding theorem. Thus $\text{Vec } A \ (0 + n)$, associative index rearrangements, or $\text{Vec } A \ (n + m)$ versus $\text{Vec } A \ (m + n)$ exercise propositional transport; $\text{Vec } A \ (n + 0)$ normally does not.^[54] Test native definitional equality before adding a cast.^[N1,N3–N7,N9]

Choose a terminating deterministic index normalizer N for a declared fragment, returning $e_i : i = N(i)$. Constructor arithmetic or canonical affine expressions are possible first theories. Normalize endpoints, compose their certificates, and reuse one transport construction for the normalized pair. For vector append, one proved index equation can serve many corresponding terms. Outside the fragment, keep a symbolic equality obligation; different normal forms do not prove inequality unless the fragment procedure is complete for that claim. Arbitrary `simp` success is a proof service, not a definition of global canonical form.^[N1–N9]

Normalize identity casts and compatible compositions with checked lemmas. Transport along $p : A = B$ followed by $q : B = C$ agrees with transport along their composite, by equality elimination on p and q . This optimization retains endpoints. Orient rewrites or bound saturation so arithmetic equalities do not create rewriting loops. The policy, lemma set, and certificates belong in cache identity and receipts. Place casts consistently at constructor/provider boundaries rather than generating every distribution through internal nodes. Initially restrict registered families and index theories.^[N4,N5,N8,N9]

Heterogeneous equality can express relations before types coincide; converting that relation into the final homogeneous inhabitant still needs reconstruction. Typed e-graphs or local congruence classes may share checked equalities, but dependency-aware congruence and transport-producing extraction are obligations. A setoid changes the semantic language and requires every operation and contract to respect its relation. Neither provides a general cheap quotient of all Lean terms by arbitrary provable equality.

Proof evidence may have low or zero presentation/ranking weight, but transport work remains real. Use a positive syntax grade, a certified canonical representation, or an explicit finite transport bound to avoid infinitely many zero-cost paths. The earlier bound of two casts is an experiment that restricts the grammar, not a completeness theorem. Hard deduplication requires a common expected type and checked definitional equality or an equality certificate; a coarser proof-insensitive fingerprint may group, not delete, unequivalent candidates. If display hides casts, either emit the exact checked term too or re-elaborate the sugared source and certify its intended equality. Do not publish untypeable Lean as a paste-ready result.^[N1–N9]

9.4 Demand-driven unfolding and index narrowing

Canonical describes exposing function aliases such as `Set` and `Not`, reducible definitions, and class-instance structure in its translation; it also notes dependency expansion and abstraction costs. The documented `sauto` controls distinguish heuristic `unfold`: (default explicit list empty) from forced `unfold!;`; basic logical definitions receive special treatment. `red` simplification and `ered` eager scheduling are documented default-on options, with separate context-change simplification and backtracking solvers. N6 explicitly did not verify those implementation defaults; the more precise statements come from the documentation evidence in N1, N3, N7, and N9, not from a comparative run. Pin revisions and options in any experiment.^[6, 59]

Use a phase-sensitive transparency ladder:

1. expose only the binder, function spine, constructor family, class head, or rigid application head needed for retrieval or the next rule;
2. normalize registered index expressions using checked equations;
3. unfold a demanded head blocking matching, a scrutinee, or motive construction, charging the particular request;
4. offer a bounded broader-unfolding or theorem-rewrite fallback.

Retrieval keeps cheap normalized heads; speculative unification can escalate for a concrete proposed application; observation evaluation uses supported definitions/equations; proof simplification uses a bounded explicit rewrite set. Ordinary provider names remain compact components

even when their bodies are unfolded for reasoning. Expanding the whole transitive provider closure can destroy indexing selectivity and duplicate enormous expressions.^[N1–N9]

Record the request’s purpose, transparency, unfolded declarations, environment, local context, and assignment dependencies. A shallow mismatch means unresolved at this stage, not proof of nonconvertibility. It cannot permanently remove a candidate requiring a later stage allowed by the advertised grammar. Escalation is resumable work charged to the ledger, with per-constant budgets and cycle protection. Opaque constants remain opaque unless an allowed checked equation or explicit interface permits reasoning about them.

If $F(?n)$ is stuck because the index determines its head, repeatedly unfolding F adds no information. Schedule another goal that determines $?n$, or enumerate constructor forms of the inductive index under the split budget. This is *narrowing*, a search alternative sharing the native assignment state, not a normalization theorem. Keep all admitted forms and their dependencies.^[N7]

Compare fixed weak-head/reducible policies, demand-driven escalation, and bounded up-front normalization on indexed vectors, arithmetic, and large libraries. Count matches unlocked, failed conversions, reduction work, expression growth, cache invalidations/memory, and verified candidates at matched logical budgets. Compare Canonical, `exact?`, and translated `sauto`/Agda cases to locate capability boundaries; changed elaboration and libraries make raw cross-assistant timings informative case studies, not an implementation theorem or a demonstrated optimal policy.^[N1–N9]

9.5 A version-pinned native motive adapter

The Lean 4.34.0 source inspected by the responses exposes `Lean.Meta.getMajorTypeIndices`, `Lean.Meta.mkRecursorAppPrefix`, and `Lean.MVarId.induction`. The first requires suitable variable indices and checks repeated or improperly dependent occurrences. The prefix builder abstracts the major premise and indices, chooses recursor universes, and checks the motive/elimination sort. Induction reverts dependencies, reintroduces them, and returns `InductionSubgoal` values containing goals, fields, and a `FVarSubst`.^{[51]^[N1–N9]}

Listing 3: Signatures reported from the pinned source; integration still needs compilation tests.

```
Lean.Meta.mkRecursorAppPrefix
  (mvarId : MVarId) (tacticName : Name)
  (majorFVarId : FVarId) (recursorInfo : RecursorInfo)
  (indices : Array Expr) : MetaM Expr

Lean.MVarId.induction
  (mvarId : MVarId) (majorFVarId : FVarId)
  (recursorName : Name)
  (givenNames : Array AltVarNames := #[]) :
  MetaM (Array InductionSubgoal)
```

Do not duplicate reversion already performed by the native API, and do not assume success of high-level `cases` means every low-level induction call accepts the unchanged major premise. Compound/repeated indices may need the elaborator’s generalization path or remembered equalities first. N3 also identifies `ElimApp.mkElimApp` and `ElimApp.setMotiveArg` in tactic elaboration as implementation references, with their surrounding state and preconditions intact. Functional induction over an existing definition is another available principle; it does not itself invent the desired recipe.^[61, 74]

A trial accepts the major premise, generalized telescope, recursor/principle, and recipe. Save complete speculative state, validate the recipe, invoke the native operation, inspect branch targets, and retain substitutions and transports. Return a persistent successor or replay recipe; raw

metavariable identifiers from a rolled-back state must not escape. Apply the same substitution to observations and recursive capabilities instead of guessing new locals from printed names or positions.^[N1–N9]

Use structured outcomes:

Listing 4: Proposed adapter results; pseudocode.

```
MotiveAttempt :=
  success(prefix, branches, substitutions, transports, replay)
| unsupportedMajorOrRecursor(reason)
| indexPreparationNeeded(indices)
| illegalEliminationOrUniverse(reason)
| illTypedMotive(diagnostic)
| missingInstanceOrUnresolvedConstraint(constraint)
| resourceExhausted
```

Preserve native diagnostics as detail, not fragile exception-text predicates for impossibility. Failed attempts restore assignments, universes, local instances, messages, delayed constraints, substitutions, and branch caches. Cancellation and internal/resource interruption propagate to the controller; they are not ordinary motive failures. Limit heartbeats/recursion per trial, but do not promise cheap failure merely because the function is callable from **MetaM**. Memoize only with the complete goal/context, recipe, recursor, policy, and relevant assignment identity.

The first adapter suite should include lists, naturals, options, vector map, vector zip, dependent pairs, **Fin** *n* locals, compound/repeated indices, major-value-dependent result families, forbidden elimination from **Prop**, and a near-identical ill-scoped recipe. Repeated failed trials must leave the next trial observationally unchanged. Measure instantiation and failure cost separately from branch search. Compile against the actual pinned toolchain before declaring this a supported API, and keep specialized fast constructors behind the same validation boundary.^[N1–N9]

9.6 New questions after the four original answers

The finite-template, transport, unfolding, and API-access questions above have concrete bounded answers. Their unresolved extensions are:

- R4.N1. Which explicit motive recipe grammar, including nonmaximal index abstractions and dependency-closed changing parameters, gives the best measured coverage before new carrier or invariant invention is required?
- R4.N2. For which registered dependent families and index theories can transport normalization be proved terminating and reconstructible while keeping its enumeration grade and proof cost bounded?
- R4.N3. Which phase-sensitive transparency policy improves verified results at matched logical budgets without turning shallow conversion failures into permanent exclusions?
- R4.N4. Can the pinned induction adapter preserve substitutions and every speculative state component across failure and interruption, and what are its measured success/failure costs on compound indices and value-dependent motives?

10 R5. General recursion from incomplete examples

10.1 The missing capability and revised comparisons

The historical implementation admitted outermost constant-motive recursion on an index-free recursive inductive other than **Nat**, exposing induction hypotheses as its recursive calls. Its list-length/map and tree-size successes do not establish natural recursion, indexed motives, changing

parameters, derived measures, helper invention, or mutual/nested recursion. The intended end state retains these requirements. A small scheme portfolio specifies an implementation order, not a replacement for them.

Several examples need a corrected diagnosis. Zip can recurse on one list while generalizing and matching the other. Fibonacci can use a pair of successive values under ordinary natural recursion rather than necessarily requiring sub-subterm calls. Tree flattening can use append, an accumulator, or a difference-list carrier; with append already provided, body depth and search order may dominate. Merge admits nested structural organization or a measure on the joint argument tuple. These alternatives separate expressibility, representation cost, and discovery difficulty.^[N1–N9]

Lean’s definition elaborators, recursor metadata, equation theorems, and termination principles provide realization infrastructure. Size-change and lexicographic analysis supply established termination methodology, but the synthesizer still proposes calls and discharges decrease obligations [27]. The historical estimate of tens of milliseconds per elaborated candidate motivates measuring and amortizing realization; it is not a demonstrated universal cost bound.

Myth uses recursive examples in a trace-complete regime. Smyth explicitly supports interdependent holes and sparse recursive traces; grouping it under a blanket trace-completeness requirement was incorrect. Burst supplies angelic execution and strengthening, Trio builds recursive programs from nonrecursive expressions, and Synquid uses refinement/measure information. These are different synthesis and validation settings, not transferable Leant2 scores [24, 28, 5, 34, 30]. The mixed-quantifier recursive-sketch and counterexample-derived syntactic constraints of the July 2026 Cataclyst preprint are another comparison point, not a dependent Lean implementation or a ready-checked pruning rule.^{[60][N1,N3–N9]}

10.2 Typed capabilities and explicit realization

Retain the unified proposal’s recursion plan: function telescope, selected major argument, fixed/generalized parameters, motive, branch telescopes and index equations, scoped recursive capabilities, program/proof holes, termination obligations, final specialization, and executable lowering. Search may use equation-like bodies while the kernel checks realized recursor or well-founded terms. Neither representation alone proves their agreement.

Let X contain the complete recursive state, $B : X \rightarrow \text{Sort } u$ be the result family, and $R : X \rightarrow X \rightarrow \text{Prop}$ a chosen well-founded relation. At state x , the capability is

$$\text{rec}_x : (y : X) \rightarrow R(y, x) \rightarrow B(y). \quad (8)$$

The result is $B(y)$, not a fixed type or $B(x)$. Changing accumulators, other arguments, and mutual-recursion tags belong to X or are explicitly quantified in the result family. A measure may depend on only part of X , but call identity and admissibility use the full state. Otherwise unequal calls can be conflated and the indexed programs of Section 9 excluded.^[N1–N9]

The body has no unrestricted provisional self-reference. Capabilities are scoped to a recursion frame and cannot leak into ordinary providers. Structural constructor paths cheaply supply some decrease evidence; measure calls create proof obligations. Decrease on one sample is not a universal proof. An unresolved proof hole may remain a guarded obligation, but deductions depending on it stay heuristic until their hypotheses hold. Never introduce a global temporary axiom naming the recursive function and overlook it during the dependency audit.

A typed body functional has type

$$F : \prod_{x:X} \left(\left(\prod_{y:X} R(y, x) \rightarrow B(y) \right) \rightarrow B(x) \right).$$

Realization produces $f : \prod_x B(x)$ and a checked equation

$$f(x) = F(x, \lambda y h. f(y)), \quad (9)$$

including administrative parameters and casts of the chosen encoding. Keep ordinary expressions and types native to Lean; the additional IR need only encode recursive-call structure, guards, branch parameters, shared holes, and pending termination obligations.^[N2,N5,N6,N9]

Two realization paths remain useful. Send promising closed typed bodies through Lean’s equation elaborator and retain declarations/equations; or instantiate a small library of already certified recursive templates. The second can avoid repeated full elaboration but must not become an unverified parallel termination compiler. Realize selected survivors or reusable helpers, cache checked equations, and charge realization/certification to the reported end-to-end budget. Generated declarations, equations, and termination evidence extend the environment fingerprint and acceptance audit.

10.3 Interpreter agreement is generally propositional

Structural encodings can have convenient definitional equations. General well-founded definitions often require propositions such as `WellFounded.fix_eq`; Lean documents examples whose expected equation does not close by `rf1`. The opposite blanket claim is also too strong: N5 and N8 identify Lean 4.27’s reducible fuel-based implementation for relevant natural-measure definitions. State the encoding and pinned-version behavior rather than asserting that every well-founded equation is only propositional. ^[56, 55]^[N1–N9]

Proposition 10.1 (Interpreter–realizer agreement). *Suppose a typed body interpreter and realization use the same body semantics, every call carries valid $R(y, x)$ evidence, nonrecursive operations preserve their intended dependent meanings, and the realization satisfies Equation 9. Every successful finite interpreter evaluation at x returning $v : B(x)$ establishes $f(x) = v$.*

Proof. Use well-founded induction on x . The decrease proof of each call y makes the induction hypothesis available, identifying its interpreted result with $f(y)$. Substitute these equalities into the proved semantics of the nonrecursive operations, using the same dependent substitutions and transports. The realization’s equation identifies the resulting value with $f(x)$. Equivalently, induct on a successful finite evaluation derivation explicitly recording all recursive calls. $\square$

Equality preservation is an obligation, not a consequence of matching input/output types. For reflected bodies, prove it by induction on syntax. If the proof equates higher-order recursive oracles using extensionality, account for that principle in the axiom profile. The equivalent uniqueness formulation says two total interpretations satisfying the same well-founded equations agree pointwise when the body respects pointwise equality of recursive results.^[N2,N3,N9]

A fuelled interface records a derivation or checked trace and satisfies

$$\text{runFuel}(k, b, x) = \text{some}(v) \implies \text{realize}(b)(x) = v. \quad (10)$$

Fuel exhaustion gives unknown, neither divergence nor a bad contract value. Adequate-fuel/eventual-success is a separate theorem for a declared total fragment. A decreasing natural measure may bound recursion depth, not every reduction or branching evaluation-tree size; define the fuel unit. Termination does not supply a small uniform interactive-budget bound. Simulation of closed programs also does not justify open pruning without the all-completions condition of Section 8.4.^[N1–N9]

For closed observations, rewrite with generated equations, use proof-producing simplification or `cbv/decide_cbv`, or evaluate a reified body and reconstruct the observation proof through

Equation 10. Benchmark native equation evaluation before a universal interpreter. Keep three trust profiles distinct: direct kernel reduction; proof-producing interpretation checked under the original axioms; and compiled computation with explicitly approved additional assumptions. Compiled execution can screen, rank, or nominate counterexamples; kernel-only hard pruning needs checked negative evidence. The source reports identify native reduction axioms such as `Lean.ofReduceBool` as excluded providers, so quietly using them changes the policy.^[52, 53]^[N1,N4,N6,N7,N9]

10.4 Publishing an executable definition

The recorded code-generator failure on a logically accepted recursor term is an independent boundary. Preserve the logical object, construct a compilable equation-style definition, and prove pointwise correspondence or reprove the original contract for that definition. Attaching `implemented_by` is not an equality proof.

Lean’s `Nat.recCompiled`, `Nat.rec_eq_recCompiled`, and its `@[csimp]` attribute supply a proof-backed compilation-rewrite precedent in the pinned core. Its comment identifies recursor compiler support as the motivation. Investigate a small publication adapter using this pattern before a universal recursor decompiler.^[54]^[N3,N5]

The bridge proof may introduce axioms absent from the logical term, especially through extensionality; audit it separately. A pointwise bridge or direct contract proof may avoid an unnecessarily strong equality obligation. Logical equivalence does not itself verify the compiler/runtime, and unaccounted implementation substitutions remain relevant. Record kernel acceptance, contract certification, export, compilation, and representative execution separately. A noncompiling logical result remains useful with explicit status; a compiling substitute lacking the bridge is not automatically the certified program. Include publication work in time to usable result.^[76, 80]^[N1–N9]

For an invented helper bundle, construct its declaration-dependency graph and check an acyclic bundle in topological order. Each declaration must close over the advertised telescope, depend only on imported or earlier accepted declarations, preserve its universe parameters, and satisfy the transitive axiom policy. Recheck the final program and contract in the exported bundle; scratch-environment success is insufficient. Mutual recursion requires an explicitly supported bundle rule with its own joint termination and realization checks, rather than bypassing the acyclic audit.^[N5]

10.5 Top-down angels, sharing, and helper invariants

Section 8.5 states the oracle semantics and finite theorem once. Top-down search gains expected types, local telescopes, induction information, and sketches. Bottom-up version spaces or finite-tree automata share completed subterms and behaviors more directly. Neither dominates without a matched experiment, and changing direction does not inherit Burst’s space savings or correctness proof.^[N1–N9]

A branch carries its partial equation body and a persistent call-constraint graph. A descriptor includes frame/function identity, full typed arguments, universes, local substitution, and assumptions. Hash-cons with checked identity. Filling constructor fields propagates demands; filling guards selects conditional constraints; refining arguments can identify a call with a known input or another call. Assignment watchers invalidate both forward residuals and affected oracle assumptions. Repeated occurrences of a hole remain one shared unknown.

Keep three evidence levels: answers forced by the original contract; existential answers that could make the sketch pass; and actual results of the closed realized candidate. The first constrain accepted completions, the second guide or enter sound relational solving, and the third support

final checking. From $r_1 + r_2 = 5$, committing to 0, 5 and discarding other explanations is invalid. If both symbols denote the same call, include their equality. If one observation permits zero or one and another demands positivity, prematurely choosing zero loses the feasible option.^[N2,N6]

When execution contradicts a guessed result, reject the completed body or refine that combination of skeleton and assumptions. A failed unobserved-call guess does not ban all bodies or globally ban the input. Nogoods retain exactly their supporting assumptions; specification-relative counterexample caches are not universal facts about every candidate.^[N4,N7,N8]

For an invented worker $g(s, a)$ with $f(s) = g(s, a_0)$, observations of f normally constrain only seed a_0 . Reusing them at arbitrary a is invalid. Supply a generalized invariant, derived typed examples, or concrete execution. The reversal invariant $g(xs, acc) = \text{reverse}(xs) ++ acc$, universally quantified over acc , illustrates joint generalization of a changed parameter and its correctness property. Helper/carrier alternatives remain reopenable when a proposed strengthening is unsupported.^[N1,N5,N9]

Use a hybrid: shallow closed components for a specific telescope and target, typed recipes retained behind behavioral summaries, offered to top-down refinement. Cross-context reuse needs checked renaming/substitution. A behavior vector is an index, not a permanent quotient: new recursive inputs may split its bucket. Begin with observed calls, then symbolic unknown calls for ordering, then a complete certified relational fragment for pruning. Execute the finished recursion ordinarily and check the original full contract with no remaining angels. Convenient assumptions do not fabricate trace-complete examples. ^[N1–N9]

10.6 A staged recursion portfolio and its limits

No response establishes a canonical minimal basis or Leant2 coverage percentage. A universal well-founded recursor can make a basis syntactically tiny while hiding difficult measure, algebra, and proof searches. Optimize coverage at bounded total cost under a fixed component inventory, including scheme selection, body filling, decrease proofs, realization, and certification.

Paramorphisms are a useful published starting point. For a polynomial functor F , an algebra $F(\mu F \times K) \rightarrow K$ sees each original child and recursive result; a fold ignores the original child. For lists,

$$\text{para}(z, s, []) = z, \quad \text{para}(z, s, a :: xs) = s(a, xs, \text{para}(z, s, xs)).$$

Natural primitive recursion similarly exposes predecessor and prior result. Para uses typed multi-hole templates with recursion-free fillers and reports a 59-benchmark study. Its non-kernel validation and resource setting do not establish Leant2's ten-second dependent-synthesis coverage.^[44] ^[N1,N8]

Tier	Purpose, witness, and additional obligation
Structural	Registered folds/paramorphisms, including naturals and supported indexed families: map, length, tree size, powers. Validate motives and constructor-field capabilities.
Generalized/stateful	Changing parameters, function accumulators, option-like empty state, products, dependent result strengthening. Reverse, zip on one structural list, extrema, Fibonacci pairs, and difference-list flattening need their initializer, updater, and finisher.
Nested/helper	Bounded nesting over other arguments/substructures; initially one synthesized helper with a closed parameter interface, without hidden access to a parent's changing context.

Tier	Purpose, witness, and additional obligation
Course-of-values	Certified deeper-predecessor access when structural/product encodings are unsuitable. Record constructor paths, indices, and admitted calls; depth two is a syntactic bound, not all course-of-values coverage.
Measured	Finite measure/lexicographic grammar on full states, checked well-foundedness and per-call decrease. Bound arithmetic expressions, tuple lengths, relation choices, and decrease-proof search.
Mutual/nested families	Later adapters using actual recursor metadata, tagged joint states or generated definition bundles, and their real induction/termination interfaces; avoid brittle generated-name rules.

Rows overlap: count all admissible reference encodings and measure the cheapest verified one. Task names do not determine necessary schemes. Providers can hide substantial recursion, so report their capabilities alongside visible template complexity.^[N1–N9]

The measured tier includes merge with sum of remaining lengths; Euclidean gcd with a nonzero-branch modulus-decrease theorem; division/binary search with decreasing intervals; and quicksort with filtering-length bounds. Calling `omega` does not supply every semantic lemma. `sizeof`, length, natural arguments, bounded differences, and lexicographic tuples require precise semantics and proof plans. A failed measure/proof attempt rejects that proposal, not the original problem. Prefer measures whose obligations lie in a declared proof fragment before broad invention.^[N1,N2,N5–N9]

When body search reaches a consistently constrained hole beyond its depth, propose a helper with a dependency-closed parameter interface and derived necessary observations. Do not assume those observations fully specify it. Retain caller reconstruction and the original final contract. One helper per query is an experiment, not an expressiveness theorem. Mutual/nested inductives need dedicated tracks. Coinductive or intentionally partial programs require a different semantic acceptance profile; interpreter fuel must not quietly admit them into a total-function claim.^[N1,N5,N7,N8]

10.7 Coverage experiments and new questions

First translate a withheld reference witness into the declared scheme grammar and kernel-check it: this establishes representation coverage. Next run with scheme and carrier supplied, scheme alone, and neither. Distinguish providers enabled/withheld and disclose helpers. These separate body search, carrier invention, scheme choice, and retrieval. A handwritten witness is neither discovery nor timing evidence.^[N1–N9]

Stratify list/natural/tree exercises, unresolved Church/indexed cases, and adversarial variations of index expressions, parameter order, and termination measures. Hold out families when assessing guidance. Compare trace-complete examples, sparse examples with neutral calls, and sparse examples with angels at matched body/scheme bounds. Include a case omitting all internal calls and require actual realized execution to decide acceptance. Record overlapping scheme encodings before attributing marginal coverage to one tier.

Report first sufficient/successful schemes, carriers, components, helpers, trace completeness, universal-proof requirements, decrease-proof costs, realization, kernel checking, compilation, and execution. Distinguish out-of-grammar, interrupted search, unresolved decrease, behavioral counterexample, and unresolved contract proof. Give logical-work and wall-clock curves through

the usable exported artifact. Original ten-second recursion probes and regression scores remain gates; a toy model or supplied-scheme success does not prove automatic invention at that budget.^[N1–N9]

The original interpreter, direction, and basis questions have the conditional answers above. Remaining questions are:

- R5.N1. Which minimal typed recursive-body IR supports checked simulation into native equation elaboration and reusable templates, including dependent results, and what adequate-fuel theorem is practical for that fragment?
- R5.N2. How should a persistent top-down oracle store represent helper invariants and assumption-scoped nogoods so refinements retain all alternatives while recovering useful bottom-up sharing?
- R5.N3. Which recursion portfolio and measure/proof grammar give the best verified coverage when representation, supplied-scheme filling, automatic invention, and executable publication are measured separately?
- R5.N4. Can a small proof-backed publication adapter, using equation definitions or compiler simplification theorems, preserve each permitted axiom profile while making the synthesized recursor families executable?

11 R6. Contracts beyond decidable observations

11.1 The problem and the implementation boundary

Closed decidable equations support evaluation followed by an ordinary Lean proof. Universal length preservation, involution, sortedness with permutation, and agreement with a reference implementation require more: tests can refute a candidate, but only a proof certifies the requested universal contract. Refinement signatures such as `(xs : List A) -> {ys : List B // ys.length = xs.length}` expose local proof obligations during construction. Laws such as `f (f xs) = xs` or `f (xs ++ ys) = f xs ++ f ys` couple several executions and need not decompose into independent pointwise postconditions.

The incoming implementation audits already find root program–proof pairing: `Engine.lean` wraps a contracted target in a `Subtype` and extracts its value and property for acceptance. R6 does not propose inventing that pairing again. The missing extension is to carry a suitable invariant through recursive motives so each recursive result comes with the property needed by its caller. The inspected `proofPortfolio` in `Search/Core.lean` refuses a target containing free variables as well as unresolved metavariables. A legitimate local branch goal usually contains free variables and an induction hypothesis. It needs a contextual proof adapter, or a checked abstraction over its telescope; adding a tactic to the closed-goal list does not supply that adapter.

N1–N9 agree on these architectural boundaries. N8 explicitly identifies the existing root subtype; N1, N2, N4, and N7 identify the contextual portfolio gap. These source observations do not establish that the proposed service has been implemented or benchmarked. The unified proposal already specifies proof-service transactions, typed counterexamples, and an unresolved queue. The following refines readiness and induction certification; the common authority and result distinctions remain those of Section 3.

11.2 Dependency-sensitive program and proof construction

The original assertion that a proof about a program with holes is meaningless is too strong. Reflexivity can solve `?n = ?n` uniformly, or solve `?n = 3` by assigning a construction choice. Constructor disjointness can exclude an impossible branch before its arguments close. A condi-

tional lemma can leave explicit assumptions for later fillings. These are three effects: a uniform proof, a branch-local assignment, and a proof under residual obligations. Record which occurred; a timed-out tactic supplies none of them.

For each proof job retain the exact goal and local telescope, owning candidate, value/type/universe dependencies, local hypotheses and instance choices read, assignments it may make, certified consequences, attempted tiers, and support fingerprint. Cheap conversion, reflexivity, constructor reasoning, rigid index constraints, and applicable lemmas run when informative. Expensive proof search runs when its particular dependencies stabilize or its expected pruning benefit justifies an early attempt. A tactic assignment is a transactional program-search choice, visible to dependent goals and restored on backtracking. Metavariables represent existential choices; explicit binders or interface assumptions express a theorem uniform in all remaining choices. Conflating the two can incorrectly reuse a proof from one filling at another.

Keep value construction, ready local proofs, termination obligations, and global proof debt as roles in the same dependency graph. A required instance may determine an output type or computational operation and cannot always wait until every type hole is ground. Wake suspended tasks only after a relevant support change or increased tier. Cache a completed local proof only after abstracting its dependency-closed telescope and checking the template on reuse.

For a pointwise specification, construct

$$g : \prod_{x:X} \{y : B(x) \mid Q(x, y)\}, \quad f(x) = g(x).val.$$

For length-preserving map, the nil branch returns $([], \text{rfl})$. Given recursive pair (ys, h) with $h : |ys| = |xs|$, the cons branch returns

$$(a'(a) :: ys, \text{congrArg}(\text{Nat.succ}, h)),$$

after the computation equations for length. This is a complete local proof pattern: its proof follows the construction instead of rediscovering induction on a finished anonymous recursor. A wrong-length branch can expose an early local contradiction. For accumulator or multi-call laws, strengthen Q to an invariant I and retain a checked implication back to Q ; failure may reopen the invariant or carrier choice.

Synquid motivates specification decomposition, but its solver-supported refinement fragment is not unrestricted Lean **Prop** [30]. A native adaptation uses a finite vocabulary of length equations, nonemptiness, bounds, membership, or ordering predicates, with checked introduction, elimination, implication, monotonicity, and component-transfer lemmas. Abstract joins forget unsupported facts; failed implication search means unknown. This gives useful refinement propagation without claiming complete liquid inference for arbitrary propositions. All nine incoming treatments share this restricted interpretation; their repeated examples are consolidated here.

11.3 A certificate for a recognizable induction class

Let $s(xs) = \text{foldr } \delta \ z \ xs$, let $I(xs, r)$ relate the input to its summary, and let ϕ finish it. Generate

$$I([], z), \tag{11}$$

$$\forall a, xs, r, \quad I(xs, r) \rightarrow I(a :: xs, \delta(a, r)), \tag{12}$$

$$\forall xs, r, \quad I(xs, r) \rightarrow Q(xs, \phi(r)). \tag{13}$$

Proposition 11.1 (Local invariant certification). *Proofs of (11)–(13) imply*

$$\forall xs, \quad Q(xs, \phi(\text{foldr } \delta \ z \ xs)).$$

Proof. Structural induction proves $I(xs, s(xs))$. The nil case is (11); the cons case uses the induction hypothesis in (12) and the fold computation equation. Apply (13). Each composition is an ordinary proof term. $\square$

For a polynomial datatype, generate one certificate per constructor c :

$$\left(\bigwedge_{j=1}^m I(x_j, r_j) \right) \longrightarrow I(c(\vec{v}, \vec{x}), a_c(\vec{v}, \vec{r})).$$

All nonrecursive fields and other hypotheses are universally quantified; a base constructor has $m = 0$. Induction supplies exactly the recursive hypotheses licensed by the datatype. Indexed families require each result at the correct motive fiber and explicit transport obligations. If recursive calls change an accumulator, quantify it in the induction hypothesis. For well-founded recursion use its checked induction principle and equation correspondence rather than treating arbitrary calls as structurally smaller children.

Define the initial recognizer by a finite grammar of recursion/invariant/generalization plans, a terminating oriented rewrite policy with proved rules, and a declared residual logic. A plan generates branch conditions, applies its hypotheses and rewrites, checks residual syntax, and requests proof-producing leaf decisions. A useful first fragment contains constructor/projection facts and quantifier-free linear arithmetic over naturals and integers, including only fixed-coefficient multiplication. Recognition is decidable for the bounded plan class when plan enumeration, normalization, and leaf checking terminate. Completeness is relative to this class and to an actually complete solver for the declared fragment. The names **omega** or **grind**, especially with resource caps, do not establish that premise by themselves.

Map length reduces from $|ys| = |xs|$ to equality of successor lengths. Affine size bounds often have similar leaves. Reverse may need an append lemma or generalized accumulator invariant; sorting needs order and permutation facts; involution can need a lemma connecting two traversals. These are additional certificate requirements, not evidence that the properties are false. Return the failed local condition and selected plan with one of: certified; template applicable but invariant missing; outside the current fragment; or unresolved at the allotted bound. Missing invariants guide carrier/generalization proposals or lemma retrieval.

N8 adds a crucial scope restriction: even a checked counterexample to a *local invariant condition* need only reject that program-invariant pair. A too-strong invariant can fail while the original postcondition is true. Refuting the program requires a contradiction with the original contract, not merely failure of a sufficient proof plan. N1–N9’s local-certification results are mathematical design arguments; their finite host-language models are not native Lean implementations of this recognizer.

11.4 A bounded native induction service

Use an explicit planner around Lean’s induction facilities. The incoming source audits identify `MVarId.induction` and, in the pinned 4.34.0 elaborator, `evalFunInduction` and `fun_cases` [51, 61]. N1 and other reports also cite a moving manual; those citations must not be mistaken for a release-specific integration test. Named supported recursive definitions may have functional induction principles [74]; an anonymous recursor need not have the theorem name expected by a high-level tactic. Preserve recursive equations, major premise, generalizations, and realization correspondence. Construct needed declarations in a scratch environment and include their dependencies in replay.

The adapter introduces quantified inputs; chooses a small ordered set of structural or functional principles and dependency-closed generalizations; generates branches; unfolds checked equations;

and runs local assumptions, reflexivity, curated simplification, arithmetic, then bounded saturation. Library lookup can precede repeated speculative induction when a theorem already settles a branch. Separate principle-selection, branch-generation, and branch-closing budgets. Save the proof term, not only a tactic string.

Aesop supplies configurable rule search, Canonical offers dependent inhabitation and recursor reasoning, and LeanHammer supplies premise selection and proof reconstruction [2, 6, 21]. None establishes automatic discovery of arbitrary invariants under Leant2’s interactive budget. N7’s HipSpec-style adaptation proposes a bounded grammar of equations from failed branches, tests conjectures, then proves them before adding them to a reusable lemma bank [45]. An unproved conjecture is never a simplification axiom. N8’s observations about induction-oriented `try?` development [75] likewise motivate a version-pinned experiment, not an assumed drop-in solution. No incoming report establishes a complete maintained rippling/lemma-invention service for this task.

Every service preserves full state on ordinary failure, propagates cancellation distinctly, checks scope and the original proposition, and audits transitive axioms. Its result distinguishes proof, checked counterexample, reduced or suspended obligation, unsupported fragment, and resource failure. Heartbeats and recursion limits constrain supported work; a deadline checked between calls does not preempt one long call. A service that cannot cooperate needs an isolated process for a hard wall-clock bound. Optional solvers, hammers, or remote retrieval remain outside the core-only default; startup, reconstruction, and communication costs are measured.

11.5 Paying proof debt without starvation

There is a simple optimal ordering only under a much simpler model. For independent jobs with fixed positive costs t_i, t_j , success probabilities p_i, p_j , and stopping at the first success, the expected costs are

$$t_i + (1 - p_i)t_j \quad \text{and} \quad t_j + (1 - p_j)t_i.$$

The first is no larger exactly when $p_i/t_i \geq p_j/t_j$. N2 states the analogous first-rejection calculation for tests; N3 supplies the first-proof-success form. Both follow by the same exchange argument. Actual proof jobs violate independence and stationarity: assignments, shared lemmas, counterexamples, warm contexts, and earlier failed tiers change other jobs’ costs and prospects. General anytime and metareasoning ideas motivate a policy, not an independent-bandit optimality theorem [71, 72].

A proposed score for an eligible next tier is

$$U(j) = \frac{\hat{p}_{\text{cert}} V_{\text{cert}} + \hat{p}_{\text{refute}} V_{\text{prune}} + \lambda \hat{I}_{\text{shared}}}{\hat{c}_j + \hat{c}_{\text{switch}} + \varepsilon}.$$

Utility includes candidate quality, downstream work avoided, and reusable information. These are measured estimates, not admissible search bounds. Freeze them within recorded logical epochs. Compare learned policies with deterministic fixed tiers and round-robin resumption before choosing a default.

The baseline uses weighted round-robin or deficit scheduling among value search, refutation, ready local proofs, termination, and global proofs. Reserve positive service for each eligible nonempty queue and age old jobs. Give increasing logical-work quanta, record attempted tiers and dependency versions, and resume continuations when the solver permits; otherwise account for restart and reuse only valid preprocessing. Batch jobs sharing a context when measured setup cost warrants it. Bound active debt by suspending states, not rejecting them. Waiting until search is idle can starve proofs forever; the earlier illustrative 5/50/500 ms ladder is a calibration hypothesis, not a portable semantic exchange rate. A small certification reserve near a deadline improves opportunity without guaranteeing success.

Before costly tiers, test survivors against the bank of checked counterexamples. Generate well-typed dependent inputs with indices and precondition proofs; confirm violations in the admitted semantics. An invalid input, failed precondition, evaluator timeout, or failed proof is not a counterexample. Reuse counterexamples only for the same frozen contract and interpretation. Passing tests never replaces the universal proof. Keep unproved survivors as *unrefuted*, *not certified*, available for continuation and separate from verified answers.

11.6 Acceptance measurements and remaining questions

Historical R6.1 is answered by dependency-sensitive readiness; R6.2 has a sufficient certificate class; R6.3 has concrete induction infrastructure but no general invariant-invention service; R6.4 has a fair baseline and a restricted exchange argument, not an optimal policy for coupled search. Use at least a twenty-contract study with hand proofs classified as simplification, arithmetic, structural induction, generalized induction, retrieved lemma, and invented lemma. Include over-strong invariants and subtype branches using local hypotheses. Record time to the first certified useful result and first unproved survivor, debt count and age, productive wakes, repeated unchanged attempts, refutation latency, cancellation, and cost by proof stage. Aggregate tactic success does not establish improved certified synthesis.

R6.N1. What smallest invariant and generalization grammar covers the hand-classified quantified-contract suite, and which failed local conditions require a new invariant rather than a stronger leaf solver?

R6.N2. Can a contextual proof service return replayable assignment patches and closed proof templates while preserving the native dependency graph under backtracking, including local instances and universe assignments?

R6.N3. At equal logical work and equal end-to-end deadlines, does support-aware resumption and context batching improve time to a certified useful answer over fixed tiers and fair round-robin, without starving any eligible obligation class?

12 R7. Component retrieval at library scale

12.1 The problem and established precedents

A curated provider list cannot cover an ambient Lean library. Polymorphic providers such as `List.foldr`, `Option.getD`, and `id` can fit many goals; retrieving more names can therefore increase search cost even when lookup is fast. The historical probes missing `Nat.succ_pos` and `Nat.le_trans` motivate a proof-lemma retrieval lane, but adding a retrieval interface alone does not prove that those probes will close under their original budgets.

Lean indexed lookup, Loogle-style type queries, and theorem premise selectors supply useful baselines. SyPet’s component reachability and Hoogle+’s type-guided abstraction refinement supply composition precedents [31, 16]. All N1–N9 correct the original question about type classes: Hoogle+ already uses dictionary passing, described in its Sections 2.5 and 5.1. Its reported experiment uses 291 components and 44 queries; neither this nor its first-order treatment establishes retrieval over all dependent declarations in a large Lean environment. Higher-kinded cases such as general `Monad m` and Lean’s dependent constraints remain adaptation work.

Use separate proposition-lemma and data-component retrieval modes behind one typed interface. A proof lemma relevant to length preservation need not compute a list; several data programs can share `List A → List A` while examples distinguish identity, reversal, and duplication. Retrieval success is a proposed component set, not an accepted term or a global impossibility certificate.

12.2 Constraint-carrying AND-hypergraphs

For a provider

$$c : \prod_{x_1:T_1} \cdots \prod_{x_n:T_n(x_1,\dots,x_{n-1})} R(x_1,\dots,x_n),$$

represent an application as an AND-hyperedge requiring all arguments under compatible substitutions. With only an A available, a provider $A \rightarrow B \rightarrow C$ does not construct C unless its B premise is supplied. Independent edges $A \rightarrow C$ and $B \rightarrow C$ forget this conjunction. Such a graph can be a deliberately loose over-approximation for ranking, but its paths are not realizability witnesses.

Retain a hierarchy of descriptors:

1. The sort, rigid head, bounded function spine, arity, and explicit/implicit/instance binder roles.
2. Repeated type-variable identities and parameter positions; `List A -> Option A` must preserve the relation between its occurrences of `A`. Forgetting it permits spurious `List Nat -> Option Bool` paths.
3. Query-relevant index shapes (zero, successor, shared variable, bounded affine expression, or unknown), and symbolic universe relations, including restrictions on elimination out of `Prop`.
4. Dictionary and proof prerequisites, and resource annotations for any deliberately restricted eliminator use.

For vector append, retain

$$\text{Vec } A \ n \rightarrow \text{Vec } A \ m \rightarrow \text{Vec } A \ (n + m).$$

Forgetting the sum is an over-approximation; incorrectly equating n and m is not. A dependent head can change after assignment: if $F(\text{true}) = \text{Nat}$ and $F(\text{false}) = \text{Bool}$, index $F(?b)$ by a union or top until its discriminator resolves. Do not guess one head, specialize an unresolved universe to `Prop`, or confuse a function argument with a value of its codomain.

Ordinary Lean values can be reused or ignored. A consuming Petri-net representation must explicitly support contraction and weakening; otherwise it can exclude `fun x => (x,x)` or legitimate projections. A search-specific one-use Church-eliminator policy is a separate grammar restriction, not a property of Lean's type system.

Proposition 12.1 (Safe abstract nonreachability). *Fix a concrete grammar and construction bound. Suppose every permitted starting term maps to an abstract initial fact and every concrete construction step maps to an abstract transition, with bounds correspondingly interpreted. If exhaustive abstract reachability finds no abstraction containing the target within that bound, no concrete derivation within the bound constructs it.*

Proof. Induct on a hypothetical concrete derivation. The initial cases have abstract counterparts; each construction rule lifts reachable premises to an abstract result containing the concrete result. Its final target would therefore be abstractly reachable, contradicting the assumption. $\square$

The simulation premise is the substantive obligation. Include constants, locals, partial applications, lambdas, constructors, projections, relevant conversion and casts, dictionary construction, admitted eliminations, recursion templates, and invented helpers whenever the concrete grammar includes them. An abstraction of first-order library calls alone cannot justify exclusion from a higher-order search. An abstract timeout is not established unreachability. A path-length limit of three is a declared bounded fast lane, not evidence that no length-four solution exists.

12.3 A concrete refinement and retrieval loop

Build an index once per environment fingerprint and schema version, including relevant local additions and instance changes. Query it by rigid heads and argument patterns, then perform relaxed hypergraph reachability. Each proposed composition is concretized by instantiating its full telescope in a fresh transaction and asking Lean to check its dependent arguments, universes, and result. On a justified rigid mismatch, refine the abstraction with the distinguishing shared parameter, index relation, universe constraint, dictionary, or proof premise, and retry. Retain the scope of that obstruction: a failed instantiation does not refute every use of its provider.

An incomplete higher-order unifier, unresolved instance, capped normalization, or timeout supplies no general impossibility certificate. Deprioritize or suspend those cases. Rejecting individual paths repeatedly without changing the abstraction is concrete filtering, not type-guided abstraction refinement. Initially use the graph to order a small provider pool and retain a fair wider stream; promote exclusions only for a documented fragment satisfying Proposition 12.1. Record top- k , path limits, and the actual admitted providers in the bound descriptor.

A discrimination tree is output-sensitive: returning many wildcard matches can dominate time regardless of query size. Measure shortlist size, concrete applicability checks, mismatch causes, and downstream search, not just index lookup. A universal type-variable node connecting everything is a reason to retain more correlations, not to discard polymorphic providers silently.

Compatible closed behavioral probes can rank providers by output shape, size change, callback use, prefixes, permutations, or subterms. A component that already meets the observations is still subject to the full original contract and execution policy. A component that does not equal the desired final output can be an essential intermediate step, so standalone mismatch is not a compositional refutation. Compiled probes provide guidance under the chosen evaluation policy; they do not acquire certificate authority merely because the eventual winner is kernel-checked.

12.4 Dictionary-aware obligations and caching

The existing Hoogle+ construction translates a constrained method and a list instance schematically to

$$eqMethod : EqD(A) \rightarrow A \rightarrow A \rightarrow \text{Bool}, \quad eqList : EqD(A) \rightarrow EqD(\text{List } A).$$

Methods consume dictionary arguments and instance components construct them. N7 also notes Hoogle+'s functional-dependency treatment. The transferable representation is explicit prerequisites; Lean's native machinery remains authoritative for concrete terms.

Lean adds multiple and dependent class parameters, output parameters, universes, local instances with priorities, and assignments shared with ordinary arguments. Lazy resolution after sufficient information is available avoids repeated speculative work, but some instances themselves determine missing information. Suspend underdetermined requests with their blockers rather than enforcing a rigid all-types-first phase.

A reuse key includes the entire instantiated class expression, relevant universe and assignment dependencies, environment fingerprint, local-instance context and priorities, transparency, and resolution options. The shorthand (`class`, `type`) is insufficient. Abstract a successful dictionary over its required context, reinstantiate it, and recheck on reuse. Distinguish a checked dictionary, a certified incompatibility in a complete restricted fragment, and ordinary failure or timeout. Only the certified case supports semantic exclusion; even bounded failure on a closed goal is not automatically nonexistence.

Computational dictionaries are data. Two `Add A`, comparison, or equality structures may yield different observations. Preserve actual dictionary identity in candidates, evaluation caches, and

equivalence checks unless an adequate checked equivalence permits replacement. Proof irrelevance concerns proofs of the same proposition, not arbitrary class structures. Class laws used in pruning must be supplied as evidence. This connects retrieval directly to the semantic identity policy in Section 13.2.

12.5 The data-goal adapter and a leakage-controlled corpus

N2, N3, N6, N8, and N9 inspect Lean 4.34.0’s `Lean.LibrarySuggestions.Basic` interface [62]:

```
abbrev Selector := MVarId -> Config -> MetaM (Array Suggestion)
-- Relevant Config fields:
maxSuggestions : Nat
caller         : Option String
filter         : Name -> MetaM Bool
hint           : Option String
```

The goal need not be a proposition. Core supplies the interface without a default suggestion engine; the arbitrary textual hint may be ignored. N1 also points to an indexed-lookup data example `example : Nat := by exact?` [81]. These facts answer interface capability, not example-conditioned retrieval accuracy.

Define a Leant2 request containing the exact target, local telescope and current dependencies, available instances, typed observations, carrier/helper role, allowed-provider policy, and requested count. A backend can use a caller tag such as `leant2.data`, a safe-declaration filter, and a compact hint; a data-aware backend must explicitly implement how it consumes that hint. Combine deterministic type matching, symbol overlap, and conservative reachability before learned relevance. Returned names pass the same native application and trust checks as other providers.

LeanDojo, LeanHammer, and the incoming LeanSearch references establish theorem-retrieval precedents, not measured transfer to program components [20, 21, 67]. N6 and N8 also report cloud-selector cold starts, including a documented 10–20 second session-preparation example in N8 [78]. These are upstream reported conditions, not Leant2 measurements; remote selection cannot be uncharged work in a short query. Keep the default local and model-free; an optional network backend has a declared data-sharing policy, recorded response manifest, and full latency accounting.

Construct a corpus from safe executable definitions in `core/Std`, then an optional ambient-library track. Keep the target type, legitimate context, supplied or generated observations, and reference-body component dependencies as training labels. Such labels are not a unique correct component set: inlining and alternative algorithms can change them. Noncomputable definitions require a declared observation semantics rather than automatic compiled-example generation.

Freeze each synthesis environment before the target where possible. Withhold the target constant, aliases, generated equations, compiler helpers, wrappers, and designated transitive reconstructions revealing its body. A theorem mentioning a hidden target must not reintroduce it as a callable shortcut. The reference implementation may be an external observation oracle but never a synthesis provider; record whether specifications are human supplied, theorem extracted, or generated. Audit import/source-order availability and permitted lemma closure.

Split by semantic family, module or repository, and dependency clusters rather than random near-duplicate declarations. Local exclusions do not eliminate public-corpus model-pretraining contamination. Maintain API-composition/reuse, restricted body-synthesis, provider-given oracle, retrieval-required, helper-invention, and indexed-family tracks. Report recall of *some sufficient component set*, exact applicability, dictionary cost, cold imports/indexing, warm retrieval, and

end-to-end certified success at equal budgets. High reference-dependency recall can still produce an unusably large search.

12.6 Remaining questions

Historical R7.1 now has a simulation obligation and concrete staged design; R7.2 is answered positively by dictionary passing; R7.3’s API question is answered, while its program-relevance and corpus-performance questions remain empirical. The active questions are:

- R7.N1.** Which minimum combination of shared-parameter, dependent-head, index, and universe information makes hypergraph retrieval selective at preserved recall, and can its simulation be checked for a concrete higher-order provider grammar?
- R7.N2.** Can native instance resolution expose reusable dependency-complete dictionary certificates, including output-parameter assignments and local-instance priorities, without unsound negative caching?
- R7.N3.** On leakage-controlled body-synthesis and ambient-library tracks, does example-conditioned retrieval improve certified solve rate after charging import, indexing, dictionary, and proposal costs, relative to type-only retrieval and a provider oracle?

13 R8. Ranking and candidate equivalence

13.1 A declared preference rather than a correctness criterion

The historical implementation ranks by unused outer binders, eliminator occurrences, and raw expression size, then removes equal printed forms. Its grace period limits waiting after the first answer. These are useful implementation policies, but they do not define semantic equivalence, user intention, or optimality. The original examples of twelve type-former variants and indistinguishable-looking candidates expose separate presentation, ranking, and identity problems.

First require the exact type, contract, axiom profile, and requested execution status. Within this feasible set, N1–N9 propose a frozen conditional description-length or Bayesian preference. For example,

$$J_Q(p, \pi) = L_{\mathcal{G}_Q}(p) + \lambda_h L(\text{helpers}(p)) + \lambda_r \widehat{\text{runtime}}(p) + \lambda_\pi \text{proofCost}(\pi) + \lambda_d \text{dependencyCost}(p). \quad (14)$$

The nonnegative weights, grammar, provider library, normalization, and stable tie-breaker are versioned. Count carrier, seed, step, finisher, and invented helper bodies, not merely a short outer name hiding an arbitrary implementation. A library operation can receive a short code as a declared familiarity prior, without claiming its body or runtime is small. Keep program, proof, exported presentation, and execution cost distinguishable. A lexicographic profile or small Pareto frontier is an alternative when the units should not be combined into one scalar.

For a proper grammar distribution, $L(p) = -\log P_{\mathcal{G}_Q}(p)$ has a coding interpretation; an explicit prefix encoding is another defensible definition. Arbitrary scores are not automatically probabilities. N2 adds that grammar probabilities often describe *derivations*, not programs: multiple construction paths can distort program probability. Choose canonical derivations, a minimum-derivation cost, or a correctly aggregated program distribution, and make the implementation match that choice. The completeness lane gives each legal production a finite usable grade and avoids zero-cost cycles that prevent progress. A subprobability model must be declared as such.

For noiseless finite data D , every consistent program has equal likelihood under a hard-consistency model, so

$$-\log P(p \mid T, D, \mathcal{E}) = -\log P(p \mid T, \mathcal{E}) + \text{constant} \quad (p \models D).$$

The preference therefore comes from the prior, not from a theorem that examples identify the intended program. A universal request adds an actual proof constraint; a noisy-example likelihood, if separately offered, cannot weaken an ordinary Lean contract. Estimated runtime is a preference feature, not a complexity theorem. Use deterministic operation counts or freeze measured features for reproducible ranking; new wall-time measurements are not stable lower bounds.

Unused inputs are soft, contract-sensitive features. Constants, projections, length computations, and short-circuiting traversals can legitimately ignore inputs or recursive results. Conversely, $x - x$ mentions x without depending on it extensionally. Occurrence only in a type or erased proof is different from computational use, and a dictionary or callback can hide relevant behavior. Normalized syntactic analysis and probe perturbations are useful approximations, not exact information-flow decision procedures. Label the actual feature, such as an argument not syntactically used, rather than claiming semantic independence without proof.

Learned ranking in PBE, DeepCoder, Probe, and DreamCoder motivates experimentation [9, 25, 10]. N6 cites real-time PBE ranking and user disambiguation studies [70, 69]; N8 cites Singh–Gulwani’s ranking of appropriate programs rather than one prescribed syntax [73]. None establishes a universally preferred ordering for dependent Lean inhabitants. Use blinded pairwise judgments, readability and proof-maintenance assessments, reference rank modulo declared equivalence, accepted-result rank, edit distance to the retained solution, top- k usefulness, and number of extra examples needed for disambiguation. Keep multiple reasonable references for underdetermined tasks; matching one syntax is not the only success.

13.2 Exact identity, proved equality, and observational groups

Use four separately named layers:

1. Typed alpha-canonical structural identity, retaining universes, binder roles, context abstraction, relevant metadata, dictionaries, and dependencies.
2. Native definitional equality under the pinned environment, checked transactionally so comparison cannot mutate branch unknowns. A bounded failure is unknown, not inequality.
3. Explicit propositional or pointwise equality with all needed transport or extensionality evidence under the permitted axiom profile.
4. Agreement of defined values on a named finite set of typed observations.

Use a structural fingerprint for indexing and an exact collision check before discarding a candidate. Pretty printing can hide implicits, dictionaries, universes, coercions, or proof terms; it is not an injective semantic encoding. Proof-insensitive hashing can shortlist comparisons, but is not itself the equality certificate.

Write the last relation as

$$p \approx_D q \iff p \text{ and } q \text{ have equal defined observations at every probe in } D.$$

A stuck result is unknown, not evidence of equal values. Contract inputs often cannot separate accepted candidates because all of them already satisfy those constraints. Add fresh deterministic typed probes, bounded generated values, and canonical positional inputs only within an audited parametric fragment. For dependent functions $(x : X) \rightarrow B(x)$, generate x first and use observers typed in $B(x)$. Values in different fibers require justified type alignment or an appropriate heterogeneous relation. Proof irrelevance may collapse proofs of the same proposition; it does not erase data indices, computational dictionaries, or transport endpoints.

The incoming reports give several versions of one obstruction. Identity and constant zero agree at zero but differ at one. More generally,

$$f(n) = n, \quad g_b(n) = \begin{cases} n & n \leq b, \\ n + 1 & n > b \end{cases}$$

agree on $0, \dots, b$ and disagree at $b + 1$. N8's 0 versus $n(n - 1)$ agrees at zero and one but differs at two. Even element-parametric list functions can agree on all lengths up to m and differ at $m + 1$: compare identity with returning the empty list above the cutoff. Parametricity can reduce the element alphabet for a fixed length; it does not eliminate unbounded lengths.

For display, keep one readable representative and the member count of each *observational group*, retaining alternatives or their regeneration recipes. New observations split groups; parked members also need a route back to certification because an equal-observation alternative may have a cheaper proof. A found distinguishing input explains a difference, not which candidate the user intended. Certify its actual output difference; failure to find one is not equality. For type-valued answers there may be no useful universal ground observer: use conservative typed-shape/elimination summaries instead of collapsing `Empty`, `Unit`, or all unobserved type formers.

Destructive search quotienting requires more than closed terms or current sample agreement. The relation must be a congruence for every permitted future context, preserve interface/proof constraints, and preserve the relevant feasibility or cost bound. A surrounding function application can introduce a new input; recursion or CEGIS can introduce new observations. A restricted straight-line DSL whose operators act pointwise on complete retained observation vectors can meet a suitable congruence theorem. General higher-order dependent Lean terms do not inherit that result. Without it, keep alternatives recoverable. Even proved extensional equality need not identify implementations under a runtime objective: quadratic fold reversal and linear accumulator reversal can deserve separate representatives.

13.3 What a stopping certificate must cover

Proposition 13.1 (Stopping under a frozen objective). *Let $p_\star$ be a certified incumbent under the fixed objective J . Suppose every unexplored or uncertified candidate in the declared grammar is represented by a frontier state s , with a validated lower bound $L(s)$ on the final cost of every completion. If*

$$J(p_\star) \leq \inf_{s \in \mathcal{F}} L(s),$$

no covered candidate is strictly cheaper.

Proof. A hypothetical cheaper candidate has a frontier ancestor by coverage. Its final cost is at least that ancestor's bound, which is at least $J(p_\star)$, a contradiction. $\square$

Coverage includes pending carrier proposals, retrieved providers, suspended branches, proof debt, auxiliary lanes, parked alternatives, and recorded grammar extensions. Equal-cost ties require an appropriate tie policy; a strict frontier inequality excludes unseen equal-cost competitors. For top- k distinct semantic classes, formulate the incumbent threshold, lower bounds, and duplicate policy over those classes; a one-candidate certificate does not establish a diversity optimum.

The bound must measure the *same* objective as publication ranking. Rule depth, CPU work already spent, or stale learned queue probabilities do not bound unused-input penalties, rendered size, or runtime automatically. Learning may reorder exploration while J stays frozen, but cannot silently change either this objective or frontier coverage. A margin-based rule has only the corresponding approximation guarantee once the bound is specified. A first-success grace

period gives responsiveness and *best found*, not optimality or uniqueness. N1–N9’s repeated frontier arguments are consolidated in Proposition 13.1; no existing search receipt is thereby upgraded to an optimality certificate.

13.4 Remaining ranking questions

The existence of a defensible preference objective and the necessary distinction between equality layers answer historical R8.1–R8.2 at the design level. Empirical preference and fragment-specific quotienting remain open:

- R8.N1.** Which frozen ordering over full program/helper cost, readable presentation, and proof or execution cost best predicts blinded user preferences on underdetermined Lean contracts, compared with the existing three-key heuristic?
- R8.N2.** For which concrete typed sublanguage can observational replacement be proved congruent and cost-preserving, including dictionaries, dependent indices, recursive calls, and later counterexamples?
- R8.N3.** Can a practical frontier ledger account for deferred proof, retrieval, proposal, and diversity choices strongly enough to emit a checkable bounded top- k stopping certificate?

14 R9. Learned guidance under a kernel gate

14.1 A measured proposal service

A model may propose carriers, generalizations, helper signatures, components, recursion schemes, or guards. DeepCoder’s component guidance, DreamCoder’s library learning, and LeanDojo/LeanHammer’s proof retrieval are relevant precedents [9, 10, 20, 21]. All N1–N9 agree that these results provide no transferable accuracy percentage for the exact Leant2 task: only a type and examples, a declared carrier/helper grammar and providers, bounded downstream synthesis, and a proof of the original contract. N1’s broader 2026 synthesis comparison [68] is adjacent evidence with different prompts and verifiers; it supplies no such percentage either. No incoming model-check script constitutes a model-accuracy experiment.

Separate the success events:

1. The proposal parses, scopes, elaborates, and meets grammar, universe, and profile restrictions.
2. A solution using it is expressible in the permitted filler grammar, witnessed independently when possible.
3. The fixed downstream synthesizer discovers a compatible complete implementation within its allotted work.
4. The implementation proves the original contract, passes axiom and requested export/execution checks, and improves end-to-end usefulness after all proposal costs are charged.

A sufficient carrier can be difficult to fill; a plausible helper can be uninhabited; sample agreement can overfit. Isomorphic or alternative sufficient carriers need not match the reference type textually. Judge adequacy by an allowed witnessed implementation and declared cost, rather than a carrier name. An oracle supplied body establishes expressibility but not discovery. A finite bounded miss does not show a carrier is mathematically insufficient.

Use paired arms: fixed native grammar; hand-ranked or distinction-guided symbolic proposals; a recorded learned reordering within that same grammar; restricted model proposals; and an explicit expanded-grammar arm. Add carrier-given and motive/helper-given oracle arms to locate remaining body-search and proof bottlenecks. A direct model-generated-program comparison, suggested by N5, belongs in a separate arm because it supplies more than a carrier. Guidance-only

comparisons keep providers, examples, axiom policy, and filler/verifier fixed. Compare equal logical completion work and equal total deadlines, reporting synthesis-only timing separately.

Cross carrier-given/invented with trace-complete/incomplete examples and helper invention. Stratify option and tuple state, continuations, indexed motives, generalized accumulators, ordinary folds, and helpers. Hold out semantic families and close variants, separate textbook tasks from fresh combinations, remove revealing target names/comments/reference carriers, vary operations and encodings, and include ambiguous examples. Renaming alone cannot remove pretraining contamination. Types-and-examples prompts, full specifications, and reference-algorithm prompts are different information conditions.

At proposal counts such as $k = 1, 3, 10$, report validity, witnessed expressibility, discovered bodies, certified solutions, axiom/export outcomes, useful-template recall, time to first certified answer, regressions on previously solved tasks, and residual gap to the oracle. Include initialization, retrieval, generation, parsing, malformed proposals, elaboration, filling, proof, and final replay. Separate cold local, resident local, and remote runs. Keep every failed proposal and rejection reason. One bounded batch at preparation or a named logical epoch is the first experiment; its supposed latency advantage over a call per node remains a hypothesis.

N8 makes uncertainty reporting concrete. For x successes among n independent task units, $\hat{p} = x/n$, a Wilson interval is

$$\frac{\hat{p} + z^2/(2n) \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}.$$

State n and the chosen confidence level. Repeated prompts or related list variants are not independent tasks; aggregate by task or use family-clustered resampling. No numerical outcome is asserted until this experiment runs.

14.2 Restricted proposals and independent authority

Use one proposal interface for deterministic grammars, user sketches, retrieved components, and optional models. Inputs include the frozen query, partial program, and a named decision point. Outputs are bounded carrier ASTs, well-scoped helper telescopes, typed sketches, allowed component references, binder selections, or recursion-scheme choices. Validate scope, node/depth bounds, permitted constants, exact universes, and grammar membership before elaborating in an isolated scratch context. New helper bodies are dependency-checked and kernel-checked before use.

Arbitrary Lean commands, imports, attributes, macros, tactics, foreign code, new axioms, unsafe declarations, and altered trust options are not a harmless term-proposal language. They can execute or change the environment before a final kernel check. The proposer cannot weaken the original target or contract or choose the proof type that defines acceptance. Resource limits protect even rejected proposals. Optional remote submission also needs an explicit policy for private local definitions and contracts. The acceptance and publication boundaries of Section 3 remain independent of the generator.

N8 recommends hardening a gate API to take the frozen query and derive $C(p)$ internally. Its source audit also observes that the engine already constructs the proof type from the frozen contract; the recommendation is a stronger boundary invariant, not evidence that the inspected engine proves an unrelated statement. A checked result must retain exact program, original-contract proof, helper closure, transitive axiom inventory, and any logical-to-executable correspondence.

14.3 Reordering, extension, and replay

A reorder-only proposal must pass membership in the recorded bounded grammar $\mathcal{G}_B$. A new carrier or helper outside it creates an extension $\mathcal{G}_B \subseteq \mathcal{G}'_B$ and changes negative-result and optimality scope. Keeping only model top- k alternatives is not merely reordering unless omitted alternatives remain available. For a fixed finite structurally bounded grammar, exhaustive fair enumeration with conservative pruning sees the same set after reordering. Fixed time or node limits need not. Infinite enumeration additionally needs fairness. Reserve positive deterministic baseline work and never convert zero model probability, absent proposals, or model failure into semantic impossibility.

Admit asynchronous proposals at named logical checkpoints in stable identifier order, rather than letting arrival times silently decide search order. Record absence and cancellation as input events. Freeze or record policy updates and scores; integer-valued scores can simplify cross-platform replay. Canonicalize ephemeral scratch names for identity without changing semantically relevant binder or dependency identity.

Distinguish four audit objects, consolidating N3's four levels and the other reports' three replay notions:

Proof receipt. Rechecks the closed program against the independently retained target, exact contract, universes, and axiom policy.

Realization receipt. Relates the logical term to the exported executable representation and its execution checks.

Search receipt. Replays proposals, grammar epochs, logical work, decisions, deferred alternatives, and scoped pruning evidence; acceptance alone proves no coverage or optimality claim.

Model receipt. Attributes the proposal stream to a particular inference configuration. Regenerating its bytes is stronger than replaying recorded bytes.

A durable content-addressed record contains:

```
frozen query, contract, local telescope, universe and trust policy
repository, toolchain, imported artifacts, local-instance digest
grammar, providers, rank model, scheduler and bounds
decision points, all raw and parsed proposals, rejection reasons
logical admission order, policy epochs, work and stop reason
prune scopes, coverage checkpoints, unresolved proof obligations
closed programs, original-contract proofs, helpers, axiom audits
export correspondence and separately recorded execution status
model/tokenizer identifiers, prompts, selected context, decoding
inference backend, quantization/numerical settings when available
```

A seed and temperature zero do not fix weights, tokenizer, numerical kernels, batching, tie-breaking, inference software, or hardware. Search replay consumes the complete recorded stream without calling the model. A remote service may support transcript replay without promising bit-identical regeneration. Preserve raw bytes alongside canonical parsed forms to audit parser behavior. Digests or signatures protect provenance integrity; proof checking establishes the mathematical claim. Recheck the result in a fresh environment. N7's CoqHammer reconstruction analogy illustrates this separation of discovery from a reusable proof artifact.

The default remains local, core-only, and model-free. Optional guidance may improve discovery while leaving certification independent of model availability. It can still waste time, bias preferences, or starve alternatives; accounting, fixed objectives, grammar tracking, and fair fallback address those risks rather than assuming a kernel gate also proves useful search.

14.4 Remaining guidance questions

Historical R9.1 has no established numerical answer and is replaced by the defined experiment. R9.2's replay architecture and limits are answered; its implementation and empirical costs remain open:

- R9.N1.** Under the paired, family-held-out protocol, which proposal source improves fully certified end-to-end results after charging inference and validation, and how much remaining failure is explained by the carrier/helper oracle?
- R9.N2.** What bounded typed proposal language covers useful carrier, generalization, and helper suggestions while supporting decidable membership in the native grammar and explicit accounting for extensions?
- R9.N3.** Can a model-independent replay reproduce the complete logical admission trace, including asynchronous absence, rejection, and policy-update events, across fresh environments with canonical scratch identities?

Part IV

Plan

15 A dependency-ordered implementation and evaluation plan

15.1 One joint state, several bounded services

The implementation plans in N1–N9 are consolidated here. They retain the native kernel boundary and a reference search order while making dependencies, incomplete results and service costs explicit. Bounded carriers, sketches and native induction can proceed before unrestricted carrier inference; a best-first frontier need not precede every improvement. These are dependency choices, not predictions of speedup.

An explicit node contains the persistent native state, original request ID, typed program and obligations, universe constraints, recursive frames, observation read sets, assignment versions, proof jobs, immutable cost and an expansion cursor. The cursor resumes a rule partway through its alternatives instead of repeating its earlier prefix. Shared immutable terms and closed checked facts differ from branch-dependent residuals. Work charges remain monotone outside rollback. Eviction, spill and fallback policies are resource decisions, never proofs of exhaustion.

```
while logical work remains and the external deadline permits:
  job := chooseEligibleJobAtDeterministicCheckpoint()
  restore(job.nativeState)
  validateReadDependencies(job)
  reply := advanceOneBoundedServiceAction(job)
  match reply:
    alternatives(xs): enqueueWithStableIds(xs)
    blocked(reads): suspendUntilRelevantChange(job, reads)
    refuted(evidence, scope): validateAndRecordExclusion(evidence, scope)
    proofReady(term, proof): checkFrozenQueryAndRecord(term, proof)
    unknown(reason): retainUnresolvedJob(job, reason)
  invalidateAffectedObservations()
  reserveProofAndPublicationWork()
  chargeWorkOutsideRollbackAndCommitTraceEvent()
return checkedAnswers, publicationReceipts, frontierAndUnresolvedJobs
```

This is pseudocode. A service action may contain an expensive Lean primitive, so reduction, elaboration and external inference need resource boundaries. Model absence and cancellation are recorded input events. Proposals state whether they stay within or extend the grammar; old coverage or optimality claims do not silently extend to a larger language.

15.2 Responsibilities, not a required file layout

Service	Responsibility and boundary
Query and receipts	Freeze statement, policy and environment; retain positive and negative certificates and their actual scope.
Dependencies and agenda	Compute constraint closure, readiness and component interfaces; schedule fair resumable work separately from ranking.
Observation normalization	Derive necessary observations with reconstruction evidence; retain original Boolean semantics and unknown outcomes.
Incremental evaluation	Store typed residuals and complete read dependencies; certify supported decisive reductions.
Carriers and motives	Propose carrier/seed/step/finish/invariant tuples and dependency-closed generalizations in a graded grammar.
Recursion realization	Enforce scoped call capabilities and decrease evidence; connect interpreted bodies with checked definitions.
Retrieval and proof jobs	Preserve dictionary prerequisites; schedule contextual proof tiers without turning failure into refutation.
Publication	Independently check export, compilation and execution, including any equivalence bridge to a logical definition.

The example module names in the incoming reports are organizational suggestions, not an additional set of features or files to create verbatim. An implementation may combine responsibilities while preserving their contracts. Detailed algorithms appear in Part III; duplicating them here would create competing definitions of the same state.

15.3 Phases and exit evidence

Every implementation phase preserves the maintained regression suite at both 5 s and 10 s, with exact revisions, provider inventories and outcome meanings. The old six-harness table remains historical. Documentation integration and finite-model reruns do not satisfy an implementation phase's gates.

Phase	Work	Evidence required to leave the phase
P0	Freeze request semantics, result distinctions, receipts, identities and complete cost accounting.	Unrelated proofs, wrong universes and forbidden negative axioms are refused; timeout is unknown; certificate and publication replay are separate.
P1	Observation decomposition, closed caches, local proof adapters, indexed induction and useful primitive/provider coverage.	Cache-disabled comparisons preserve decisions; indexed and missing-lemma cases have checked witnesses; open and rollback cases remain safe.

Phase	Work	Evidence required to leave the phase
P2	Bounded carrier/motive/strengthening grammar, explicit finishers, finite-state exclusion and local invariants.	Supplied-carrier body search and full invention are separate; failed proposals are charged; finite bounds and original contracts are checked.
P3	Resumable states with original DFS first, then fixed-cost/fair policies and conservative dependency components.	Equal completed logical prefixes before policy changes; frontier memory measured; exhaustive small reference cases lose no valid in-bound witness.
P4	Typed partial evaluation, recursive capabilities, checked realization, local proof jobs and bounded helper/angelic refinement.	Supported-fragment arguments and adversarial tests; interpreter agreement; unresolved survivors retained; universal contracts proved where claimed.
P5	Retrieval and scheduling optimization; optional learned proposals and user-facing ranking.	Matched total-cost experiments, including inference, proof and compilation; held-out families; replay without the model; claims match the objective.

P1 and bounded portions of P2 may proceed together. P3 is not a prerequisite for a useful sketch or induction adapter. Stronger pruning in P4 depends on semantic support rather than solved-count improvements alone. Aspirations such as sub-millisecond nodes or solving every stretch case remain hypotheses to measure. E6 propositional countermodels need the contextual bridge in Section 3 before they count as Lean negatives.

15.4 Benchmark dimensions and accounting

Record the exact target, contract, policy, providers, grammar, bounds, reference witness and recursive capabilities. Cross these dimensions where they isolate a cause: type-only, finite examples and universal property; carrier supplied versus invented; helpers supplied, retrieved or invented; complete versus incomplete traces; reference-oracle access versus examples-only input; supplied versus invented motives and invariants; logical certification versus executable publication. Oracle and supplied-artifact arms diagnose the remaining bottleneck and must not be counted as unassisted invention. A reference establishes representability only in its declared grammar. A helper with an accumulator does not inherit a complete specification from examples of the original seed-specific function.

Keep saturated regressions, the thirteen unassisted Church stretch cases, indexed families, recursive compositions and universal contracts as separate tracks. Deduplicate imported benchmarks by semantic task family. Report construction, certified refutation, unification, observation, proof, realization and retrieval work; first candidate, first certified answer and first executable publication times; peak memory; reference-equivalent rank; and exact stop reason. Medians, tails and regressions matter as well as total score. Cold imports/indexes and warm interactive runs are distinct. Compiler/runtime, hardware, threads, cache policy and repeated-run variation belong in reports.

Ablation	Comparison that identifies the effect
Observation/cache	Same expansion order: no cache, closed cache, dependency cache; compare decisions and work.

Ablation	Comparison that identifies the effect
Search policy	Resumable DFS before queue heuristics; fixed versus learned dispatch with identical grammar and frozen ranking.
Carrier discovery	Supplied carrier, enumeration, feature proposals and full invention; body filling and proof budgets matched.
Angelic refinement	Unknown calls remain stuck versus shared typed constraints; count failed guesses and unresolved domains.
Proof construction	Closed post-hoc portfolio versus local invariants and induction-aligned verification conditions.
Retrieval	Curated baseline, ranking-only retrieval and justified bounded filtering; report missed usable components.
Learned proposals	Model-free, recorded/shuffled proposals, fixed-grammar guidance and explicit grammar extension as separate arms.

15.5 Adversarial tests before stronger claims

Each pruning test checks both the survivor’s original gate and preservation of known valid bounded completions. Positive certification alone misses false negative search results. The repeated source checklists become one matrix:

Test	Required behavior
Dependency closure	Shared type/universe variables, local lets, dictionaries or relational observations prevent unsound factorization despite different printed targets.
Rollback and delayed holes	Assignment–rollback–reassignment, identifier reuse and different closure contexts invalidate stale residuals; irrelevant changes permit valid reuse.
Samples and finite state	Missing entries are not differences; pairwise compatibility is not transitive; enumerated values are not a carrier upper bound.
Parametricity/equality	Unlawful BEq, computational dictionaries and uncertified providers do not silently acquire naturality or propositional-equality laws.
Recursive constraints	Equal calls share results; failed angelic choices leave alternatives; a guessed finite domain cannot refute a larger one without coverage.
Dependent realization	Casts remain typed; decrease obligations and interpreter agreement replay; insufficient fuel is unknown.
Program and proof	Proofs may constrain open values; inadequate invariants refute the program–invariant pair only; proof jobs receive fair service.
Original request	Weaker contracts, specialized universes, unrelated proof types and forbidden positive or negative axioms cannot replace the requested result.
Frontier and rank	Evicted/unresolved states remain visible; an incumbent is not optimal while a pending lower bound permits a better answer.
External proposals	Escaped locals, unauthorized dependencies, oversized syntax and environment changes are refused or bounded; raw and parsed proposals are retained.

Test	Required behavior
Publication/replay	Certification and compilation status differ; speed-perturbed workers reproduce logical checkpoints, not necessarily equal timed answer sets.

15.6 Risks and falsifiable decisions

Frontiers exchange repeated work for retained state; measure memory before blaming snapshots alone. Expressive carriers can slow easy cases, so retain a fair plain-grammar path and charge proposal failures. A custom evaluator needs a supported-fragment conservativity argument; closed-program agreement is a test, not that argument. Interpreted behavior guides realization only through its documented connection to the checked definition. Retrieval indexes need environment identities and cold-loading measurements. Deterministic epochs and receipts replace the claim that sorting after parallel deadlines makes discovery deterministic. Continue research phases based on these measurements and obligations, not novelty, shorter terms or plausible model suggestions alone.

16 A focused agenda for outside expertise

The nine incoming responses answer the original 31 questions through conditional theorems, bounded algorithms, counterexamples, native API directions, and experimental protocols. Those questions are no longer the active agenda. Sections 6–14 contain the *33 replacement questions*, identified by R1.N1–R9.N3; this section sets priorities and preparation requirements without repeating their wording. An answer establishing a finite model or a restricted theorem does not settle its unrestricted extension or demonstrate a useful implementation.

Question IDs	Decision the answer should change	Relevant expertise	Concrete artifact to prepare
R1.N1–N4	Component isolation, admissible bounds, policy updates and replay checkpoints	Lean metaprogramming; Aesop and Canonical; constrained search and scheduling	Effect/read-set traces, shared-assignment counterexamples, a finite abstraction and its rule simulation, interrupted logical traces
R2.N1–N5	Carrier certificates, provider parametricity, feature closure, joint schema and invariant invention	Program calculation and automata learning; AutoLifter and constructive fold derivation	A finite table certificate checker, explicit grammar and cost, reversal/prefix-sum examples, paired supplied/invented carrier tasks
R3.N1–N4	Residual semantics, dependency-complete caching, certified evaluation and relational oracles	Smyth and Hazel; incremental computation; Lean reduction and proof-producing evaluation	Typed open-term examples with completions, rollback adversaries, read-set accounting, certificate cost breakdowns

Question IDs	Decision the answer should change	Relevant expertise	Concrete artifact to prepare
R4.N1–N4	Motive recipes, transport normalization, transparency and native adapter state restoration	Lean induction elaboration; dependent inhabitation and higher-order unification	A pinned adapter prototype; vector/dependent-pair examples; changing parameters, compound indices and ill-scoped recipes
R5.N1–N4	Recursive-body simulation, oracle refinement, scheme coverage and executable publication	Lean equation elaboration and compilation; Burst, Trio and recursion-scheme synthesis	A typed body IR, interpretation/realization theorem statement, repeated-call and helper cases, separate compile/export checks
R6.N1–N3	Local invariant grammar, contextual proof effects and fair proof scheduling	Automated induction, refinement synthesis and Lean proof automation	Hand-classified quantified contracts, proof/assignment templates, context-batch traces and starvation tests
R7.N1–N3	A selective conservative abstraction, dictionary certificates and useful retrieval	Hoogler+, SyPet, Lean premise selection and type-class resolution	Concrete AND-hypergraph simulation, dependent-index counterexamples, leakage-controlled corpus split and provider oracle
R8.N1–N3	A preference objective, safe quotienting and bounded top- k certificates	Synthesis ranking, typed semantics and proof-assistant user studies	Blinded rankings, observational collision cases, full deferred-frontier ledger and cost-preserving equality witnesses
R9.N1–N3	Proposal benefit, a bounded typed language and model-independent replay	Learned retrieval and synthesis; experimental design	Family-held-out paired tasks, carrier/helper oracle, frozen model receipt, rejection/admission event replay

Preparation order. First establish the P0 interfaces of Section 15: the frozen query, permitted axioms, completion semantics, publication contract and separate result statuses. Then prepare R3’s smallest useful certified observation fragment, R4’s native adapter and R5’s realization/publication boundary alongside R1’s effect and replay examples. These decide interfaces that would otherwise be expensive to revise. R2’s finite carrier checker and bounded package enumeration can proceed immediately; they do not wait for unrestricted minimal-carrier or automatic invariant invention results.

R6 and R7 should share the same dependency and context identities. Their experiments can proceed once those identities are stable. Freeze R8’s objective before claiming search optimality or evaluating learning; defer R9’s benefit claim until a paired native baseline and the carrier/helper

oracle exist. This order follows implementation dependencies, not a claim that one research topic is intrinsically more important.

What counts as a useful response. For a semantic question, request a theorem with explicit grammar, scope, axiom, termination and completeness hypotheses, or a counterexample to the proposed boundary. For a native interface question, request a minimal reproducible example pinned to the actual Lean version, including failure and rollback. For an empirical question, agree on a decision-changing comparison before collecting results: held-out tasks, matching work and deadline budgets, cold/warm cost, and all certification/publication stages. An external benchmark or submitted Python model is a starting artifact, not a Leant2 acceptance result.

Contact and ownership. The cited projects and the Lean community are possible discussion venues; this agenda records no outreach or endorsement. The artifacts above are preparation tasks unless a repository receipt explicitly establishes that they exist. The engineering and adaptation items also require specialist review where they cross compiler, contextual-refutation or parametricity boundaries. Their classification is not evidence that every implementation detail is already solved.

17 Integration provenance and evidence boundaries

17.1 What was consolidated

The incoming expert responses N1–N9 answered the same 31 questions in the earlier version of this article. They are not the older architecture proposals P1–P9 retained in directories 01–09. Their common explanations now have one maintained home: the semantic boundary in Section 3, the topic-specific answers in Part III, and the roadmap in Section 15. Distinct constructions and assumptions are retained; repeating the same claim in several responses is not additional experimental evidence.

The complete original packages are pinned to repository revision 5c3a53c (the full SHA is recorded in the inventory). The machine-readable `integration/sources.json` identifies all 97 original files by path, Git blob and SHA-256 and records their disposition. Original articles and PDFs remain recoverable from that revision; they are superseded as reading paths by this maintained article. The 28 distinct executable-evidence and provenance files are preserved byte-for-byte under `evidence/N1/` through `evidence/N9/`. Source READMEs and standalone article build instructions are superseded by the maintained reproduction instructions, not treated as additional design recommendations.

ID	Original package/article	Distinct evidence emphasis
N1	<code>new/1/article.tex</code> , split sections	Binary length-modulo-three carrier, conflict clique, eleven model checks.
N2	<code>new/2/Leant2_Expert_Answers/</code>	Four-state two-symbol machine; direct-tail obstruction and finite search accounting.
N3	<code>new/3/leant2-expert-answers/</code>	Unary modulo-three, rollback reuse and approximation-direction countermodels.
N4	<code>new/4/leant2_expert_answers/</code>	Parity and unary carriers, conjunction-aware retrieval, partial-row graph.

ID	Original package/article	Distinct evidence emphasis
N5	<code>new/5/leant2_expert_answers/</code>	Four-state witness, strict request/publication distinctions and negative-policy audit recommendations.
N6	<code>new/6/leant2_expert_answers.tex</code>	Option min/max, Fibonacci pair, observational noncongruence and branch-qualified cache model.
N7	<code>new/7/Leant2_Expert_Questions/</code>	Feature-cover enumeration, shared angelic-call conflict, historical 31-question index.
N8	<code>new/8/leant2_expert_answers/</code>	Minimal sample-consistent carrier that overfits and is refined by a new observation.
N9	<code>new/9/leant2-expert-answers.tex</code>	Maximum-prefix-sum invariant monoid, scheme-dependent state and function-carrier reversal.

These path names are provenance identifiers, not active links to the retired incoming directories. The integration ledgers next to the source inventory map source sections and individual claims to their canonical destinations. They record duplicate treatment, corrections, alternatives and question dispositions. The bibliography consolidates references by work and pinned source rather than preserving multiple aliases for the same publication.

17.2 The historical question inventory

Answered questions are removed from the active expert agenda. The following table preserves their identifiers and scope for provenance without posing them again. Each topic's disposition paragraph records the established answer and the remaining limitation. New questions use the distinct suffix N; they ask about the residual problem, not whether an already identified method exists.

Historical IDs	Themes now covered by integrated answers	Canonical section
R1.Q1–Q4	Coupled goals, admissible cost bounds, learning versus ranking, deadline reproducibility	6
R2.Q1–Q5	Carrier minimality, polymorphic alphabets, fusion/lifting, motive bases, generalization	7
R3.Q1–Q4	Higher-order live evaluation, rollback-safe normalization, decisive evaluation, bounded angelic search	8
R4.Q1–Q4	Motive templates, transports, unfolding, native induction interfaces	9
R5.Q1–Q3	Interpreter agreement, top-down recursion constraints, practical recursion schemes	10
R6.Q1–Q4	Value/proof interaction, sufficient induction fragments, automation, proof scheduling	11
R7.Q1–Q3	Dependent abstraction, type-class prerequisites, relevance for programs	12
R8.Q1–Q2	Ranking objectives and exact versus sampled equivalence	13

Historical IDs	Themes now covered by integrated answers	Canonical section
R9.Q1–Q2	Model-proposal evaluation and replay/auditability	14

Several answers are conditional rather than universal: finite-state minimality does not identify a minimal intended Lean program; a finite motive grammar does not enumerate all dependent motives; a measurement protocol does not supply an unmeasured model-accuracy percentage. Retiring an overbroad or answered question therefore does not assert that all associated research is finished. The active agenda in Section 16 points to the new, more specific questions.

17.3 What the preserved executable checks establish

All nine Python suites were rerun in isolated copies on September 22, 2026. Every process exited zero with empty stderr; every decoded JSON result matched its archived result; all preserved input hashes were unchanged. The raw process streams, executable hash and receipt are under `evidence/rerun-2026-09-22/`. The driver `tools/check_proposal_models.py` recreates the experiment in a fresh directory and never overwrites the historical receipts.

The suites have different mathematical domains and stopping rules. N1’s binary machine search exhausts the one- and two-state spaces and stops on a three-state witness. N2/N7 account for different seed conventions than N5; their table counts must not be interchanged. N3/N6/N9 use unary modulo-three models. N8 stops on a minimal four-state sample witness which fails at length six, then refines it. N9 checks maximum-prefix-sum closure and associativity on a finite invariant domain, including a counterexample to unrestricted identity. Written general proofs and the finite executions remain distinct.

These checks execute neither Lean nor Leant2. They establish no native closure semantics, motive elaboration, engine throughput, model accuracy or universal refutation theorem. The original source manifests and checksum reports also retain their historical meaning: their article page counts and file hashes refer to the source packages, not to this consolidated PDF. The documentation build and visual review are recorded separately.

17.4 Versioned source observations and disagreements

The original performance table belongs to 33cec8b; the responses’ selected source inspections belong to 784664f. Pinned Lean 4.34.0 source advice is distinguished from moving manual pages consulted by the authors, some describing 4.35.0-rc2. A historical source observation is not a fresh implementation test. Reported external corpus counts and cloud timing conditions remain measurements of those external systems.

The consolidation does not turn differing confidence levels into agreement: documented `sauto` options do not prove that its search policy transfers to Lean; native motive construction does not solve unrestricted generalization; equation-based evaluation need not coincide with definitional reduction; library selection is not a complete dependent retrieval procedure. The topic sections retain these qualifications where they affect an algorithm. The six independent obligations of Section 3 govern all resulting claims.

References

- [1] A. Abel and B. Pientka. Higher-order dynamic pattern unification for dependent types and records. *TLCA*, 2011.

- [2] J. Limperg and A. H. From. Aesop: white-box best-first proof search for Lean. *CPP*, 2023.
- [3] F. Lindblad and M. Benke. A tool for automated theorem proving in Agda. *TYPES 2004*, LNCS 3839, 2006.
- [4] J.-P. Bernardy, P. Jansson and K. Claessen. Testing polymorphic properties. *ESOP*, 2010.
- [5] A. Miltner, A. Trejo Nuñez, A. Brendel, S. Chaudhuri, and I. Dillig. Bottom-up Synthesis of Recursive Functional Programs Using Angelic Execution. *POPL*, 2022. doi:10.1145/3498682. <https://arxiv.org/abs/2107.06253>.
- [6] C. Norman and J. Avigad. Canonical for automated theorem proving in Lean. *Interactive Theorem Proving*, LIPIcs, article 14, 2025. <https://doi.org/10.4230/LIPIcs.ITP.2025.14>; <https://arxiv.org/html/2504.06239v2>.
- [7] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia and V. Saraswat. Combinatorial sketching for finite programs. *ASPLOS*, 2006.
- [8] J. Cockx and D. Devriese. Proof-relevant unification: dependent pattern matching with only the axioms of your type theory. *Journal of Functional Programming*, 2018.
- [9] M. Balog, A. Gaunt, M. Brockschmidt, S. Nowozin and D. Tarlow. DeepCoder: learning to write programs. *ICLR*, 2017.
- [10] K. Ellis et al. DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI*, 2021.
- [11] R. Dyckhoff. Contraction-free sequent calculi for intuitionistic logic. *Journal of Symbolic Logic*, 57(3), 1992.
- [12] A. Albarghouthi, S. Gulwani and Z. Kincaid. Recursive program synthesis. *CAV*, 2013.
- [13] R. Alur, A. Radhakrishna and A. Udupa. Scaling enumerative program synthesis via divide and conquer. *TACAS*, 2017.
- [14] J. Frankle, P.-M. Osera, D. Walker and S. Zdancewic. Example-directed synthesis: a type-theoretic interpretation. *POPL*, 2016.
- [15] C. Omar, I. Voysey, R. Chugh and M. Hammer. Live functional programming with typed holes. *POPL*, 2019.
- [16] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala, and N. Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *Proceedings of the ACM on Programming Languages*, 4(POPL), 2020. DOI: 10.1145/3371080. <https://arxiv.org/abs/1911.04091>.
- [17] G. Huet. A unification algorithm for typed λ -calculus. *Theoretical Computer Science*, 1(1), 1975.
- [18] R. Korf. Depth-first iterative-deepening: an optimal admissible tree search. *Artificial Intelligence*, 27(1), 1985.
- [19] J. Feser, S. Chaudhuri and I. Dillig. Synthesizing data structure transformations from input-output examples. *PLDI*, 2015.
- [20] K. Yang et al. LeanDojo: theorem proving with retrieval-augmented language models. *NeurIPS*, 2023.
- [21] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. *Premise Selection for a Lean Hammer*. ICLR 2026; arXiv:2506.07477, version 2, 2026. <https://arxiv.org/html/2506.07477v2>. https://proceedings.iclr.cc/paper_files/paper/2026/file/cf6501108fcd72ee5c47e2151c4e153-Paper-Conference.pdf.
- [22] M. Luby, A. Sinclair and D. Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4), 1993.

- [23] D. Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4), 1991.
- [24] P.-M. Osera and S. Zdancewic. Type-and-example-directed program synthesis. *PLDI*, 2015.
- [25] S. Barke, H. Peleg and N. Polikarpova. Just-in-time learning for bottom-up enumerative synthesis. *OOPSLA*, 2020.
- [26] Ł. Czajka. Practical proof search for Coq by type inhabitation. *IJCAR*, 2020.
- [27] C. S. Lee, N. Jones and A. Ben-Amram. The size-change principle for program termination. *POPL*, 2001.
- [28] J. Lubin, N. Collins, C. Omar, and R. Chugh. Program Sketching with Live Bidirectional Evaluation. *ICFP*, 2020. doi:10.1145/3408991. Extended paper: <https://arxiv.org/abs/1911.00583>. See the formal-core example restriction and the implementation discussion for the higher-order distinction.
- [29] R. Statman. Intuitionistic propositional logic is polynomial-space complete. *Theoretical Computer Science*, 9(1), 1979.
- [30] N. Polikarpova, I. Kuraj and A. Solar-Lezama. Program synthesis from polymorphic refinement types. *PLDI*, 2016.
- [31] Y. Feng, R. Martins, Y. Wang, I. Dillig and T. Reps. Component-based synthesis for complex APIs. *POPL*, 2017.
- [32] J. Gibbons. The third homomorphism theorem. *Journal of Functional Programming*, 6(4), 1996.
- [33] A. Udupa, A. Raghavan, J. Deshmukh, S. Mador-Haim, M. Martin and R. Alur. TRANSIT: specifying protocols with concolic snippets. *PLDI*, 2013.
- [34] Woosuk Lee and Hangyeol Cho. Inductive Synthesis of Structurally Recursive Functional Programs from Non-recursive Expressions. *Proceedings of the ACM on Programming Languages* 7(POPL), article 70, 2023. <https://doi.org/10.1145/3571263>.
- [35] Z. Hu, H. Iwasaki, M. Takeichi and A. Takano. Tupling calculation eliminates multiple data traversals. *ICFP*, 1997.
- [36] P. Wadler. Theorems for free! *FPCA*, 1989.
- [37] J. Gibbons, G. Hutton and T. Altenkirch. When is a function a fold or an unfold? *Electronic Notes in Theoretical Computer Science*, 44(1), 2001.
- [38] C. Norman and J. Avigad. Implementing Dependent Type Theory Inhabitation and Unification. Preprint, 2 March 2026, version 1. <https://arxiv.org/html/2603.01463v1>.
- [39] D. Angluin. Learning Regular Sets from Queries and Counterexamples. *Information and Computation*, 75:87–106, 1987. doi:10.1016/0890-5401(87)90052-6.
- [40] R. Ji, Y. Zhao, Y. Xiong, D. Wang, L. Zhang, and Z. Hu. Decomposition-Based Synthesis for Applying Divide-and-Conquer-Like Algorithmic Paradigms. *ACM Transactions on Programming Languages and Systems*, 2024. doi:10.1145/3648440. Reviewed version: <https://arxiv.org/html/2202.12193v4>. Artifact: <https://github.com/jiry17/AutoLifter>.
- [41] Tjark Weber and James Caldwell. Constructively Characterizing Fold and Unfold. *LOPSTR 2003*, LNCS 3018, 110–127. https://doi.org/10.1007/978-3-540-25938-1_11.
- [42] N. Mulleners, J. Jeuring, and B. Heeren. Example-Based Reasoning about the Realizability of Polymorphic Programs. *ICFP*, 2024. doi:10.1145/3674636. <https://arxiv.org/abs/2406.18304>.
- [43] Shingo Eguchi, Naoki Kobayashi, and Takeshi Tsukada. Automated Synthesis of Functional Programs with Auxiliary Functions. *APLAS*, pp. 223–241, 2018. https://doi.org/10.1007/978-3-030-02768-1_13.

- [44] Q. Hong and A. Aiken. Recursive program synthesis using paramorphisms. *Proceedings of the ACM on Programming Languages* 8 (PLDI), article 151, pp. 102–125, 2024. <https://doi.org/10.1145/3656381>.
- [45] K. Claessen, M. Johansson, D. Rosén, and N. Smallbone. Automating Inductive Proofs Using Theory Exploration. *CADe-24*, LNCS 7898, pp. 392–406, 2013. DOI: 10.1007/978-3-642-38574-2_27. <https://www.cse.chalmers.se/~jomoa/papers/hipspect-cade.pdf>.
- [46] M. Johansson, D. Rosén, N. Smallbone, and K. Claessen. Hipster: Integrating Theory Exploration in a Proof Assistant. *CICM*, 2014. doi:10.1007/978-3-319-08434-3_9. <https://arxiv.org/abs/1405.3426>.
- [47] A. Farzan, D. Lette, and V. Nicolet. Recursion Synthesis with Unrealizability Witnesses. *PLDI*, 2022. doi:10.1145/3519939.3523726. Author copy: <https://www.cs.toronto.edu/~azadeh/resources/papers/pldi22-1.pdf>.
- [48] Umut A. Acar, Matthias Blume, and Jacob Donham. A Consistent Semantics of Self-Adjusting Computation. Preprint, 2011. <https://arxiv.org/abs/1106.0478>.
- [49] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster. Adapton: Composable, Demand-Driven Incremental Computation. *PLDI*, 2014. doi:10.1145/2594291.2594324. Author project and bibliography: <https://matthewhammer.org/adapton/>.
- [50] Niek Mulleners, Johan Jeuring, and Bastiaan Heeren. Program Synthesis Using Example Propagation. Preprint, 2022. <https://arxiv.org/abs/2210.13873>.
- [51] Lean developers. *Lean 4.34.0: Meta tactic induction implementation*. Contains `mkRecursorAppPrefix`, `getMajorTypeIndices`, and `MVarId.induction`. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [52] Lean contributors. *Call-by-value tactic frontend*, Lean 4.34.0. Includes `cbv` and `decide_cbv`. Accessed 21 September 2026. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Elab/Tactic/Cbv.lean>.
- [53] Lean contributors. *The Lean Language Reference: Tactic Reference*. Moving reference manual, consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [54] Lean developers. *Lean 4.34.0: Basic natural-number definitions and theorems*. Contains `Nat.recCompiled`, its `csimp` equality, and the arithmetic definitions discussed in the text. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Init/Data/Nat/Basic.lean>.
- [55] Lean developers. *Lean 4.27.0 release notes*, reduction of natural-number-measure well-founded recursion, 2026. <https://lean-lang.org/doc/reference/latest/releases/v4.27.0/>.
- [56] Lean developers. *Recursive Definitions*, in the Lean Language Reference. Consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [57] M. Carneiro. Lean4Lean: Towards a Verified Typechecker for Lean, in Lean. Preprint, 2024. <https://arxiv.org/abs/2403.14064>.
- [58] Lean4Lean project. *Lean4Lean: Mechanizing the Metatheory of Lean*. WITS 2026 conference presentation record. <https://popl26.sigplan.org/details/wits-2026-papers/11/-Lean4Lean-Mechanizing-the-Metatheory-of-Lean>.
- [59] CoqHammer project. *CoqHammer and sauto documentation*. Consulted September 21, 2026; unfolding options and limits of reconstruction tactics. <https://coqhammer.github.io/>.
- [60] Derek Egolf and Stavros Tripakis. Recursive Program Synthesis from Sketches and Mixed-Quantifier Properties. arXiv:2601.04045, version 2, 22 July 2026. <https://arxiv.org/abs/2601.04045>.
- [61] Lean contributors. *Lean/Elab/Tactic/Induction.lean*. Source at tag v4.34.0; elimination elaboration and `evalFunInduction`. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Elab/Tactic/Induction.lean>.

- [62] Lean contributors. *Lean/LibrarySuggestions/Basic.lean*. Source at tag v4.34.0; Selector, Config, and Suggestion. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/LibrarySuggestions/Basic.lean>.
- [63] M. A. Hammer et al. Incremental computation with names. *Object-Oriented Programming, Systems, Languages, and Applications*, 2015. <https://doi.org/10.1145/2814270.2814305>; <https://arxiv.org/abs/1503.07792>.
- [64] Austin Shen and Yunong Shi. Keep the Proof State Live: Snapshotting for Efficient Tactic Search in Lean 4. arXiv:2605.25556, version 1, 25 May 2026. <https://arxiv.org/abs/2605.25556>.
- [65] Márton Hajdú, Petra Hozzová, Laura Kovács, Johannes Schoisswohl, and Andrei Voronkov. Induction with Generalization in Superposition Reasoning. *CICM*, LNCS 12236, pp. 123–137, 2020. https://doi.org/10.1007/978-3-030-53518-6_8.
- [66] Mikołaj Bojańczyk, Bartek Klin, and Sławomir Lasota. Automata Theory in Nominal Sets. *Logical Methods in Computer Science* 10(3:4), 2014. [https://doi.org/10.2168/LMCS-10\(3:4\)2014](https://doi.org/10.2168/LMCS-10(3:4)2014). <https://arxiv.org/abs/1402.0897>.
- [67] Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving. Preprint, 2026. <https://arxiv.org/abs/2605.13137>.
- [68] Derek Egolf, Yuhao Zhou, and Stavros Tripakis. Can LLMs Perform Synthesis? Preprint, 2026. <https://arxiv.org/abs/2603.20264>.
- [69] M. Mayer et al. User Interaction Models for Disambiguation in Programming by Example. *UIST*, 2015. DOI: 10.1145/2807442.2807459. <https://www.microsoft.com/en-us/research/publication/user-interaction-models-disambiguation-programming-example/>.
- [70] N. Natarajan, D. Simmons, N. Datha, P. Jain, and S. Gulwani. Learning Natural Programs from a Few Examples in Real-Time. *AISTATS*, PMLR 89:1714–1722, 2019. <https://proceedings.mlr.press/v89/natarajan19a.html>.
- [71] S. Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine* 17(3), pp. 73–83, 1996. <https://doi.org/10.1609/aimag.v17i3.1232>.
- [72] A. Bhatia, J. Svegliato, S. B. Nashed, and S. Zilberstein. Tuning the hyperparameters of anytime planning: A metareasoning approach with deep reinforcement learning. *International Conference on Automated Planning and Scheduling*, 2022. <https://doi.org/10.1609/icaps.v32i1.19842>.
- [73] R. Singh and S. Gulwani. Predicting a correct program in programming by example. *Computer Aided Verification*, 2015. Authors’ publication page: <https://www.microsoft.com/en-us/research/publication/predicting-a-correct-program-in-programming-by-example/>.
- [74] Lean developers. *Lean 4.18.0 release notes*, functional-induction support, 2025. <https://lean-lang.org/doc/reference/latest/releases/v4.18.0/>.
- [75] Lean developers. *Lean 4.26.0 release notes*, induction-oriented try? development, 2025. <https://lean-lang.org/doc/reference/latest/releases/v4.26.0/>.
- [76] Lean developers. *Lean Language Reference: Elaboration and Compilation*. Live reference consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/>.
- [77] LeanDojo project. *LeanDojo-v2: A comprehensive library for AI-assisted theorem proving in Lean*. Project page and migration notice, consulted 21 September 2026. <https://leandojo.org/leandojo>.
- [78] Premise-selection project contributors. *premise-selection README*, consulted 21 September 2026; cloud selector interface, caching, and cold-start behavior. Inspected README blob: ba823550133d3b56a756787eab334f8151e03808. <https://github.com/hanwenzhu/premise-selection/blob/main/README.md>.

- [79] Aesop contributors. *Aesop README*, accessed 21 September 2026. Source blob f75ec1f8bebcf6d857fc3954cf91cf99f323a758. <https://github.com/leanprover-community/aesop/blob/master/README.md>.
- [80] Lean contributors. *The Lean Language Reference: Validating Proofs*. Current manual consulted September 21, 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [81] Lean contributors. *Library search implementation*. Lean 4.34.0, `src/Lean/Meta/Tactic/LibrarySearch.lean`, inspected lines 1–80. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/LibrarySearch.lean>.
- [82] Lean developers. *The grind tactic*, in the Lean Language Reference. Consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.